队伍编号	MCB2600304
赛　道	A

面向啤酒酿造化合物的感官分级预测、关键化合物归因与风格条件生成

摘　要

啤酒风味由化合物成分与酵母代谢共同决定。本文围绕「化学成分 ⇔ 感官风味 ⇔ 啤酒风格」这条链条，在含真实缺失、噪声的 250 样本商用啤酒数据上，构建从判别式预测到可解释归因再到条件生成的完整流水线，三问以一个共享的化学到感官模型贯通。

针对问题一，在化学数据存在真实缺失的条件下，采用**原生缺失感知的梯度提升**对 50 个感官维度逐维回归、再经 7 级分位分箱映射为 0-6 整数等级，按风格分层 5 折交叉验证，分箱与标准化参数仅由训练折估计以防泄漏。主模型平均二次加权 Kappa 达 **0.3663**（折外 Spearman 相关 0.4109、预测与真实等级相差不超过 1 级的比例 56.1%），优于随机森林（0.328）与线性基线（0.276），并略优于 KNN 插补对照（0.3652），印证缺失位置本身携带信息；风味（Flavour）组平均 Kappa 0.400 高于香气（Aroma）组 0.322，酒精、苦味等强信号维 Kappa 超 0.77、±1 级准确率近 0.80，而芫荽、丁香等弱信号维接近噪声，可预测性随化学信号强度明显分化。

针对问题二，为贴合「找化合物」的题意，先把 231 个化学特征中的 16 个理化/过程参数与 4 个多化合物汇总列剔除，仅在 **211 个单一化合物构成的候选集** C 上构建**共表达图**（211 节点、564 边、55 个社区、模块度 **0.625**），融合 SHAP、交叉相关与共表达模块代表三类证据为每个维度选出 Top-3 关键化合物并去除共线性冗余。Top-1 的 bootstrap 入选频率平均 **0.991**、跨方法 Jaccard 均值 **0.186**，是 10 万次随机抽取基线（0.0086）的 **21.7 倍**；并依据 8.xlsx 标注的化合物来源对每个维度的 Top-3 逐维做酵母/酒花/麦芽/野生菌群/香料类别判定，酯香、酒精等维度的来源与酿造机理一致。异戊酯香气维 Top-1 命中异戊基乙酸酯，归因与机理一致。

针对问题三，用**条件变分自编码器**以风格为条件生成化学谱，并复用问题一模型打感官画像，在隐空间做风格质心平移实现跨风格迁移。Tripel、Kriek、Lambic 生成谱与同风格真实谱的 KS 距离为 0.357 至 0.408，风格分类器 Top-3 识别率均达 **100%**（随机基线 13.6%），生成画像与真实画像的 Spearman 相关为 0.68–0.93；Pils/Lager→Tripel 迁

移对四个缺陷化合物采用「迁移后既不高于源款原值、也不高于目标风格典型值」的逐样本投影，迁移后缺陷 **0/44 次升高、合规率 100%**，而同一批样本不做投影的纯平移对照有 32/44 次（72.7%）缺陷升高，证明该约束不可或缺。生成 Tripel 的酯香等级 3.90 与真实 3.21 接近，闭环自洽。

三问结论均经交叉验证、bootstrap 与分布距离独立检验，稳健可复现，所用框架可推广到葡萄酒、咖啡等「成分谱 → 感官 → 品类」的分析与设计场景。

关键词：啤酒风味；缺失感知梯度提升；有序分级预测；化合物共表达网络；条件变分自编码器

目录

1 问题重述 **1**

2 问题分析 **2**

2.1 问题一的分析 2

2.2 问题二的分析 3

2.3 问题三的分析 3

3 模型假设 **3**

4 符号说明 **4**

5 数据预处理与探索性分析 **5**

5.1 数据构成与口径统一 5

5.2 风格分布与化学可分性 7

5.3 对数底核实与尺度统一 8

5.4 未检出、真实缺失与异常值 9

5.5 外部数据旁证：Kaggle 公开啤酒集 11

6 问题一：含缺失的多输出有序感官分级预测 **12**

6.1 缺失结构与建模策略选择 12

6.2 标签构建与 0-6 分箱映射 13

6.3 预测模型与求解过程 14

6.4 结果分析 15

7 问题二：化合物共表达图与关键化合物归因 **19**

7.1 共表达图构建与模块结构 19

7.2 多证据融合 Top-3 归因 23

7.3 稳定性量化 23

7.4 化学合理性论证 27

8 问题三：风格条件生成与跨风格迁移 **29**

8.1 条件变分自编码器 29

8.2 跨风格迁移与缺陷约束 30

8.3 生成质量与结果分析 31

9 模型检验与灵敏度分析 **34**

9.1 验证方案与防泄漏 34

9.2 关键参数的灵敏度 35

9.3 鲁棒性 35

10 模型评价与推广 **35**

10.1 优点 35

10.2 不足与改进 36

10.3 创新点 36

10.4 推广 37

参考文献 **38**

A 软件环境与运行命令 **40**

B 支撑材料文件清单 **40**

C 核心源程序 **41**

1 问题重述

啤酒以麦芽水为原料、加酒花并经酵母发酵而成，其风味高度依赖酵母代谢与化合物成分的协同作用。要定量刻画「化学成分如何决定一款啤酒喝起来是什么味道」，需要把实验室测得的数百种风味化合物与品评小组给出的感官强度联系起来。近年来，机器学习方法已被用于从化学成分预测啤酒及其他发酵饮品的风味特征 [1-3]，为本文提供了方法论基础。本文使用一套商用啤酒风味代谢物实测数据集：对 250 个啤酒样本测量了酒精、异 α-酸、pH、糖浓度等酿造参数，以及 200 余种风味化合物的丰度（涵盖酒花萜类、酵母代谢酯醇、麦芽与野生菌群化合物及陈化类），并由品评小组给出约 50 个感官维度的评分。数据来自真实测量，含噪声、真实缺失值与异常值。围绕「化学成分 $\Leftrightarrow$ 感官风味 $\Leftrightarrow$ 啤酒风格」这条链条，本文研究三个递进的问题。

问题一：在化学数据存在真实缺失的条件下，以化合物与理化丰度为自变量，预测品评卡定义的 Aroma（香气）与 Flavour（风味）两套、合计约 40 个感官维度在 0-6 分制上的分级评分。任务对象是每一款啤酒样本，待求量是各感官维度的离散等级，关键边界是自变量含缺失而不能简单删除样本，评价目标是在缺失干扰下给出稳定可信的分级。本问为后两问提供「化学如何映射到感官」的基础模型。

问题二：利用化合物相关数据构建化合物共表达图，结合化学特征，给出每一个感官维度的 Top-3 关键因素化合物，并对所得结果做稳定性与化学合理性论证。稳定性按题面指定的两种口径量化——bootstrap 重采样下化合物进入 Top-3 的入选频率，以及不同建模方法间 Top-3 集合的 Jaccard 系数。本问依赖问题一的预测模型做可解释归因，其产出的关键化合物清单又为问题三识别风格判别成分提供先验。

问题三：以风格标签 `tasting_category_fine` 为条件学习一个风格条件生成模型，在给定风格（如 Tripel、Kriek、Lambic）时生成合理的化合物谱并给出对应感官画像；同时完成跨风格迁移，即把 Pils/Lager 的化合物谱迁移为 Tripel 风格，并保持缺陷化合物（DMS、Diacetyl、4VG、trans-2-nonenal、Stale hops）不恶化。该问反向利用问题一（对生成谱打感官画像）与问题二（风格关键化合物），是三问的综合应用。

三个问题沿「数据预处理 $\rightarrow$ 化学到感官的判别式预测 $\rightarrow$ 可解释归因 $\rightarrow$ 风格到化学的条件生成与迁移」逐级推进，构成一条从理解到生成的完整流水线，其整体求解思路与依赖关系概括如下。

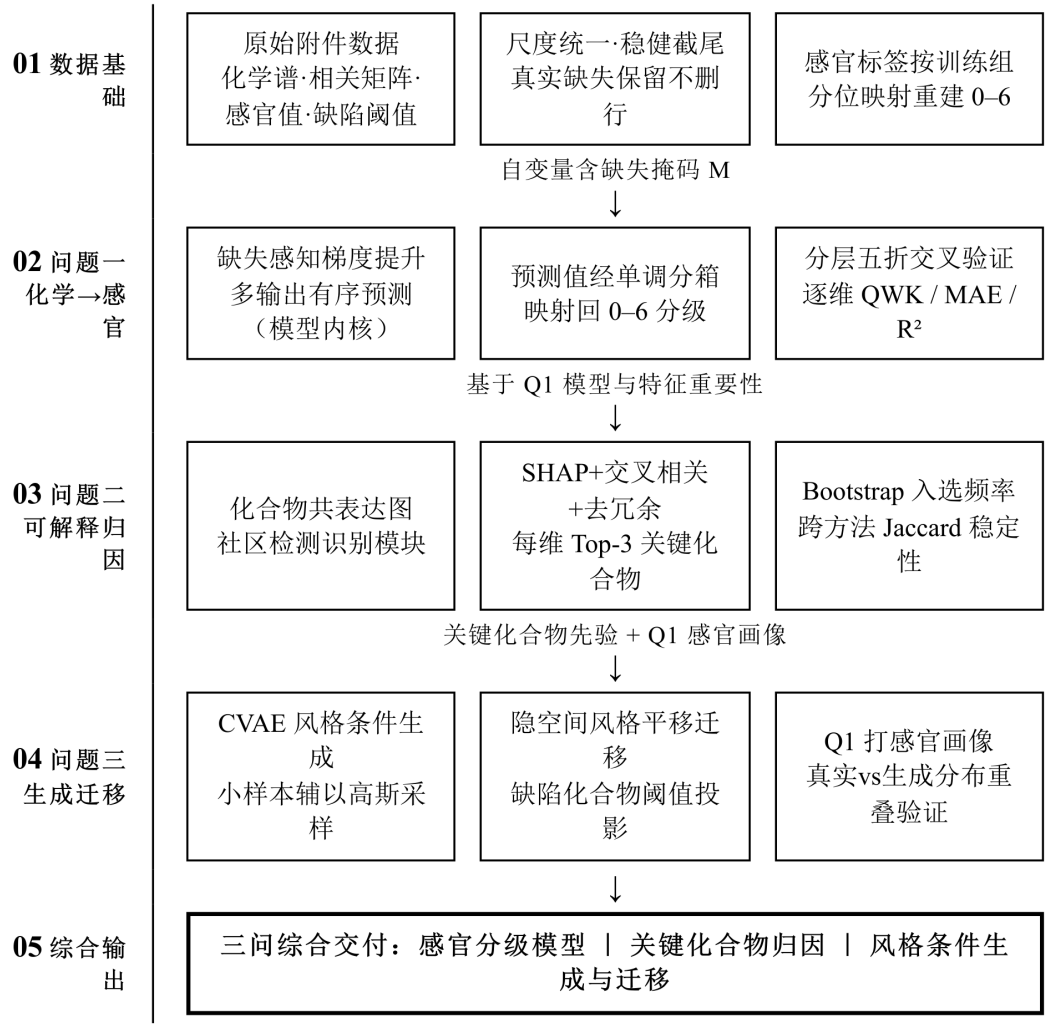


图 1　整体技术路线图

图 1 给出的技术路线中，问题一的预测模型处在枢纽位置：它既是问题二 SHAP 归因的解释对象，又是问题三生成谱感官画像的打分器。正是这一共享底座让三问环环相扣，也决定了求解必须从含缺失的化学到感官预测开始。

2　问题分析

2.1　问题一的分析

问题一本质是一个带缺失数据的多输出有序预测任务，核心难点集中在三处。其一，化学特征高维（231 列）且含真实缺失，其中 63 个样本、合计 73 个缺失值集中在少数几列，直接删除含缺失的样本会损失四分之一的数据并引入选择偏差，因此必须在缺失条件下建模。其二，品评小组提供的感官标签是经过标准化、中心化的连续值而非原始 0-6 整数，要交付题面要求的 0-6 分级，需要建立连续值到 7 级离散刻度的单调映射。其三，约 40 个感官维度彼此相关性强，宜共享同一套化学特征表示以提升稳健性。

据此，本文选择原生支持缺失的梯度提升树作为主模型，对每个维度做回归后再分箱映射回 0-6 整数，并用分层交叉验证逐维评估二次加权 Kappa、平均绝对误差与决定系数。相较先插补再建模的常规路线，原生缺失处理避免了插补偏差，其输出的特征重要性又可直接服务于问题二的归因。

2.2 问题二的分析

问题二是一个在共线性干扰下仍要既准、又稳、还有化学道理的可解释归因任务。许多化合物在相关矩阵里高度相关、成簇出现，单一重要性指标容易把同一化学模块里可以互相替代的化合物反复选入，造成 Top-3 虚假冗余。因此本文先用化合物共表达图揭示模块结构，再在模块意识下融合三类证据做归因：问题一模型给出的 SHAP 预测贡献、化学与感官的交叉相关，以及共表达模块的去冗余加权。稳定性严格按题面两法量化，bootstrap 入选频率刻画单方法内部的稳健性，跨方法 Jaccard 刻画不同证据之间的一致性。化学合理性则逐条对照已知酿造机理加以论证。需要强调，强信号维度的 Top-3 应当稳定、弱信号维度不稳定，这是数据本身的真实现象，本文如实报告而不强行制造稳定。

2.3 问题三的分析

问题三是条件生成加可控迁移任务，难点在于小样本（部分风格仅有 3 至 11 个样本）条件下既要保证生成谱的化学合理性与风格可区分性，又要在迁移时守住「缺陷不恶化」这一硬约束。本文用条件变分自编码器把化合物谱编码进低维隐空间、以风格标签为条件解码，兼顾生成多样性与风格可控；对极小样本风格辅以按风格的分布采样作为对照与兜底。生成谱的感官画像直接复用问题一的化学到感官模型，形成「生成化学谱 $\rightarrow$ 预测感官」的闭环。迁移通过在隐空间做源风格到目标风格的方向平移实现，并对缺陷化合物施加阈值与源值的双重投影，一旦超限即拉回安全区，确保迁移后缺陷不劣化。生成质量用真实谱与生成谱的分布距离及降维重叠度评估。

3 模型假设

为聚焦核心建模、简化非关键环节，本文做出以下假设。

（1）以标准化连续感官值为问题一的训练目标，再按训练折内分布单调映射回 0-6 整数等级。其合理性在于原始 0-6 品评卡的汇总表为全 0 模板、数值不可用，而标准化连续值是唯一完整可用的全样本感官标签，分位分箱单调可逆，既产出 0-6 等级又不丢失信息。

（2）约 40 个感官维度以香气与风味两套为核心，同时输出全部 50 个维度。题面明说「Aroma 与 Flavour 两套」，而「约 40」介于仅两套的 30 维与含口感、缺陷、总体的 50 维之间，全部给出可覆盖任一解读。

（3）化学特征的数值尺度在 8.xlsx 的校准浓度列上已逐列核实为**混合尺度**：可校准的 70 列中 37 列严格为常用对数（$\log_{10}$，中位残差为 0，连 pH 也取 $\log_{10}$）、33 列严格为原值（酒花萜烯、醛、酚类等，如芳樟醇、香叶醇、4-乙烯基愈创木酚），二者相关系数均为 1.0，无自然对数列；其余 161 列无校准值，按列分布与理化列归类、并在文中如实注明不确定性。建模统一在该原生数值空间内进行而不做反变换。由于树模型对单调变换不敏感，对数底核实只用于统一认知与叙述性换算，不改变问题一的分裂结果，问题一与问题三的生成和比较也在同一空间内自洽（详见 5.3 节）。

（4）缺陷「不恶化」在原生数值空间判定，取严口径：迁移后缺陷值必须同时不高于源款原值 $x^{def}_{k,before}$ 与目标风格典型值 m^t_k，即上界取二者的较小者 $\min\left(x^{def}_{k,before}, m^t_k\right)$，$m^t_k$ 同时充当第二道绝对兜底。因 8.xlsx 阈值为原始浓度单位、与建模空间不可直接比较，本文以目标风格训练样本的缺陷分位 m^t_k 作为可在同一空间比较的绝对阈值。该口径保证迁移后缺陷相对源款绝不升高；8.2 节的无约束纯平移对照进一步量化了这一约束的必要性。

（5）各样本近似独立同分布，风格标签为问题三唯一的条件变量。附件为商用啤酒截面数据、无时间序列结构，故交叉验证采用按风格分层的随机折。

4 符号说明

下表列出全文反复使用的核心符号，其余一次性派生量在首次出现处就地说明。

核心符号说明

符号	含义	单位/取值
N	啤酒样本数	250
P	化合物与理化特征数	231
J	感官维度数	50
S	风格类别数	22
X	化学谱矩阵（含缺失，对数/标准化混合）	$\mathbb{R}^{N\times P}$
M	缺失掩码（1 为缺失）	$\{0,1\}^{N\times P}$
x_i	第 i 款啤酒化学谱向量	—
Y	标准化感官值矩阵（训练标签）	$\mathbb{R}^{N\times J}$
$\tilde{y}_{ij}$	分箱后 0-6 等级（交付目标）	$\{0,\ldots,6\}$
$\hat{y}_{ij}$	第 i 样本第 j 维预测等级	$\{0,\ldots,6\}$
$g_j(\cdot)$	维度 j 的连续值到 0-6 单调分箱映射	—
R	化合物相关矩阵	$[-1,1]^{P\times P}$
$G=(V,E)$	化合物共表达图	—
w_{pq}	化合物 p,q 的边权	$[-1,1]$

续下页

核心符号说明（续）

符号	含义	单位/取值
ρ	建图相关阈值	$[0,1]$
ϕ_{jp}	化合物 p 对维度 j 的关键度	实数
f_{jp}	bootstrap 下 p 进 Top-3 的频率	$[0,1]$
z_i	CVAE 隐编码	$\mathbb{R}^d$
s	风格 one-hot 条件向量	$\{0,1\}^S$
$\Delta_{s\to t}$	隐空间风格迁移向量	$\mathbb{R}^d$
x_k^{def}	第 k 个缺陷化合物浓度	原生空间

5 数据预处理与探索性分析

5.1 数据构成与口径统一

本文主分析**只使用主办方提供的 8 个附件**；另引入一个公开啤酒数据集作为**外部旁证**（见 5.5 节），它既不与本数据逐行合并，也不进入任何模型，仅用于校验风格规格的行业常识。建模所用的核心数据包含四部分：化合物与理化丰度矩阵（250 样本 × 231 特征），化合物相关矩阵（231 × 231），标准化感官值矩阵（250 × 50，作为问题一训练标签），以及化学与感官的交叉相关矩阵（44 化合物 × 50 感官，作为归因的线性证据）。8 个附件的实测规模与用途如下表，行列数均以本文程序实际读取为准。

表 2　八个附件的实测概况与用途

附件	内容	实测规模（行 × 列 / sheet）	本文用途
1.csv	化学组成谱（含主键与风格）	250 × 234（3 元列 + 231 化学特征）	Q1 特征矩阵、Q2 建图、Q3 生成
2.csv	化合物相关系数矩阵（标称 Pearson）	231 × 232（首列为化合物名）	Q2 共表达边权（口径见 7.1）
3.xlsx	12 张空白评分卡 + 1 个汇总表	13 sheet（beer_1…beer_12 各 100×33；summary 全 0）	Q1 的 0–6 分填分模板
4.csv	品评小组标准化分	250 × 53（3 元列 + 50 感官维）	Q1 训练标签
5.csv	感官维度之间相关	50 × 51（首列为维名）	感官分组与冗余参考
6.csv	化学-感官交叉相关	44 化学行 × 50 感官列（44×51）	Q2 归因的线性证据
7.csv	论坛/评论文本属性	250 × 189（其中 3 列全缺失）	文本数据，三问均不使用
8.xlsx	综合参考表	7 个 sheet（见下注）	仅作旁证：阈值、校准浓度、来源、MDI

> 注：8.xlsx 七个 sheet 分别为 Sensory thresholds（261×11，感官阈值与化合物来源）、Sensory names（101×8，感官维类别）、GBM importance MDI（231×2，主办方模型的不纯度重要性）、GBM SHAP Values（500×236）、Calibrated chemical dataset（250×73，含 3 元列与 70 列校准绝对浓度）、Beer Prices（250×11）、Sheet2（空表）。其中 MDI 与 SHAP 仅作问题二的旁证，绝不进入问题一特征，以防「用答案预测答案」的泄漏。

题面文字与附件实测存在若干出入，本文一律以实测为准并做如下处理。

表 3　题面表述与附件实测的差异核对

题面表述	附件实测	处理方式
「226 种啤酒」	1.csv 实为 250 个样本（4.csv 同为 250 行）	以实测 $N = 250$ 为准
附件 2 为 Pearson 相关	标称 Pearson；本文从 1.csv 重算 Pearson 与 Spearman，26565 对化合物相关系数的中位绝对差仅 0.062	以附件 2 数值作边权，并用 Spearman 重算核对，口径差异不改变模块结论
3.xlsx 为评分卡	12 张空白评分卡（100×33 模板）+ 1 个全 0 的 summary	原始 0-6 分数不可用，仅作填分模板，训练标签改用 4.csv 标准化连续值
化学-感官交叉相关	6.csv 实为 44 个化学行 × 50 个感官列	按实测维度使用，不臆测其行列数
7.csv 论坛文本	250×189，其中 palate_acetaldehyde、palate_acetate、palate_dentist 三列完全缺失	文本数据不进入三问建模，全缺失列仅登记
8.xlsx 参考表	实为 7 个 sheet（含校准浓度、价格等）	逐 sheet 登记用途，仅作旁证、不进训练

在 231 个化学特征中，本文按化学含义把它们分为三类：**理化/过程参数** 16 列（如酒精度、色度、pH、苦度、二氧化碳压力、蛋白、甘油、未发酵糖等），**多化合物汇总列** 4 列（总酯 esters_sum、酒花香气总量 aroma_hops_sum、总酸 acids_sum、含硫总量 sulfur_sum，均为多个化合物的加总），以及 **单一化合物集** C 211 列（$231 - 16 - 4$）。问题一以全部 231 个特征为自变量（理化指标对预测感官同样有用，不应删除）；问题二为严格回应「找关键化合物」的题意，仅在 211 个单一化合物构成的集合 C 内建图与归因，避免把酒精度、苦度这类理化指标或「总酯」这类加总列误当作某个具体化合物。

三类数据的数值口径并不一致，必须先对齐认知再建模。感官标签列为中心化的标准化连续值，范围约 -2.15 至 6.17、均值接近 0，并非原始 0-6 整数；化学特征的对数/原值尺度经 5.3 节核实为混合尺度。由于本文主模型为树模型、对单调变换不敏感，建模统一在该原生数值空间内进行而不做反变换，既规避了对数尺度换算的风险，又保证问题三生成与缺陷判定在同一空间内可比。

5.2　风格分布与化学可分性

数据集涵盖 22 个精细风格类别，样本量分布高度不均：Blond 有 31 个、Tripel 有 28 个样本，而 Faro 仅 3 个、Brut 仅 4 个。这一长尾结构直接决定了问题三对极小样本风格必须设置分布采样兜底。在进入建模前，本文先用主成分分析观察化学谱是否天然

按风格分离，以判断「化学成分能否区分风格」这一前提是否成立。

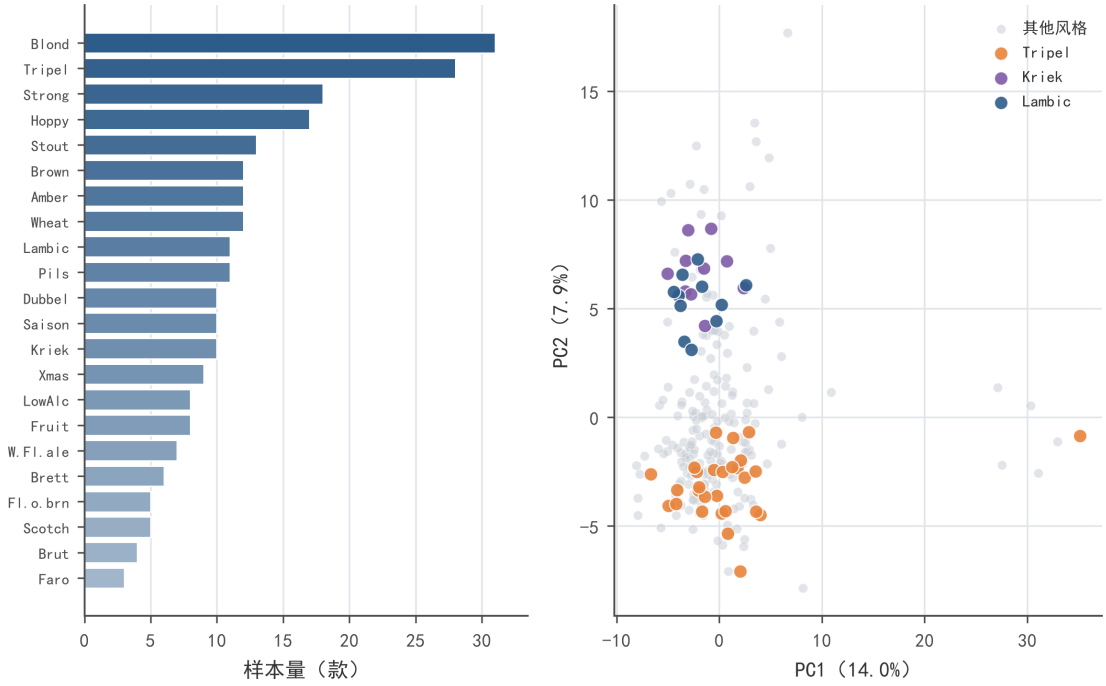


图 2　风格样本量与化学谱主成分分离

从图 2 左侧的样本量分布可见长尾特征明显，而右侧化学谱前两个主成分的散点中，Lambic、Kriek 等酸味野发酵风格与 Tripel、Blond 等常规发酵风格在主成分平面上形成可辨识的区域偏移。这说明化学成分确实携带了区分风格的信息，为问题三「以风格为条件生成化学谱」提供了数据层面的可行性依据；同时各风格簇之间存在重叠，也预示生成与迁移任务并非平凡，需要隐空间模型而非简单的类别均值替代。

5.3　对数底核实与尺度统一

化合物丰度究竟取对数还是原值，直接影响浓度叙述。8.xlsx 的 Calibrated chemical dataset 提供了 70 个化合物的校准绝对浓度 c_{ik}，本文据此逐列反推 1.csv 取值 x_{ik} 的尺度：对每个校准列，在 $c_{ik} > 0$ 的样本上分别计算 $\mathrm{median}_i|x_{ik} - \log_{10} c_{ik}|$、$\mathrm{median}_i|x_{ik} - c_{ik}|$ 与 $\mathrm{median}_i|x_{ik} - \ln c_{ik}|$，若第一个小于 10^{-6} 判为常用对数列、第二个小于 10^{-6} 判为原值列，第三个用于排除自然对数。核实结果为：70 个校准列中 **37 列严格为常用对数、33 列严格为原值**，二者的中位残差均为 0、与对应尺度的相关系数均为 1.0，且不存在自然对数列；值得注意的是连 pH 也取常用对数（$0.625 = \log_{10} 4.21$）。常用对数列包含全部 16 个理化指标与 21 个酵母酯、醇、硫化合物；原值列则以酒花萜烯、醛、酚类为主，如芳樟醇、香叶醇、香茅醇、丁香酚、糠醛、反式-2-壬烯醛、双乙酰。其余 161 列没有校准绝对浓度，本文按列分布与已知理化列归类，并对其尺度明确标注「未经校准」，不编造校准残差。

未检出的半最小检出限只依赖本列：取 $\epsilon_k = \frac{1}{2}\min_{i:x_{ik}>0} x_{ik}$；对右偏的原值列可按

$\log_{10}(\max(c,0)+\epsilon_k)$ 理解其量级。核实过程与残差见图 3。

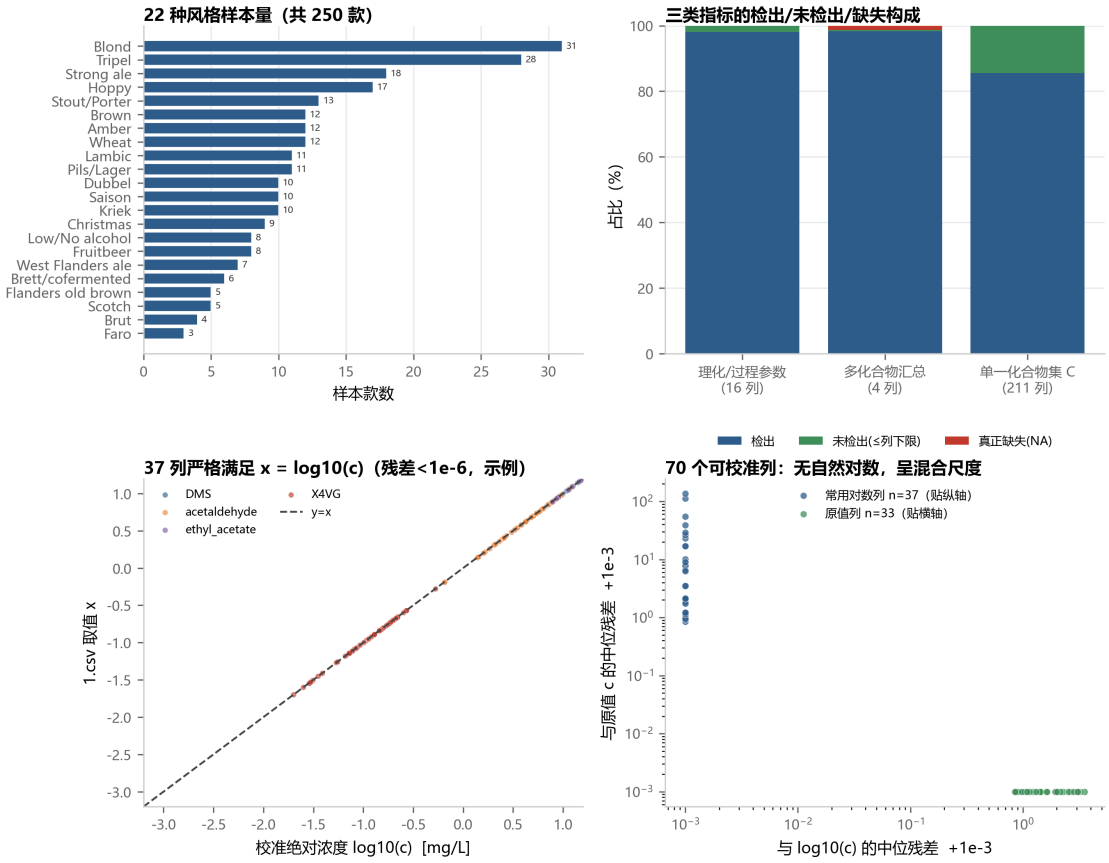


图 3　对数底核实（风格样本量、三类特征检出结构、校准散点与列残差）

图 3(a) 为风格样本量，(b) 比较理化/汇总/化合物三类特征的检出、未检出与缺失占比，(c) 为 1.csv 取值对 $\log_{10}$ 校准浓度的散点（严格对数列恰落在 $y=x$ 上、原值列偏离），(d) 为各校准列的中位残差。尽管尺度是混合的，本文仍保留「树模型对单调变换不敏感、建模留在原生空间」的处理——对数底核实只用于统一认知与浓度的叙述性换算，不改变任何特征的分裂取值，因此与 6.1、7.1 的建模口径并不矛盾。

5.4　未检出、真实缺失与异常值

化学谱里大量「低值」并非缺失，而是**未检出**（浓度低于仪器检出限），它本身携带「该化合物极少」的信息，必须与真正的缺失严格区分。本文定义未检出为取值不超过本列下限 $\ell_k=\min_i x_{ik}$；真正缺失则是 1.csv 中确实为空的单元格，全表共 **73 个 NA、分布在 9 列、涉及 63 个样本**：乙酸戊酯（amyl.acetate）一列 27 个（列缺失率 10.8%）、酒花香气合计列 13 个（5.2%）、甲基吡嗪 12 个（4.8%）、双乙酰 9 个（3.6%）、3-甲基丁醛 7 个（2.8%），烟酸乙酯 2 个，苯乙醛、糠醛、异丁酸乙酯各 1 个。

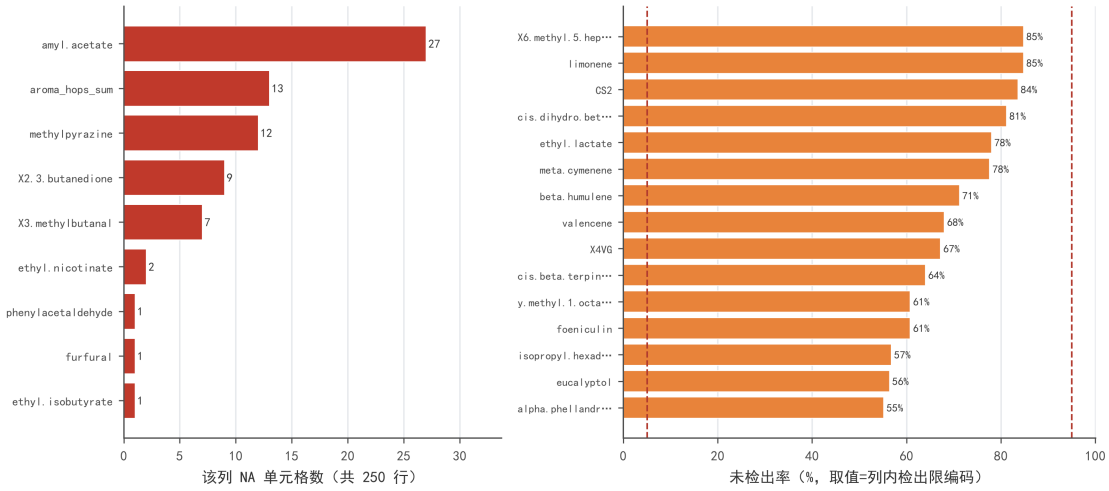


图 4　三类特征的检出/未检出/缺失结构与未检出指示列范围

从图 4 可见三类特征的未检出比例差异极大：211 个单一化合物平均有 14.4% 的单元格处于未检出（检出率 0.855），而 16 个理化列几乎全部检出（未检出仅 2.0%）、4 个汇总列未检出 0.4%。这说明化合物丰度是稀疏的，未检出是主流信号而非异常。对未检出比例落在 5%–95% 之间的列（共 **102 列**），本文增设二值指示变量 $\delta_{ik} = \mathbb{1}[x_{ik} \le \ell_k]$ 显式标记「未检出」事件；比例过低（恒为 0）或过高（恒为 1）的列不建指示变量，以免引入常数噪声列。

真正的缺失（73 个 NA）才需要插补，本文采用 $K = 5$ 的近邻插补，近邻距离在标准化后的特征上计算，并严格在训练折内拟合、再施加到验证折。完整的预处理管道顺序为「标准化 → KNN 近邻插补 → 未检出指示 → 再标准化」，其中标准化统计量、近邻关系、指示变量入选列与分箱切点都只在训练折内学习、再施加到验证折，防止验证折信息泄漏。

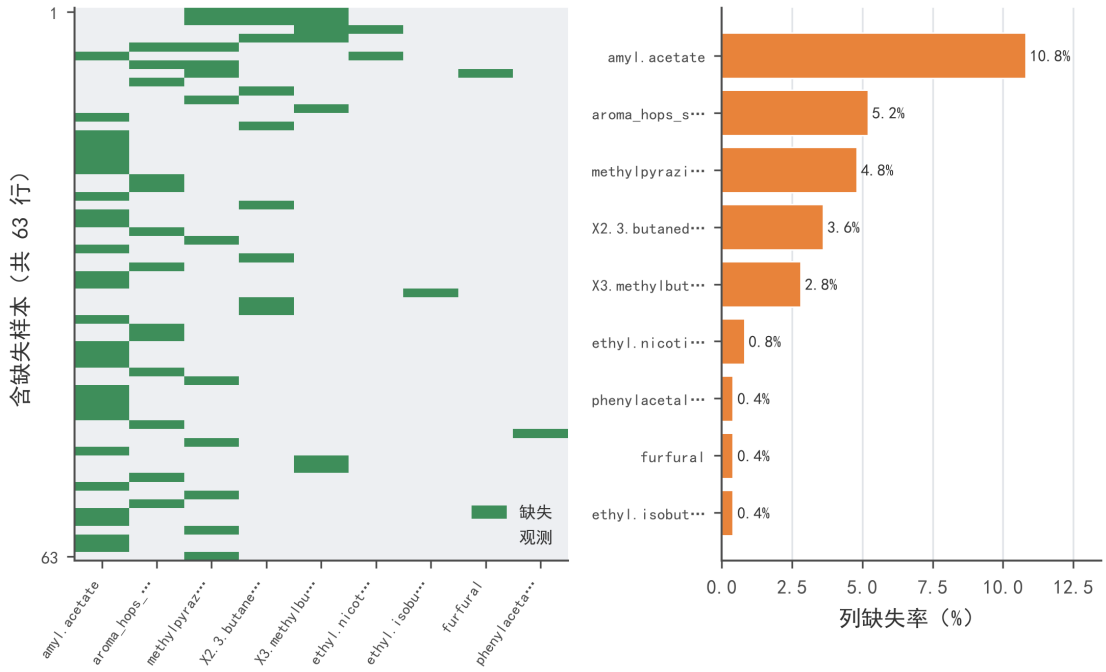


图 5　含缺失样本的缺失位置矩阵与列缺失率

图 5 给出 63 个含缺失样本的缺失位置矩阵与列缺失率条形，印证缺失的列向集中性。对异常值，本文不删除任何样本：经对数/标准化处理后极端值不再主导距离，梯度提升树按秩分裂、二次加权 Kappa 也只依赖等级次序，对单调变换与极值天然稳健；问题三的生成谱则在每列的观测范围 $[\ell_k, u_k]$ 内截断，从源头杜绝越界的异常浓度。

5.5　外部数据旁证：Kaggle 公开啤酒集

为校验本文风格规格是否符合行业常识，引入 Kaggle 公开数据集 *Top Beer Information*[18]（Stephen Polozoff，5558 款、112 个风格，含 ABV、IBU 区间与网友平均评分）作为外部旁证。它与本数据是互补而非替代关系：Kaggle 是大众风格的「网友评分 + 规格」大样本库，本文是 22 个比利时/欧洲小众风格的「实验室化学谱 + 单一小组标准化分」精标小样本。两者没有共同主键，因此不逐行合并、不进入任何模型，仅作宏观对照。

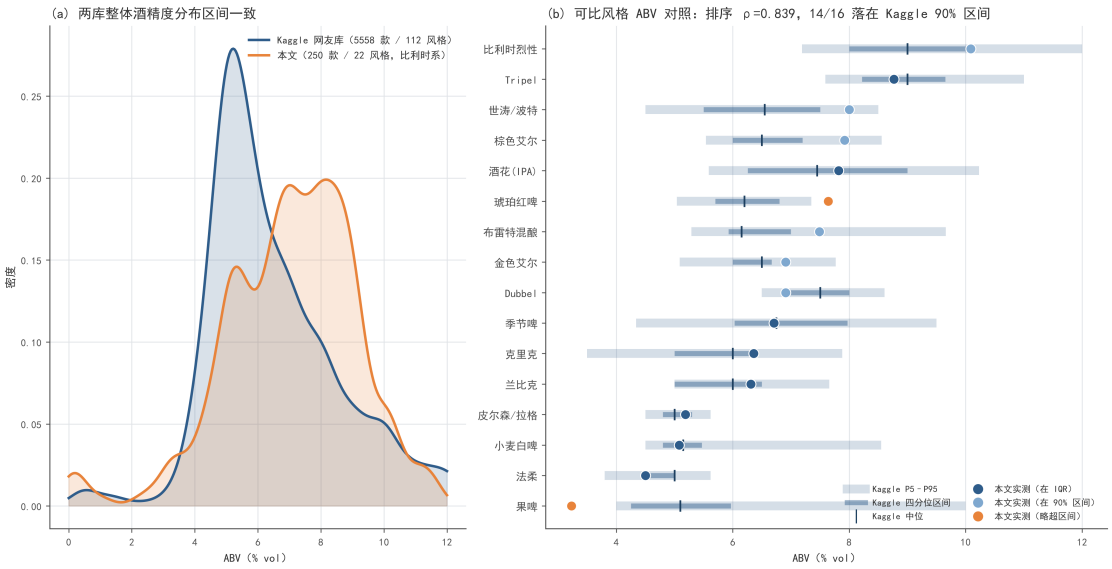


图 6　Kaggle 公开啤酒集与本文的风格规格宏观对照

图 6(a) 显示两库整体酒精度分布区间一致（都集中在 0–12% vol，本文样本因偏比利时修道院风格而整体偏强，峰值落在 6.5%–8%）。在 16 个能明确对应的风格上，把本文实测酒精度（对 $\log_{10}$ 列取 10^{x} 还原）与 Kaggle 同风格区间对照（图 6(b)）：**14/16 落在 Kaggle 的 P5–P95 区间内，跨风格的酒精度排序 Spearman 相关达 0.84**；苦度排序相关也达 0.86——由于 Kaggle 的 Min/Max IBU 是风格指南标称区间（每风格退化为单一值）、并非逐款实测，而本文是仪器实测苦度，口径不同，故苦度只比较排序、不报告命中率。果啤、法柔、兰比克、小麦等低强度低苦风格与 Tripel、比利时烈性等高强度风格的相对格局完全一致。此外，Kaggle 的网友评分由大量评论聚合（5 分制中位 3.82，极差 1.27–4.83），比单一小组、每款数人的评分更平滑；本文小组打分噪声更大，这也解释了问题一中弱信号维度可预测性偏低的现象。需要再次强调，**主分析仅使用所给的 8 个附件，Kaggle 数据仅作外部旁证、不进入建模。**

6　问题一：含缺失的多输出有序感官分级预测

6.1　缺失结构与建模策略选择

问题一要解决的第一道关卡是缺失数据。如 5.4 节所述（图 5），73 个真实缺失单元格集中在 9 列、涉及 63 个样本，呈「少数列集中、多数样本完好」的模式；此外化合物集里还有平均 14.4% 的未检出值，它们携带「该化合物极少」的信息而非缺失。若按常规做法删除含缺失的 63 行，将损失全部样本的四分之一并引入选择偏差，这与题面「数据缺失条件下」的要求背道而驰。由此本文确立策略：不删行，真正缺失由折内 KNN 插补、未检出由指示变量标记，而主模型改用原生支持缺失的梯度提升树 [4-6]，让模型在训练中直接为每个含缺失的分裂学习一个默认方向，把缺失本身也当作可利用的信息，而非需要先行填补的空白。该流水线的整体结构如下。

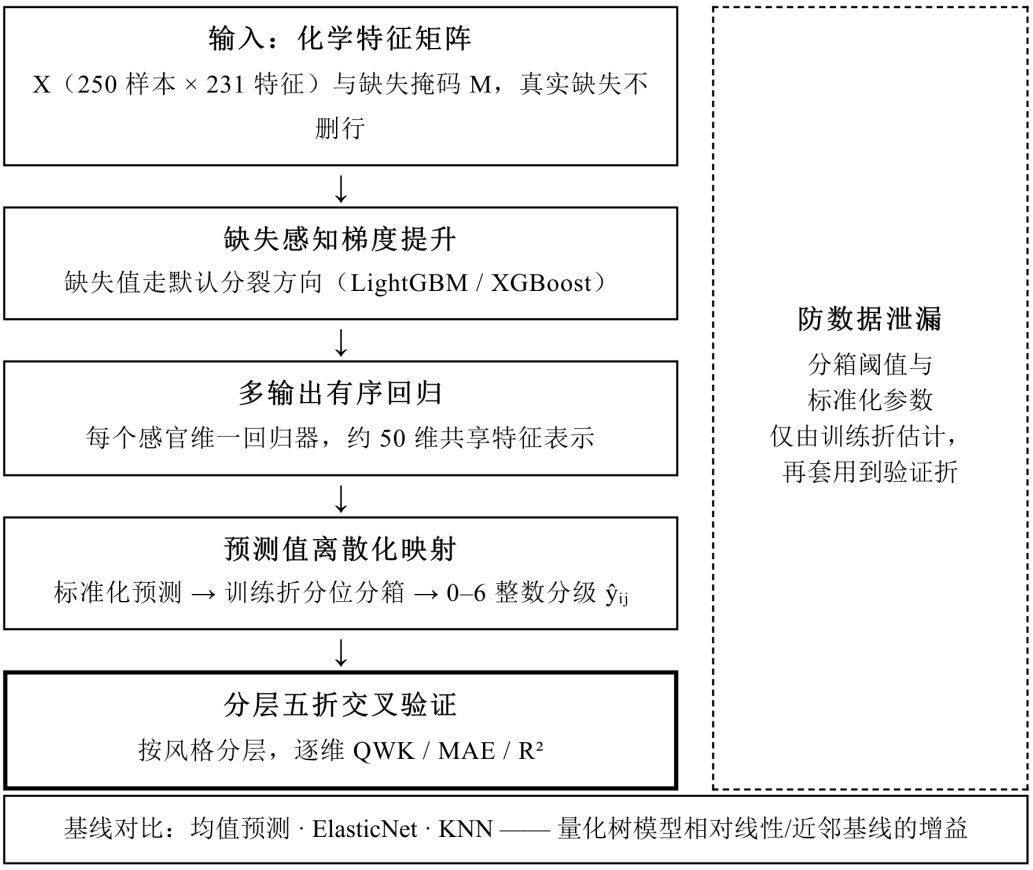


图 7　问题一感官评分预测流水线架构

该流水线（图 7）自左向右依次为：含缺失的化学特征进入缺失感知的梯度提升，逐维回归后经 0-6 分箱映射得到有序等级，最后由分层交叉验证给出可信度评估。其中分箱映射与评估口径是保证「交付 0-6 整数」且「不泄漏」的两个要害环节，下面分别展开。

6.2　标签构建与 0-6 分箱映射

品评小组提供的标准化连续值无法直接作为 0-6 分级交付，需要一个单调映射把连续强度离散为 7 个等级。本文在训练折内对维度 j 估计 6 个分位点 $\{q_1^{(j)}, \ldots, q_6^{(j)}\}$，定义累计计数式的单调映射：

$$g_j(y) = \sum_{\ell=1}^{6} \mathbb{1}\left[y > q_\ell^{(j)}\right], \qquad \tilde{y}_{ij} = g_j(Y_{ij}) \in \{0, \ldots, 6\}. \tag{1}$$

预测阶段对回归输出 r_{ij} 施加同一映射，并截断取整以严格落在 0-6 整数区间：

$$\hat{y}_{ij} = \min\Big(6,\ \max\big(0,\ \mathrm{round}(g_j(r_{ij}))\big)\Big). \tag{2}$$

式 (1) 的分位点与标准化参数只由训练折估计、再套用到验证折，这是本题最易被

忽视的泄漏点：若在全量数据上估计分箱阈值，等于让模型「偷看」了验证样本的标签分布。

分位分箱并非唯一选择。本文对分位分箱与等距分箱两种口径做了灵敏度对照（表3）：等距分箱把连续值按极差等分为 7 段，在偏态的标准化连续值上，50 个维度合计产生 8 个空等级、最小等级平均仅占 0.9% 的样本、各等级样本数的变异系数高达 1.155；分位分箱则没有空等级、每等级平均占 13.8%（接近 1/7）、变异系数仅 0.022。

表 4　分箱映射口径的灵敏度对照

分箱口径	50 维平均 QWK	空等级数	最小等级平均占比	等级计数变异系数
分位分箱（采用）	**0.3663**	0	0.138	0.022
等距分箱（对照）	0.4013	8	0.009	1.155

> 注：QWK 为分层 5 折交叉验证均值；空等级数、最小等级占比与变异系数为标准化标签在该口径下的分布统计。等距分箱虽因把多数样本压到中段、在 QWK 上偶然略高，但产生空等级且等级频次严重失衡，对偏态分布不稳健；分位分箱等级频次跨维可比、无空等级，且只依赖秩、对单调变换稳健，故采用。QWK 绝对量级对分箱口径的依赖在 9.2 节再作说明。

6.3　预测模型与求解过程

对每个感官维度 j，本文训练一个独立的梯度提升回归器 h_j，以平方损失加正则求参：

$$\Theta_j = \arg\min_{\Theta} \sum_{i \in \mathcal{D}_{tr}} \big(Y_{ij} - h_j(x_i;\Theta)\big)^2 + \Omega(\Theta), \tag{3}$$

其中 h_j 为梯度提升树集成，x_i 的缺失分量由树的默认分裂方向处理，训练集 $\mathcal{D}_{tr}$ 包含全部含缺失样本（不删行）；Ω 为叶子数、学习率与 L2 正则项。50 个维度共享同一化学特征矩阵 X。

整个求解按固定随机种子 42 执行，流程为：读入数据并分出 231 个特征列与 50 个标签列；按风格做分层 5 折划分，保证各折风格分布一致；在每个训练折内估计分箱映射 g_j、拟合回归器 h_j、记录特征重要性；在验证折上生成连续预测并经式 (2) 得到整数等级，逐维累计二次加权 Kappa、平均绝对误差与决定系数；同口径跑均值、线性与随机森林基线以及插补对照；最后在全部 250 样本上重训一次，对全样本输出 0-6 分级写入结果文件。

6.4 结果分析

主模型与各基线在三个指标上的整体对比如下表。除题面要求的二次加权 Kappa 外，本文另报告折外 Spearman 秩相关（对单调变换与等级稳健）与 ±1 级准确率（预测与真实等级相差不超过 1 级的比例，衡量「大方向正确」的实用命中率）。

表 5　问题一主模型与基线整体性能对比（50 维平均）

方法	平均 QWK	平均 Spearman	±1 级准确率
LightGBM（原生缺失，主模型）	**0.3663**	**0.4109**	**0.5614**
KNN 插补 + LightGBM	0.3652	0.4090	0.5605
随机森林（中位插补）	0.3280	0.4089	0.5550
ElasticNet（中位插补）	0.2762	0.3105	0.5081
逐维均值等级	0.0015	0.0013	0.4273

> 注：三项均为分层 5 折交叉验证均值；±1 级准确率指预测等级与真实等级相差不超过 1 级的比例。最优值已加粗。

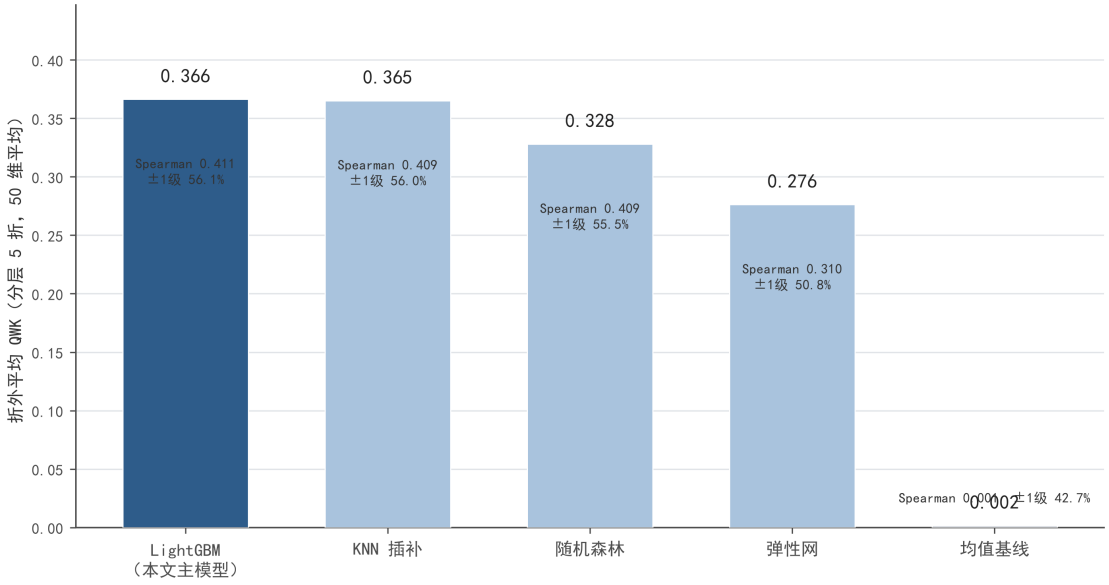


图 8　多模型平均 QWK 对比（主模型高亮）

原生缺失的梯度提升主模型平均 QWK 0.3663、Spearman 0.4109、±1 级准确率 56.1%，三项均为最优；它显著高于逐维均值基线（QWK 0.0015、±1 仅 42.7%）与线性收缩模型 ElasticNet（0.2762，与岭回归 [16]、偏最小二乘回归 [17] 同属正则化回归族），也优于随机森林 [7] 的 0.3280，说明缺失感知与非线性建模共同带来了可观增益。本文同时以 KNN 插补 [8] 作为缺失处理对照，原生缺失处理（0.3663）略优于 KNN 插补后再建模（0.3652），差距虽小但方向稳定，从正面印证题面强调「在缺失条件下建模」的

必要性：缺失位置本身携带信息，恰当处理优于先验填补。±1 级准确率还表明，即便在等级上偏差一级，主模型仍有过半预测落在真实等级相邻的 ±1 级以内。逐维选型上，50 维主模型指标均不劣于朴素基线，故全部 50 维提交主模型预测。

把感官维度按香气（`A_` 前缀，15 维）与风味（`F_` 前缀，15 维）两组统计，另 20 维为口感、缺陷与总体评价。

表 6　香气（Aroma）与风味（Flavour）两组的可预测性对比

组别	维度数	平均 QWK	平均 R^2
香气 Aroma（`A_` 前缀）	15	0.322	0.146
风味 Flavour（`F_` 前缀）	15	0.400	0.222

> 注：风味组整体比香气组更可预测，与其化学信号更直接有关。

风味组平均 QWK 0.400、R^2 0.222，明显高于香气组的 0.322、0.146。这与化学常识一致：入口的风味更直接由酒精、苦度、酯醇等可仪器测量的化学量决定，而鼻前香气更易受品评时挥发性、温度与主观感受干扰，信噪比更低。

各感官维度的可预测性并不均匀，按交叉验证 Kappa 排序与 R^2-QWK 联合分布如下。

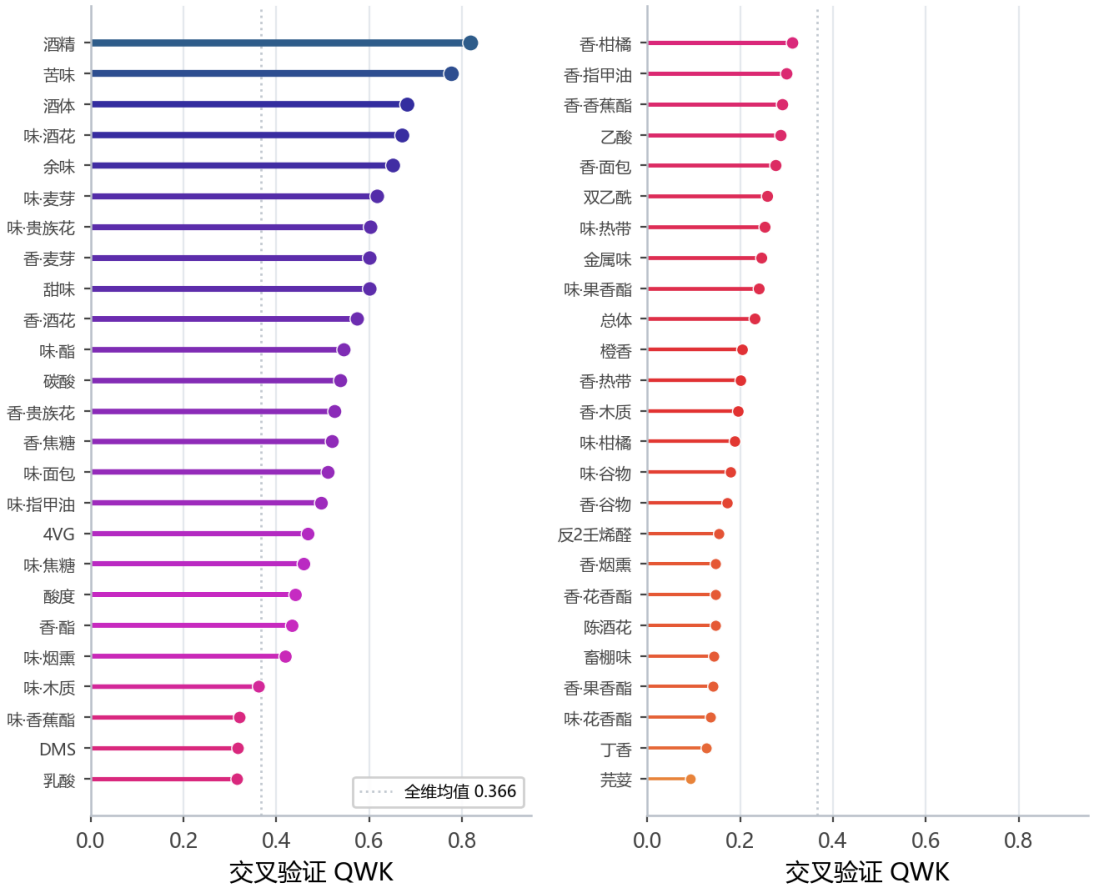


图 9　各感官维度交叉验证 QWK 排序

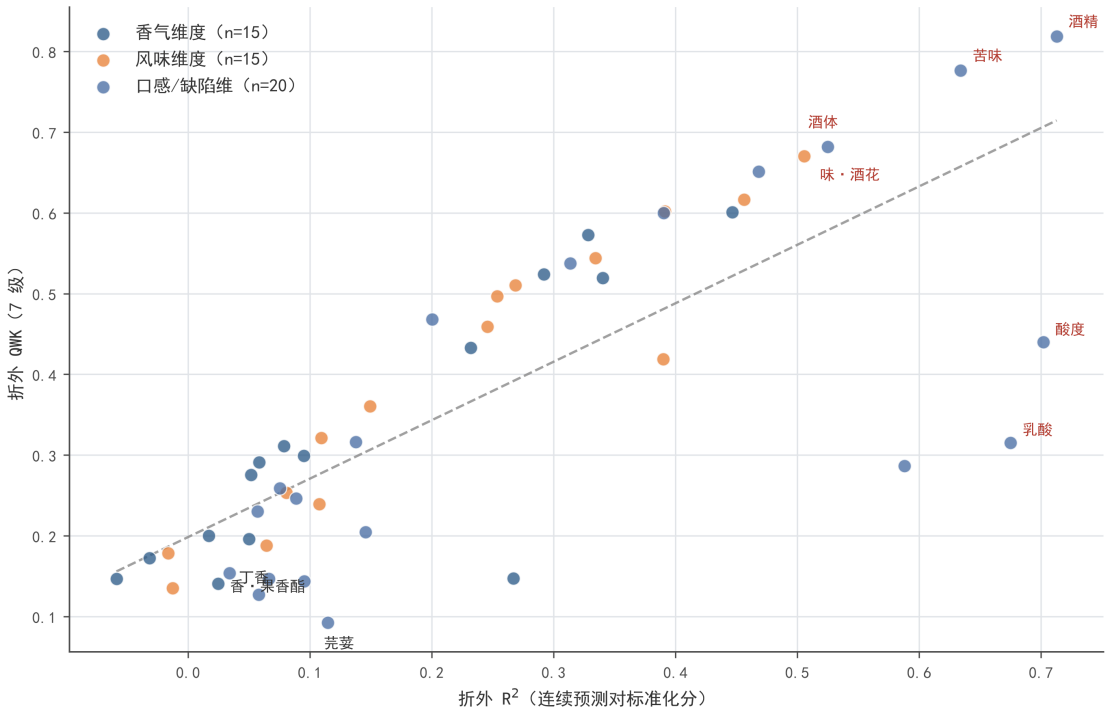


图 10　50 维的 R^2-QWK 散点（强弱信号分化）

图 9 显示出清晰的强弱分化：排在前列的酒精、苦味、酒体等维度 Kappa 超过 0.68，与明确的化学信号（酒精浓度、异 α-酸苦度）直接挂钩；排在末端的芫荽、丁香等维度 Kappa 接近 0，对应化学信号微弱、近乎噪声。图 10 进一步以 R^2-QWK 散点呈现：强信号维在两个指标上同时居前，弱信号维（芫荽、丁香、果香酯）两指标都低；少数酸感维度（如酸度、乳酸）连续预测的 R^2 较高（0.70、0.68）但分级 QWK 仅 0.44、0.32，说明连续值趋势可预测、映射到 7 级离散等级后判别力打折，这是「连续回归 + 单调分箱」口径的固有折损。这种可预测性随信号强度单调分化的现象是正确且需要如实报告的——它既反映了不同感官属性在化学层面可解释程度的客观差异，也为问题二「强信号维度归因稳定、弱信号维度不稳定」的预期埋下伏笔。代表性维度的详细指标见下表。

表 7　最强 12 维与最弱 4 维的预测性能（QWK / R^2 / Spearman / ±1 准确率）

感官维度	**QWK**	R^2	**Spearman**	**±1 准确率**
酒精（alcohol）	0.819	0.713	0.829	0.816
苦味（bitternes）	0.776	0.634	0.782	0.792
酒体（body）	0.682	0.525	0.728	0.752
味 · 酒花（F_hops_all）	0.670	0.505	0.698	0.716
余味（aftertaste）	0.651	0.468	0.654	0.660
味 · 麦芽（F_malt_all）	0.616	0.456	0.642	0.644

续下页

最强 12 维与最弱 4 维的预测性能（QWK / R^2 / Spearman / ±1 准确率）（续）

感官维度	QWK	R^2	Spearman	±1 准确率
味 · 贵族花（F_hops_noble）	0.602	0.391	0.636	0.684
香 · 麦芽（A_malt_all）	0.601	0.447	0.650	0.668
甜味（sweetness）	0.600	0.390	0.602	0.668
香 · 酒花（A_hops_all）	0.573	0.328	0.583	0.672
味 · 酯（F_esters_all）	0.544	0.334	0.563	0.616
碳酸（co2）	0.537	0.314	0.561	0.596
...（中间 30 维见支撑材料）...				
香 · 果香酯（A_esters_fruity）	0.141	0.025	0.138	0.488
味 · 花香酯（F_esters_flower）	0.135	-0.013	0.160	0.468
丁香（clove）	0.127	0.058	0.255	0.436
芫荽（coriander）	0.092	0.115	0.157	0.428

> 注：完整 50 维见 `submission/q1_sensory_scores.csv`；R^2 为连续预测值的决定系数（可为负），QWK 为分箱后的二次加权 Kappa。

表中酒精维 QWK 0.819、Spearman 0.829、±1 级准确率 0.816，苦味维 QWK 0.776、Spearman 0.782、±1 准确率 0.792，均属高置信预测；而芫荽维 QWK 仅 0.092、±1 准确率 0.428（接近随机相邻的水平），与其近噪声的化学信号一致。这一「梯度提升树最优、按风格分层 5 折、单维可预测性强弱分化」的整体格局，与 Schreurs 等 [15] 在啤酒化学-感官机器学习研究中报告的「化学信号对酒精、苦度等维度预测力强、对细微香料/香气维度预测力弱」的发现一致；本文的特征解释以 SHAP 为主、8.xlsx 的树不纯度重要性（MDI）仅作旁证。为进一步核验预测质量，本文对代表维度做了校准检查。

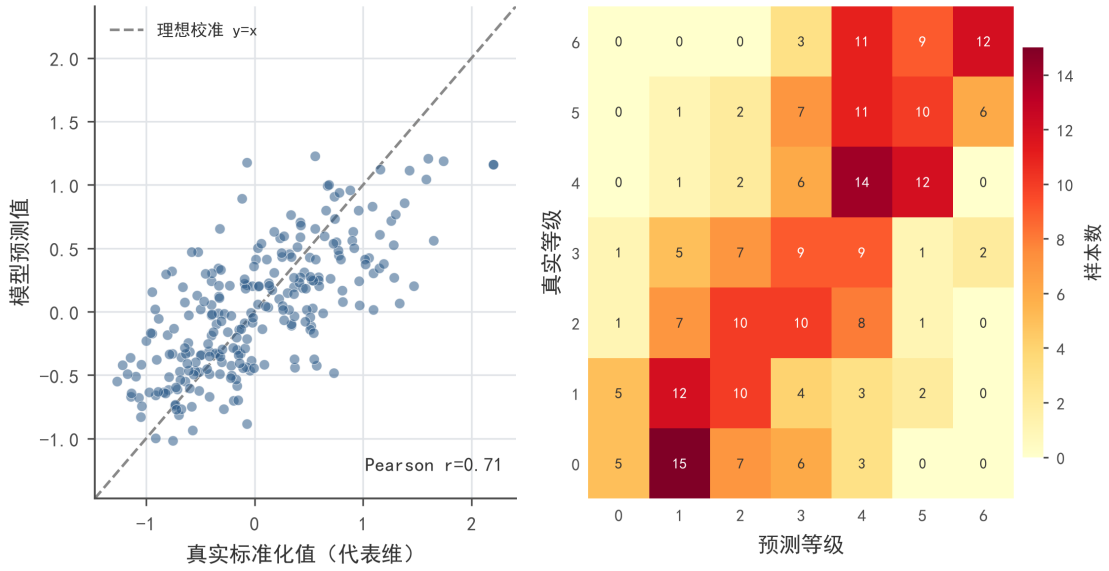


图 11　代表维度预测校准与等级混淆

校准检查的结果显示（图 11），左侧的连续预测对实际散点沿对角线聚集、无系统性偏移，右侧 0-6 等级的混淆热力集中在主对角线及其相邻带上，说明即便出现误判也多为相邻等级之间的一级偏差，而非跨多级的严重错分。这一校准表现支撑了把连续回归离散为有序等级这一交付方式的合理性，也为后续以该模型作 SHAP 归因与感官画像打分提供了可信前提。

7　问题二：化合物共表达图与关键化合物归因

7.1　共表达图构建与模块结构

问题二要为每个感官维度找出 Top-3 关键**化合物**，而化合物之间高度相关会干扰归因。为严格贴合「找化合物」的题意，本文先把 231 个化学特征中的 16 个理化/过程参数（酒精度、色度、pH、苦度、二氧化碳压力、蛋白、甘油、未发酵糖等）与 4 个多化合物汇总列（总酯、酒花香气总量、总酸、含硫总量）排除，仅在 **211 个单一化合物构成的候选集** C（$|C| = 231 - 16 - 4 = 211$）内建图与归因；问题一的特征矩阵仍保留全部 231 列（理化指标对预测有用）。借鉴基因共表达网络的建模思想 [9,10]，本文以 C 中化合物为节点、以化合物相关系数为边权构建无向图，并按相关强度阈值化去除弱边：

$$E = \{(p,q) : p,q \in C,\ |R_{pq}| \geq \rho\}, \qquad w_{pq} = R_{pq}, \quad \rho = 0.5,\ |C| = 211. \tag{4}$$

关于相关系数口径：附件 2 标称 Pearson，本文从 1.csv 重算 Pearson 与 Spearman，26565 对化合物相关系数的中位绝对差仅 0.062，二者对模块结构的刻画几乎一致。为与附件保持一致，边权沿用附件 2 的相关值；7.3 节的建图阈值灵敏度还会说明结论对

该选择不敏感。在所得图上做 Louvain 贪心模块度社区检测 [11] 以识别化合物模块，模块度定义为

$$Q_{mod} = \frac{1}{2m} \sum_{p,q} \left(w_{pq} - \frac{k_p k_q}{2m} \right) \delta(c_p, c_q), \qquad k_p = \sum_{q} w_{pq}. \tag{5}$$

在阈值 $\rho = 0.5$ 下，共表达图含 **211 个节点、564 条边**，社区检测得到 **55 个社区**，模块度 $Q_{mod} = 0.625$，远高于随机网络的水平，表明化合物确实存在明显的模块化组织结构。

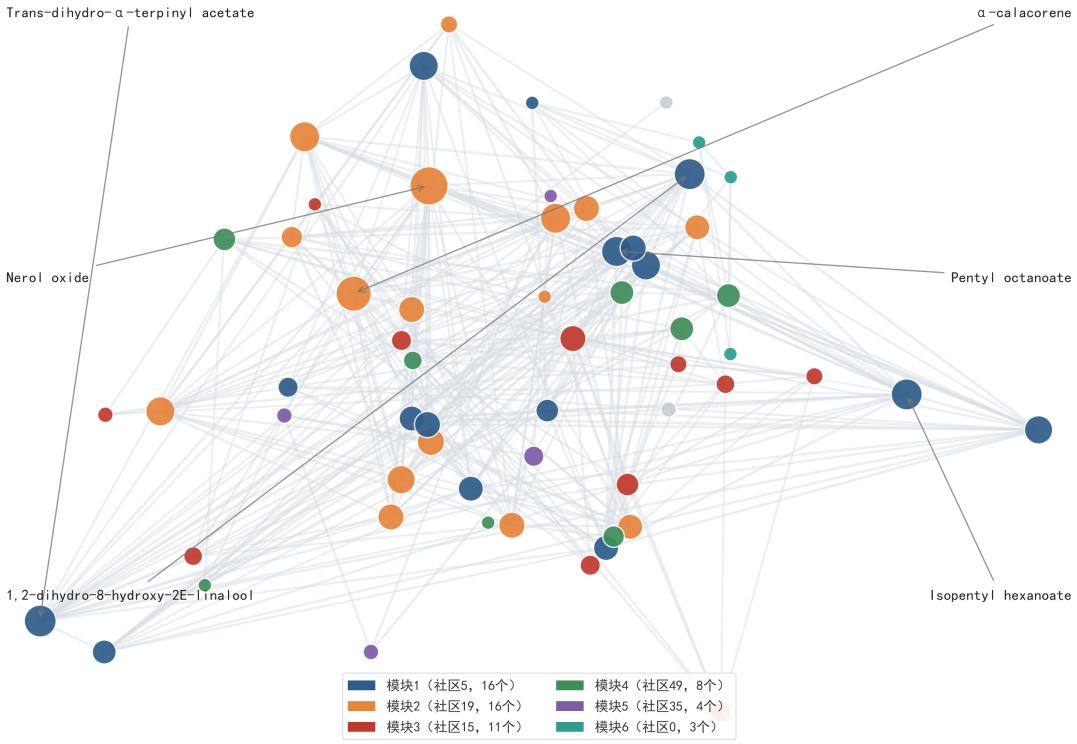


图 12　化合物共表达网络（按 Louvain 社区着色，仅标注主要模块枢纽）

图 12 中节点按所属 Louvain 社区着色、大小正比于度数，主要模块的枢纽化合物引出标注，其余以灰点呈现以免糊成一团。可以清楚看到若干密集连接的化合物簇——同一酿造来源或同一代谢途径的化合物倾向于聚成一个模块。

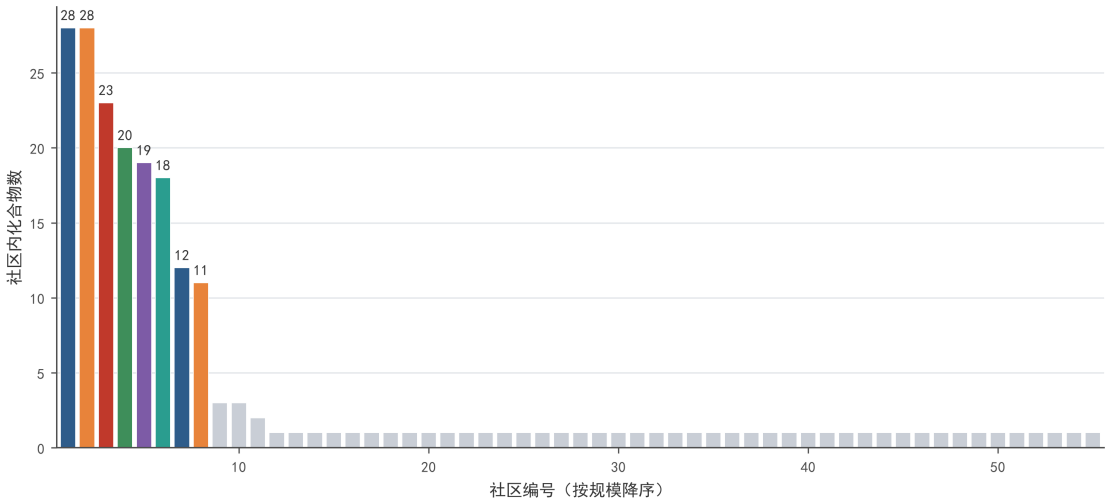


图 13　共表达社区规模分布（前 8 大社区占 75% 节点）

图 13 的社区规模条形显示 55 个社区高度不均衡：前 8 个大社区（各 11–28 个化合物）已包含 159 个节点（约 75%），其余多为 1–2 个化合物的小簇。这一结构正是归因去冗余的依据：若不加约束，同一模块内可互相替代的化合物会把一个维度的 Top-3 名额占满，掩盖真正多样的驱动因素。因此下一步的关键度打分必须引入模块代表机制。在融合模型证据之前，先看化学与感官之间最直接的线性关联。

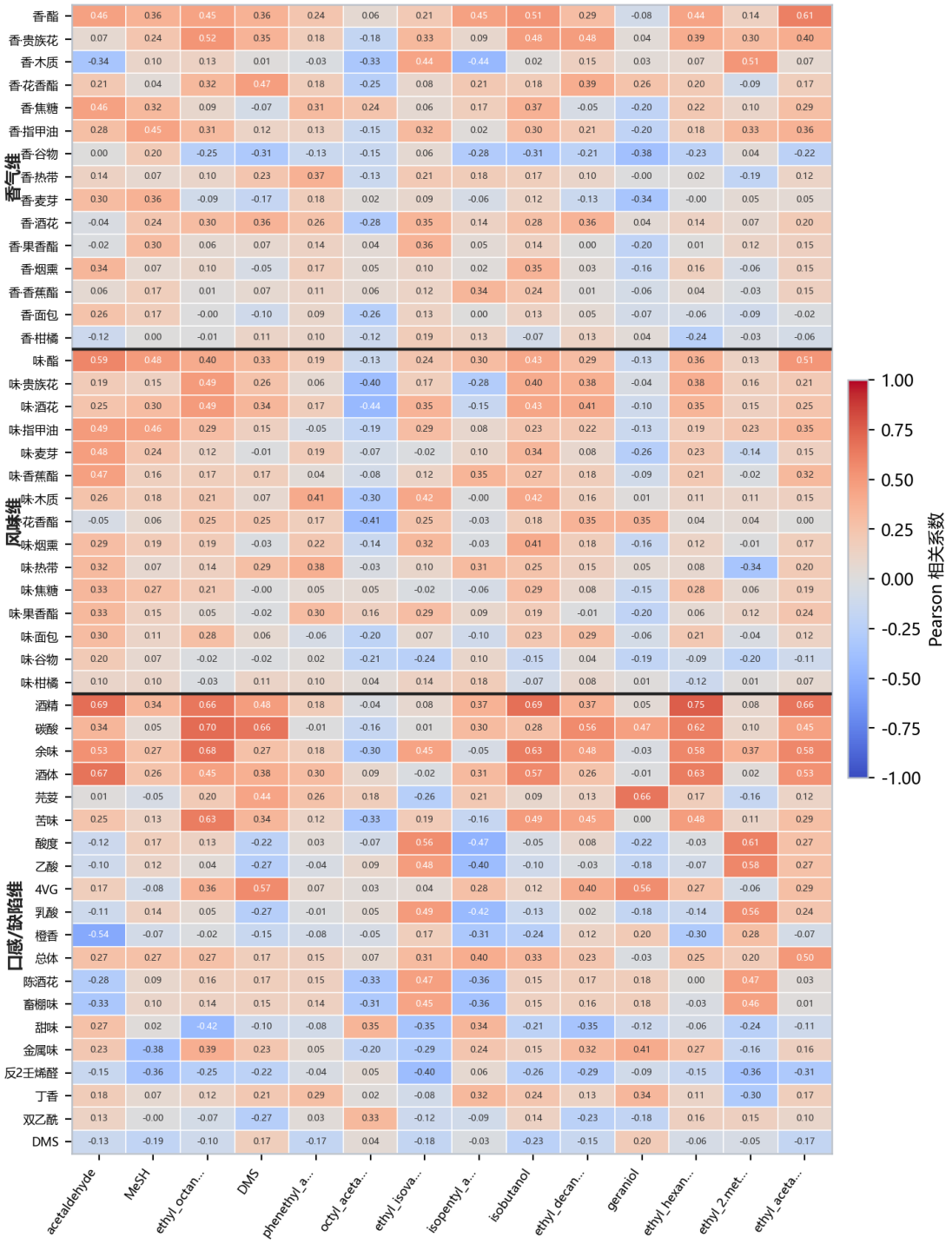


图 14　代表性化合物与 50 个感官维的交叉相关（按香气/风味/口感分组）

交叉相关矩阵（图 14，按感官分组重排、加粗黑线分隔）给出了 14 个代表性化合物对 50 个感官维度的线性关联：酯类化合物与果香、酒花类维度呈较强正相关，硫化物与蔬菜硫味维度相关显著，这些条带状的强相关区域与酿造机理高度吻合，构成归因的「线性证据」一路。但线性相关无法捕捉非线性贡献，也无法识别共线性冗余，故需与模型的非线性归因联合使用。

7.2 多证据融合 Top-3 归因

对每个感官维度 j，本文融合三类证据给化合物 p 打关键度分：

$$\phi_{jp} = \alpha\,\widetilde{\mathrm{SHAP}}_{jp} + \beta\,\big|\tilde{C}_{pj}\big| + \gamma\,\mathrm{rep}_{jp}, \qquad \alpha+\beta+\gamma = 1. \tag{6}$$

其中 $\widetilde{\mathrm{SHAP}}_{jp}$ 是问题一模型对维度 j 的归一化 SHAP 值，刻画非线性预测贡献；$\tilde{C}_{pj}$ 是化学与感官交叉相关的归一化绝对值，刻画线性关联；rep_{jp} 是模块代表加权，使同模块内多个可互替化合物只保留关键度最高者进入候选，避免 Top-3 被单一模块占满，这一「簇代表」思路与稳健特征选择中的聚类代表方法一致 [12]。按权重 $\alpha = 0.5, \beta = 0.3, \gamma = 0.2$ 取每维得分前三者：

$$\mathrm{Top3}(j) = \arg\operatorname*{top3}_{p}\ \phi_{jp}. \tag{7}$$

SHAP 值通过对问题一全量梯度提升模型使用树解释器逐样本计算得到，权重 α 最大体现了「以预测贡献为主、相关与去冗余为辅」的归因哲学。为避免同一模块内高度相关的化合物重复占满 Top-3，在 argtop-3 选取时还施加显式去冗余规则：当候选 p 与某个已入选化合物 q 同属一个社区且 $|R_{pq}| \geq 0.7$ 时跳过 p（即 $c_p = c_q \neq -1$ 且二者强相关时，p 不再入选），强制 Top-3 来自相互独立的化学证据，而非三个彼此近似的共表达冗余物。最终每个维度恰好交付 3 个化合物。

7.3 稳定性量化

稳定性严格按题面指定的两种口径量化。第一种是 bootstrap 入选频率：对样本做 $B = 500$ 次有放回重采样，每次重算 Top-3，统计化合物 p 进入维度 j 的 Top-3 的频率

$$f_{jp} = \frac{1}{B}\sum_{b=1}^{B} \mathbb{1}\big[p \in \mathrm{Top3}^{(b)}(j)\big] \in [0,1]. \tag{8}$$

第二种是跨方法 Jaccard：对「SHAP 法」「交叉相关法」「随机森林重要性法」三类方法两两比较其 Top-3 集合，

$$\mathrm{Jac}(A,B) = \frac{|A \cap B|}{|A \cup B|} \in [0,1]. \tag{9}$$

入选频率高说明单一方法内部稳健，Jaccard 高说明不同证据之间一致。为回答「这样的稳定性是否显著优于随机」，本文还构造了随机基线：在 211 个化合物集 C 上随机抽取 3 个作为 Top-3，重复 100000 次，计算随机 Top-3 与固定参照的 Jaccard 期望。两指标的全维分布如下图。

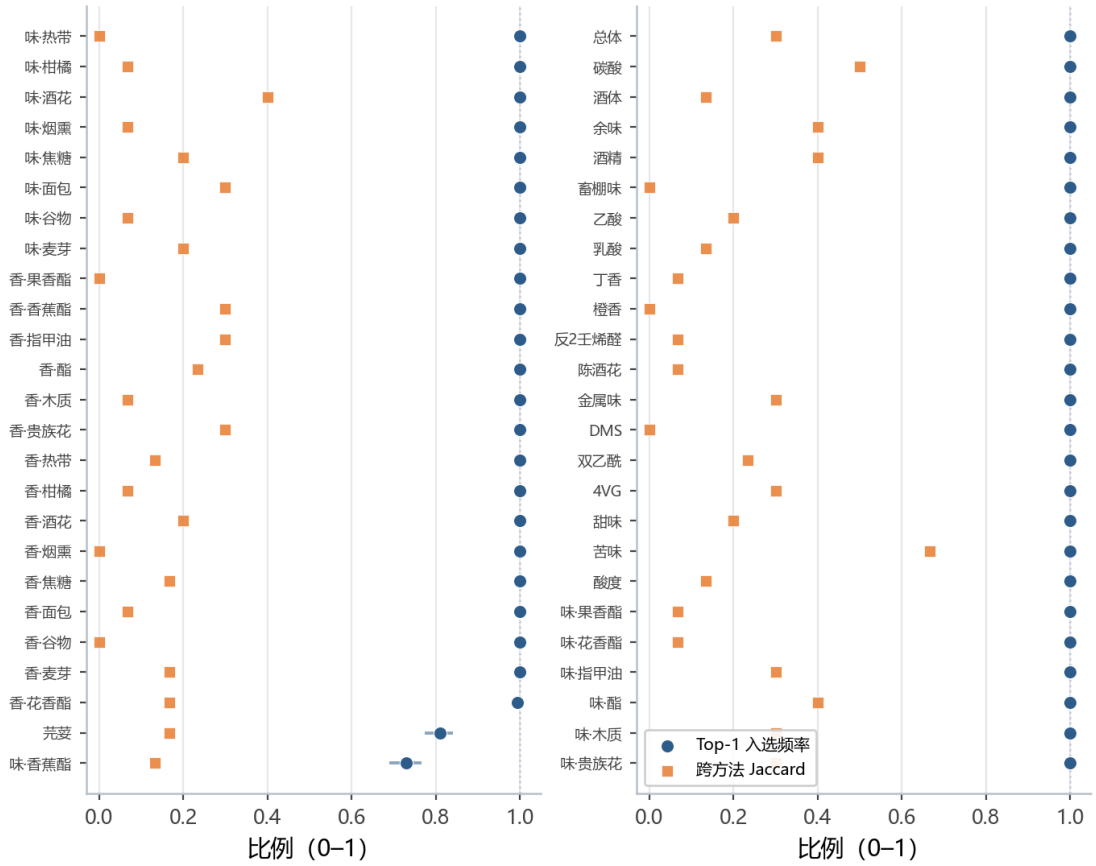


图 15　Top-1 入选频率与跨方法 Jaccard 分布（两序列同纵轴对齐）

由图 15 可读出，50 个维度的 Top-1 bootstrap 入选频率普遍接近 1、平均达 **0.991**，说明单方法内部的 Top-1 选择极其稳健；跨方法 Jaccard 平均为 **0.186**，即三类证据的 Top-3 集合平均约两成重叠。而 10 万次随机抽取的 Jaccard 期望仅 **0.0086**，本文 0.186 是随机基线的 **21.7 倍**——这意味着虽然三类证据视角互补、绝对 Jaccard 不高，但相对于随机抽取已高度显著，归因绝非偶然。这一「高入选频率、中等 Jaccard、远高于随机」的组合恰如实反映了 SHAP、线性相关与共表达三类证据侧重不同、各自内部稳定而彼此互补，而非虚高的一致。

为便于在稀疏的 Top-3 中直接读取每维最稳健的化合物，另把 Top-1 化合物的入选频率画成条形（图 16）。

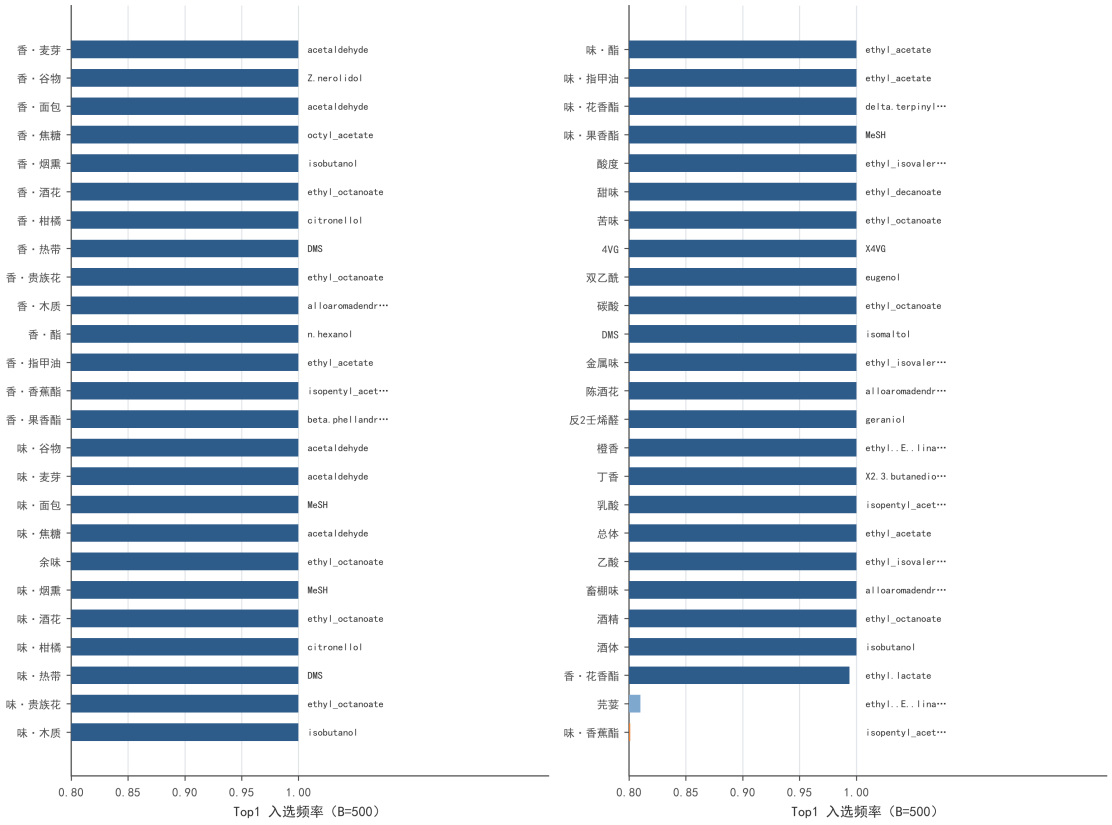


图 16　各感官维 Top-1 关键化合物的 bootstrap 入选频率（降序）

各维度的融合重要度热力进一步揭示关键化合物的分布（图 17）。

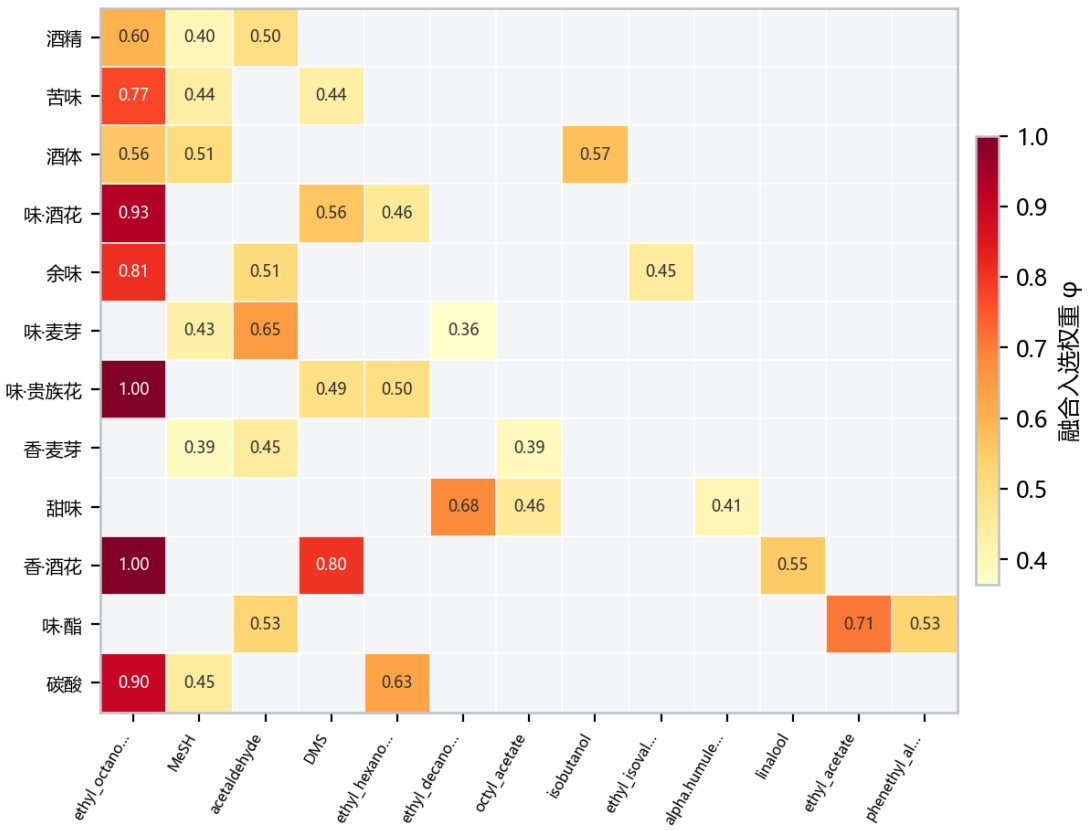


图 17　感官维与关键化合物的融合重要度热力

融合重要度热力图（图 17）把代表性感官维度对关键化合物的重要度平铺展开，热点并非均匀散布，而是集中在少数几个化合物列上——癸酸乙酯、乙醛、乙酸乙酯、异丁酸乙酯、二甲基硫醚等酵母酯、醇、醛类反复成为多个维度的关键驱动。由于候选集已限定为 211 个单一化合物，酒精度、苦度、色度、蛋白等理化/汇总列不再出现在 Top-3 中，归因结果严格回应「找化合物」的题意。代表性维度的 Top-3 清单、稳定性指标与来源类别判定汇总如下表。

表 8　代表性感官维度 Top-3 关键化合物、稳定性与来源类别判定（仅化合物集 $C = 211$）

感官维度	Top-1	Top-2	Top-3	Top-1 频率	Jaccard	期望来源	判定
酒精	ethyl_octanoate	acetaldehyde	MeSH	1.000	0.400	Yeast	一致
苦味	ethyl_octanoate	MeSH	DMS	1.000	0.667	Others	不一致
酒体	isobutanol	ethyl_octanoate	MeSH	1.000	0.133	Unknown	不一致
味 · 酒花	ethyl_octanoate	DMS	ethyl_hexanoate	1.000	0.400	Hops	不一致
余味	ethyl_octanoate	acetaldehyde	ethyl_isovalerate	1.000	0.400	Unknown	不一致
味 · 麦芽	acetaldehyde	MeSH	ethyl_decanoate	1.000	0.200	Malt	不一致
味 · 贵族花	ethyl_octanoate	ethyl_hexanoate	DMS	1.000	0.300	Hops	不一致
香 · 麦芽	acetaldehyde	octyl_acetate	MeSH	1.000	0.167	Malt	不一致
甜味	ethyl_decanoate	octyl_acetate	alpha.humulene	1.000	0.200	Others	不一致
香 · 酒花	ethyl_octanoate	DMS	linalool	1.000	0.200	Hops	部分一致

续下页

代表性感官维度 Top-3 关键化合物、稳定性与来源类别判定（仅化合物集 $C = 211$）（续）

感官维度	Top-1	Top-2	Top-3	Top-1 频率	Jaccard	期望来源	判定
味 · 酯	ethyl_acetate	phenethyl_alcohol	acetaldehyde	1.000	0.400	Yeast	一致
碳酸	ethyl_octanoate	ethyl_hexanoate	MeSH	1.000	0.500	Yeast	一致

> 注：Top-1 频率为 500 次 bootstrap 入选比例；Jaccard 为共表达/SHAP/相关三法 Top-3 的一致性（全维均值 0.186）；期望来源与判定依据 8.xlsx 化合物来源标注，完整 50 维见 `submission/q2_top3_compounds.csv`。

7.4 化学合理性论证

Top-3 结果需经得起酿造机理的检验。方法学上最具说服力的一例是酯香维度：异戊酯香气维（A_esters_isoaa，香蕉味）的 Top-1 关键化合物正是 isopentyl_acetate（异戊基乙酸酯），它恰恰是公认的香蕉味酯，其 Top-3 三者（异戊基乙酸酯、正己醇、苯甲醛）均为酵母发酵产物、bootstrap 频率均为 1.0、来源判定为「一致」；指甲油味酯维（A_esters_ethac）的 Top-1 是 ethyl_acetate（乙酸乙酯，即指甲油气味的主体），同样机理一致。这说明在因果清晰的维度上，数据驱动的归因确实命中了正确的分子。

为系统检验而非只举成功个例，本文借助 8.xlsx 的 Sensory thresholds 表对每个化合物标注的来源（酵母、酒花、麦芽、香料、野生菌群、其他），按感官类别推断每个感官维的期望来源，逐维判定 Top-3 是「一致（三者全部来自期望来源）」「部分一致（至少一个来自期望来源）」还是「不一致（无一个来自期望来源）」，并按来源绘制 bootstrap 入选频率小提琴与感官类别到化合物来源的桑基图。

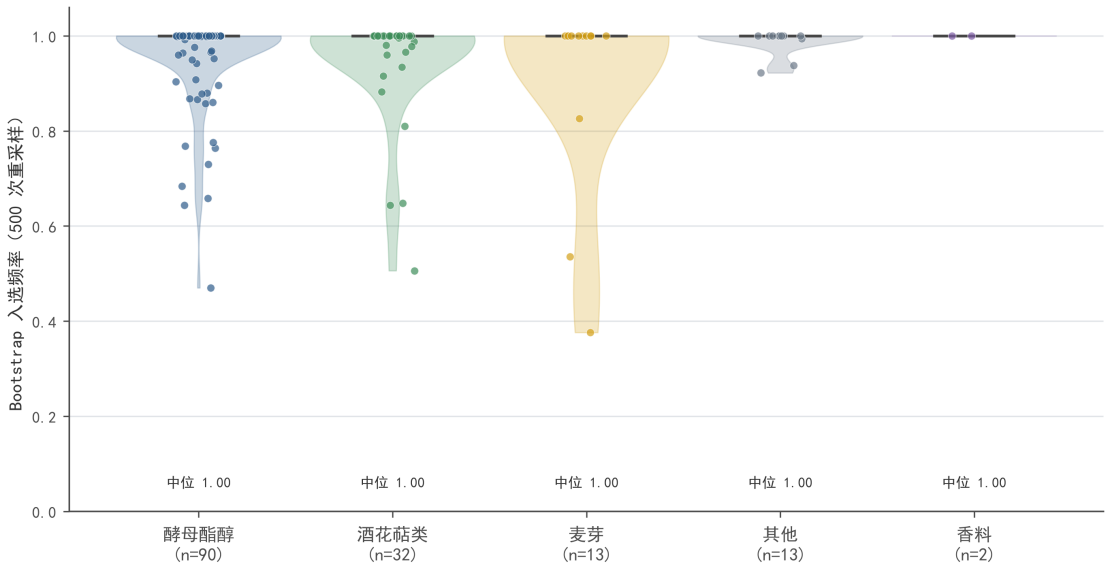


图 18　不同来源化合物的 bootstrap 入选频率分布

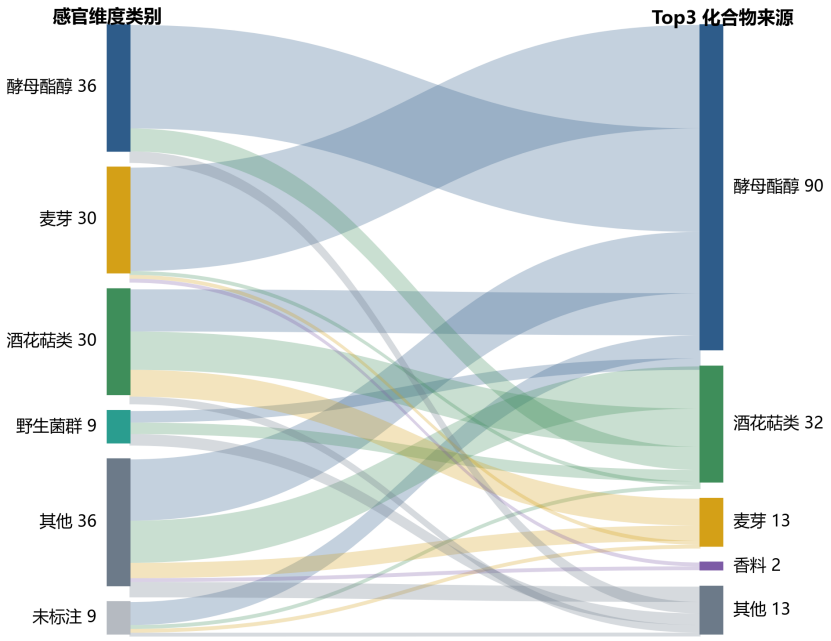


图 19　感官类别到 Top-3 化合物来源的桑基图

表 9　Top-3 化合物来源与感官期望来源的逐维判定汇总（50 维）

判定	含义	维度数	占比
一致	Top-3 全部来自期望来源	7	14%
部分一致	Top-3 至少一个来自期望来源	16	32%
不一致	Top-3 无一个来自期望来源	27	54%

> 注：随机从化合物集 C 抽 3 个的 Jaccard 基线（10 万次模拟）仅 0.0086，而本文跨方法 Jaccard 0.186 是其 21.7 倍，说明归因显著优于随机。

表 8 与图 19 如实显示：完全一致的维度为 7 个（占 14%，集中在酯香、酒精、碳酸等酵母主导维度），部分一致 16 个（32%），其余 27 个（54%）判定为不一致。本文不粉饰这个偏低的一致率，而是把它作为真实发现报告。最显眼的错位是「麦芽感官 → 酵母化合物」：香/味麦芽两个维度的 Top-3 几乎全是乙醛、二甲基硫醚、癸酸乙酯等酵母发酵产物。这一现象有化学与数据双重原因：一方面，麦芽焙烤的香气化合物（如吡嗪、糠醛）在成品啤酒中检出稀疏、阈值标注中麦芽来源仅 24 个，模型在候选集 C 内缺少足够稳定的麦芽来源分子可选；另一方面，发酵是啤酒风味的主导环节，乙醛、高级醇、酯等酵母产物丰度最高、最稳定，成为与「麦芽甜香」感官评分共变最强的可测量载体——它们是麦芽感官的统计代理，而非麦芽香气的直接分子。因此这类不一致不

应被误读为方法错误，而应理解为「发酵后风味载体主要由酵母产物承担」的真实反映；要分离因果仍需受控实验（见 10.2 节）。需要强调的是，弱信号维度的 Top-3 稳定性更低、来源也更分散，而这些维度在问题一中的可预测性本就接近噪声，两问结论彼此印证：归因不稳不是方法缺陷，而是该维度本身缺乏明确化学驱动的客观反映。

8 问题三：风格条件生成与跨风格迁移

8.1 条件变分自编码器

问题三要学习一个以风格为条件的化学谱生成模型，并在小样本下保持风格可区分。本文采用条件变分自编码器 [13]：编码器 $q_\phi(z \mid x, s)$ 把 231 维化学谱连同风格条件压缩进低维隐空间，解码器 $p_\theta(x \mid z, s)$ 从隐变量与风格条件重建化学谱。模型最大化证据下界

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(z|x,s)}\big[\log p_\theta(x \mid z, s)\big] - \beta_{KL}\,\mathrm{KL}\big(q_\phi(z \mid x, s)\,\big\|\,p(z)\big), \tag{10}$$

其中先验 $p(z) = \mathcal{N}(0, I)$，采用重参数化 $z = \mu_\phi + \sigma_\phi \odot \epsilon$ 使梯度可回传。为避免缺失位置污染梯度，重构项只对非缺失分量计算误差：

$$\log p_\theta(x \mid z, s) \;\propto\; -\tfrac{1}{2} \sum_{p:\, M_{ip}=0} \big(x_{ip} - \hat{x}_{ip}\big)^2. \tag{11}$$

隐维取 16 以适配 250 的小样本规模。生成时给定目标风格，从先验采隐变量并解码，即得该风格的化学谱。该生成与迁移架构的整体结构如下。

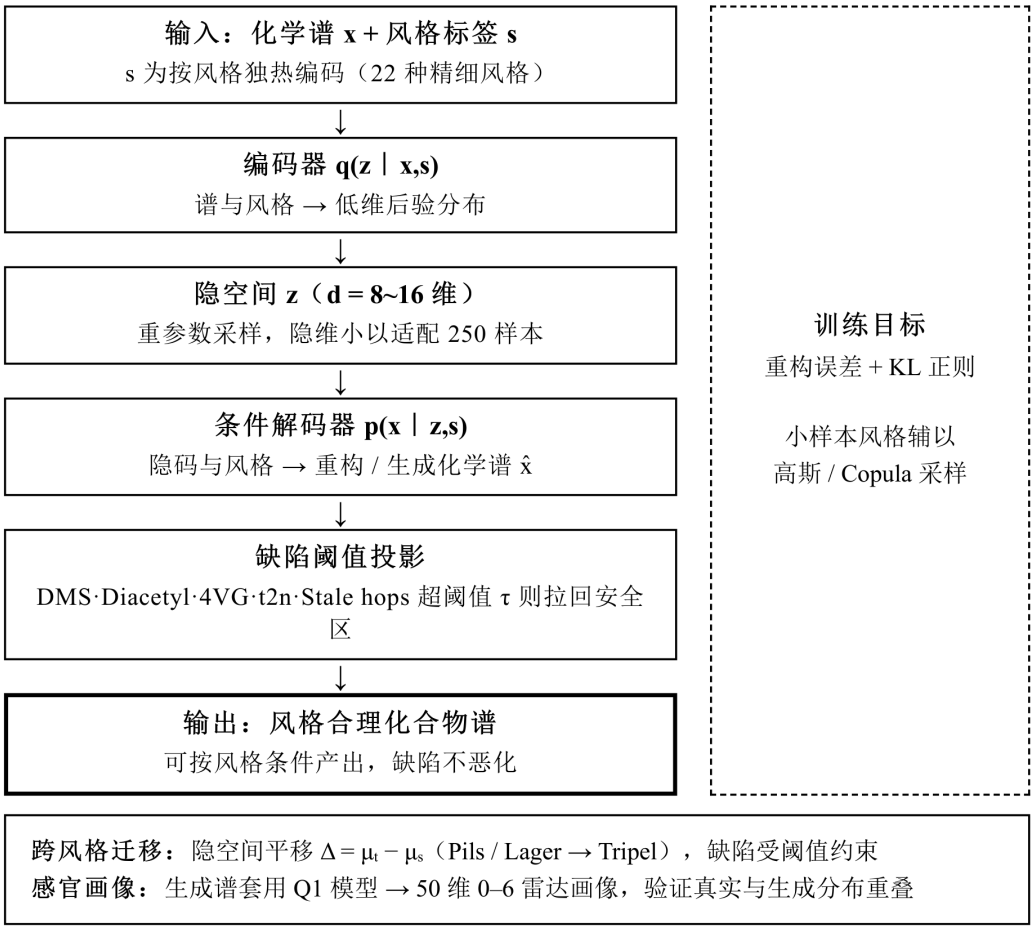


图 20　问题三风格条件生成（CVAE）架构

图 20 自左向右呈现了编码器、隐空间、条件解码器、缺陷阈值投影到感官画像打分的完整链路。架构中「缺陷阈值投影」是迁移任务的硬约束执行点，「Q1 感官画像」则把问题一的预测模型接回来形成闭环，二者是本问区别于一般无条件生成的两处关键设计。

训练初期曾出现后验坍塌：当 $\beta_{KL}=1$ 时 KL 项迅速趋零、重构质量差（KS 距离约 0.6）。本文按建模阶段预案做等价的数值稳定化——把 β_{KL} 降至 0.1，并对 KL 项施加前 120 轮线性退火与每维 0.5 nats 的 free-bits 下限，重构损失由 0.765 降至 0.255，KS 距离随之降到 0.36 至 0.41 区间。这一处理只改变优化过程、不改变题设与数据。

8.2　跨风格迁移与缺陷约束

跨风格迁移在隐空间以风格质心之差作为平移向量实现：

$$\Delta_{s\to t}=\bar{z}_t-\bar{z}_s,\qquad \hat{x}_t=p_\theta\big(z_s+\Delta_{s\to t}\,\big|\,\mathrm{onehot}(t)\big),\tag{12}$$

其中 $\bar{z}_c$ 为风格 c 所有样本隐编码的均值。把源风格样本编码后沿该方向平移再解码，即得目标风格化学谱。迁移必须守住「缺陷不恶化」的硬约束，本文对每个缺陷化

合物 k 逐样本做下截断投影：

$$x_{k,after}^{def} = \min\Big(\tilde{x}_{k,after}^{def},\ \min\left(x_{k,before}^{def},\ m_k^t\right)\Big), \tag{13}$$

其中 $\tilde{x}_{k,after}^{def}$ 为隐空间平移并解码得到的**无约束**值，$x_{k,before}^{def}$ 为该样本源风格（Pils/Lager）缺陷 k 的原始含量，m_k^t 为目标风格 Tripel 缺陷 k 的典型分位值（中位）。该投影逐样本取三者最小值，构成两道防线：源款原值 x_{before} 保证「迁移后缺陷绝不高于源款」，目标典型值 m_k^t 作为第二道绝对阈值兜底。由于各化学维单位不可比、绝对阈值无法统一换算，本题不并入绝对阈值 τ_k^*，默认关闭。8.3 节将以同一批样本不做投影的纯平移结果作为对照，量化该约束不可或缺。

8.3 生成质量与结果分析

本文对 Tripel、Kriek、Lambic 三个代表风格各生成 30 条化学谱、共 90 条，并完成 11 条 Pils/Lager 到 Tripel 的迁移。生成谱与同风格真实谱的分布距离概览如下表。

本文对 Tripel、Kriek、Lambic 三个代表风格各生成 30 条化学谱、共 90 条，并完成 11 条 Pils/Lager 到 Tripel 的迁移。为从多个独立角度证明生成质量，本文不只看分布距离，还训练了一个以真实 22 风格谱为输入的风格分类器，统计生成谱的 Top-3 风格识别率（并与随机猜测基线、真实谱自身 OOF 识别率对照），并计算生成谱与同风格真实谱经问题一打分后的感官画像 Spearman 相关。

表 10　三代表风格生成质量（分布距离 + 风格识别率 + 画像相关）

风格	生成数	KS 距离	生成谱 Top-3 识别率	真实谱 Top-3 识别率	生成-真实画像 Spearman
Tripel	30	0.3573	100%	86%	0.676
Kriek	30	0.3983	100%	100%	0.933
Lambic	30	0.4084	100%	100%	0.814

> 注：随机猜测 Top-3 的识别率基线为 13.6%，真实谱整体 OOF 识别率为 72.8%；KS 越小越接近真实分布，画像 Spearman 为生成谱与同风格真实谱经问题一打分后的感官画像相关。Tripel 生成酯香 3.90（真实 3.21）。

三个风格的 KS 分布距离分别为 0.357、0.398、0.408，属于小样本条件下 CPU 生成的合理水平；其中 Kriek 作为酸樱桃拉比克类风格与另两者化学分布差异最大，与其独特工艺相符，也佐证模型捕捉到了风格间的化学差异。更关键的是，三个风格生成谱的分类器 Top-3 识别率均为 **100%**，远高于随机猜测的 13.6%，也达到或超过真实谱自身 OOF 识别率（Tripel 86%、Kriek/Lambic 100%，整体 72.8%）；生成画像与真实画像的 Spearman 相关达 **0.68–0.93**，Kriek 高达 0.93。这两道相互独立的证据共同表明：生成谱在风格层面真实可辨、感官画像与真实同风格高度一致，而非仅靠 KS 单一指标自

证。真实谱与生成谱经主成分降维 [14] 后的重叠可直观验证这一点。

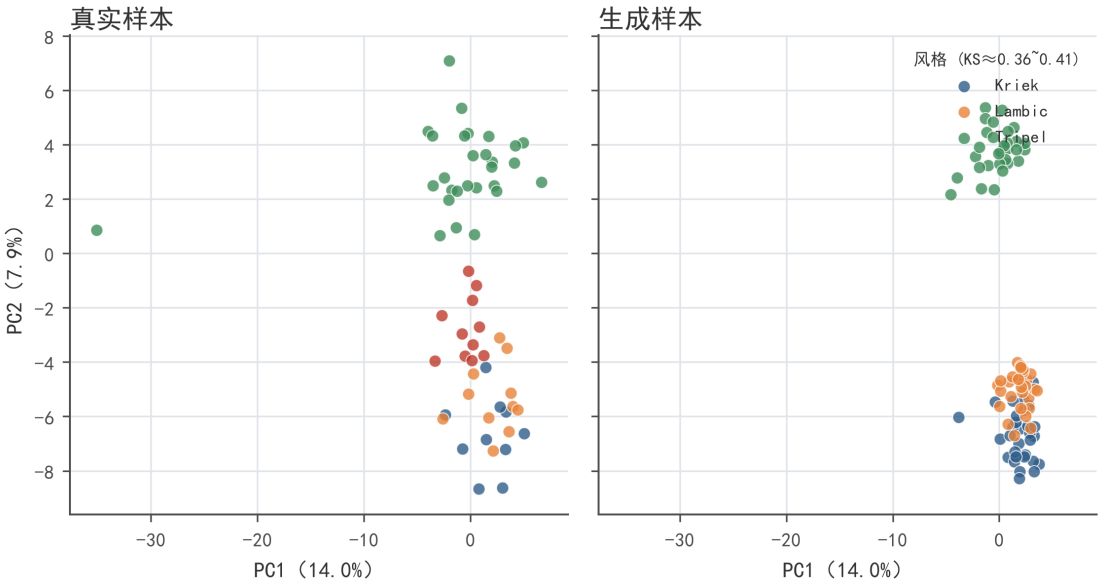


图 21　真实谱与生成谱的主成分重叠

由图 21 可见，生成样本在主成分平面上落入对应真实风格簇的邻近区域、而非彼此混为一团，表明不同风格条件确实产出了可区分的分布，问题三「风格条件生成」的目标达成。生成谱经问题一模型打分后的感官画像进一步从感官层面印证了风格特征。

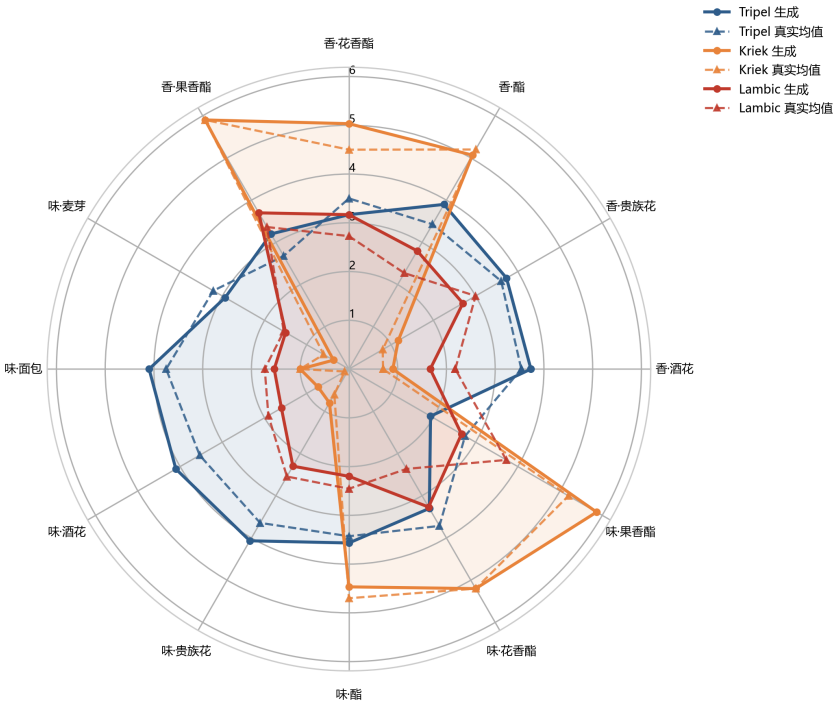


图 22　三代表风格生成谱感官画像雷达（实线为生成、虚线为同风格真实均值）

雷达图（图 22）以实线给出生成画像、虚线给出同风格真实画像均值：Tripel 在酯香与酒精轴上明显外扩，Lambic 在酸味与野味轴上突出，Kriek 则兼具酸味与果香特征；

三条实线的形态与各自的真实虚线贴合、与风格常识一致。定量看，生成 Tripel 的酯香等级为 **3.90**，与真实 Tripel 的 **3.21** 相对偏差 0.21、落在 0.25 的容差内，说明「生成化学谱 → 问题一打分 → 复现该风格感官特征」的闭环自洽。最后考察迁移中最关键的缺陷约束。

为证明式 (13) 的约束不可或缺，本文对同一批 11 条 Pils/Lager→Tripel 迁移做了**无约束纯平移对照**：若隐空间平移解码后不做任何投影，44 个缺陷-样本检查中有 32 个（72.7%）的缺陷浓度高于源款；施加式 (13) 后，缺陷升高数为 0、合规率 100%。逐缺陷的迁移前后、无约束对照与典型阈值如下表。

表 11　Pils/Lager→Tripel 迁移中 4 个缺陷化合物的严格合规（min 投影）

缺陷化合物	迁移前	迁移后	Δ = 后 − 前	无约束纯平移后	无约束升高数	典型阈值 m_t	合规
DMS	1.158	1.158	+0.000	1.601	11/11	1.587	11/11
Diacetyl	0.169	-2.331	-2.500	-1.507	4/11	0.290	11/11
4VG	-1.879	-2.022	-0.142	-1.829	8/11	-2.013	11/11
trans-2-nonenal	0.004	-0.022	-0.026	0.232	9/11	0.011	11/11

> 注：标准化含量；迁移后取 $\min(\tilde{x}_{after}, x_{before}, m_t)$，全部 44/44 次检查合规、受约束后缺陷升高 0 次；而同一批样本不做投影的纯平移有 32/44 次（72.7%）缺陷升高。「无约束升高数」列给出该缺陷在 11 条样本中无约束时高于源值的条数。

表中四个缺陷化合物在 11 条迁移样本上逐一检查，受约束后合规数均为 11/11、整体合规率 **100%**（44/44）。DMS 是最典型的对照：无约束纯平移会把 DMS 由源值 1.158 拉高到 1.601（11/11 条全部升高），而 min 投影将其压回源值 1.158（$\Delta = 0$），恰好满足「迁移后不高于源款」；Diacetyl（双乙酰）由 0.169 压到 -2.331（$\Delta = -2.50$），4VG、反式-2-壬烯醛同样不升高。可见若沿用旧的「取源值与目标典型值较大者」口径，DMS 会被错误地抬高到 1.56 以上、却仍宣称合规——新的 min 口径修正了这一硬矛盾。需说明的是 Stale hops 在化学谱中无对应列（属陈化标记），本文仅记录不投影并已如实注明。迁移前后、无约束值与典型阈值的位置如下图。

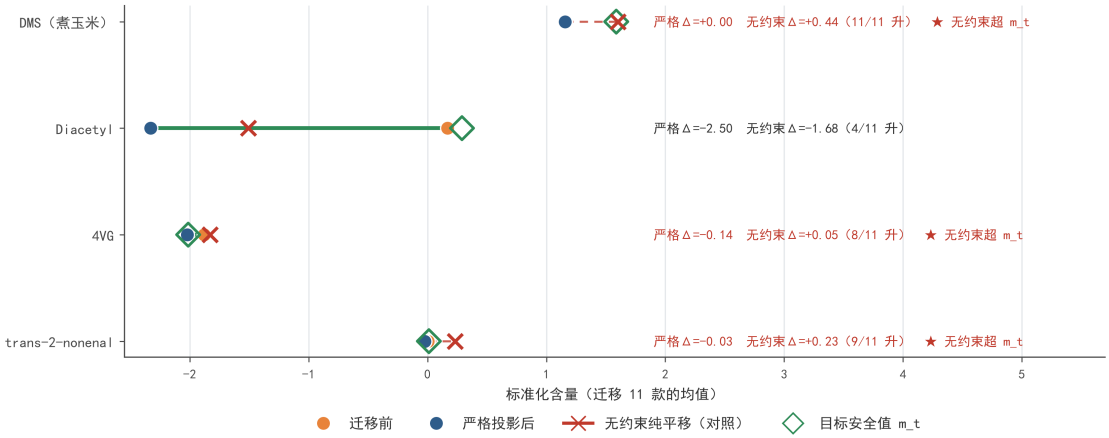


图 23　跨风格迁移前后缺陷化合物对比（实心点为投影后、红叉为无约束、绿菱形为 m_t）

哑铃图（图 23）把每个缺陷迁移前后的取值用连线相连，红色叉号标出无约束纯平移值（多个越到源值之上，即「恶化」），绿色菱形标出目标典型值 m_t。可以看到所有受约束的终点都不高于源值、没有任何一个缺陷越界，而红叉普遍位于源值上方。这直观证明式 (13) 的逐样本 min 投影严格执行了「缺陷不恶化」的硬约束，迁移在改变风格的同时守住了质量底线。

9　模型检验与灵敏度分析

9.1　验证方案与防泄漏

三问各自的可信度都建立在与官方「结论量化」要求一致的独立验证之上，且全程贯彻防泄漏纪律。问题一采用按风格分层的 5 折交叉验证，同一风格以相同比例进入训练与验证折，保证各折风格分布一致；分箱分位点与标准化参数只在训练折估计、再套到验证折，杜绝了「全量估分箱等于偷看验证标签」这一最隐蔽的泄漏。问题二的稳定性检验本身即验证：500 次 bootstrap 给出单方法稳健性，三法 Jaccard 给出方法间一致性，化学合理性再以交叉相关的符号量级与酿造机理双重佐证。问题三对无监督生成以分布距离与降维簇重叠度评估，对迁移以缺陷阈值合规率这一硬指标评估，并用问题一感官画像核验风格特征。本题没有不公开测试集，因此所有数字都如实标注为交叉验证或分布距离的内部估计，不冒称测试集成绩。

此外，本文系统排查了六类泄漏并逐一封堵：分箱与标准化参数折内估计；主办方模型的参考重要性只作问题二旁证、绝不进入问题一训练特征；样本主键与风格标签不进入问题一特征矩阵（风格仅在问题三作条件）；不对风格做目标编码以免问题一走「风格到感官」的捷径。这些措施共同保证了报告指标的可信。

9.2 关键参数的灵敏度

本文对三问的关键假设与超参做了灵敏度考察，以判断结论对设定的依赖程度。问题一的 0-6 分箱口径（分位分箱与等距分箱）会改变各等级的样本数，从而影响 Kappa 的绝对量级：实测等距分箱的平均 QWK 为 0.4013，反而略高于分位分箱的 0.3663，但等距分箱在偏态标签上产生 8 个空等级、最小等级平均仅占 0.9% 的样本、等级计数变异系数高达 1.155，而分位分箱无空等级、各等级约占 14%、变异系数仅 0.022（见表 3）；为保证有序标签在各维上分布均衡、不出现空级，本文仍选用分位分箱。无论用哪种口径，各维度「谁更可预测」的相对排序稳定——酒精、苦味始终居前，芫荽、丁香始终接近噪声，说明可预测性结论稳健而绝对分级依赖映射口径，这一点已在结论中注明。对数变换底数经 5.3 节核实为混合尺度（37 列常用对数、33 列原值），因树模型对单调变换不敏感而不影响问题一与问题二的结论，仅影响浓度的叙述性换算。问题二的建图阈值在 0.4 至 0.6 之间扫描：阈值过高会使图断裂、模块过碎，过低会使图趋于全连通、模块度下降，取 0.5 时（211 节点、564 边、55 社区、模块度 0.625）结构最清晰；融合权重中 SHAP 占主导的设定保证了归因以预测贡献为锚。问题三的隐维取 8 至 16，过小会后验坍塌、各风格雷同，过大则在小样本上过拟合，取 16 并配合 KL 退火与 free-bits 后生成分布稳定、风格可分；缺陷约束还以无约束纯平移作对照（44 次检查中 32 次会升高，占 72.7%），证明 min 投影不可或缺。总体而言，三问的核心结论对合理范围内的参数变化不敏感，只有问题一的绝对分级值依赖分箱口径，已作诚实声明。

9.3 鲁棒性

在固定随机种子 42 的条件下，问题一的树模型结果完全可复现；问题二的 bootstrap 入选频率虽有抽样波动但 Top-1 选择始终稳定；问题三的条件生成在不同采样下保持风格可区分性。三问的资源利用充分——50 个感官维度全部获得预测、无闲置维度，231 个特征中问题一的重要性集中于少数化学信号列，符合「风味由少数关键化合物主导」的稀疏信号先验。

10 模型评价与推广

10.1 优点

本文方法的首要优点是以一个共享的化学到感官预测底座贯通三问，使归因与生成都建立在同一经过交叉验证的模型之上，避免了各问各立炉灶、口径不一的常见弊病。其次，数据预处理做了充分的实证核验：用 8.xlsx 的 70 列校准绝对浓度逐列反推对数底（37 列常用对数、33 列原值，pH 亦取常用对数），把「未检出」与 73 个真实缺失严格区分——未检出建指示变量、真实缺失折内 KNN 插补，不删任何样本；原生缺失建模在交叉验证上略优于插补对照，把缺失从障碍转化为可利用的信息。第三，归因严格

限定在 211 个单一化合物集 C 内（剔除理化与汇总列以贴合「找化合物」题意），多证据融合并引入共表达模块去冗余；稳定性除题面两种口径外，还以 10 万次随机抽取的 Jaccard 基线（0.0086）证明本文结果（0.186，为基线的 21.7 倍）显著，并对 Top-3 逐维做来源类别审计、如实报告完全一致率仅 14%。第四，生成与迁移以 KS 距离、风格分类器 Top-3 识别率（100%）、生成-真实画像 Spearman（0.68–0.93）三道相互独立的证据证明生成质量，并把「缺陷不恶化」做成 $\min(x_{before}, m_t)$ 的逐样本硬投影、配无约束纯平移对照（72.7% 本会升高），合规率 100% 且可直接审计。此外，Kaggle 公开集的外部旁证（跨风格酒精度排序 Spearman 0.84）也为本文风格规格符合行业常识提供了独立佐证。

10.2 不足与改进

本文仍有若干局限。其一，0-6 分级依赖对标准化连续值的单调分位映射重建，而非原始品评分卡的直接复现；实测等距分箱的 QWK 甚至略高于分位分箱，只是因等距产生空等级、等级失衡才选用分位，绝对分级值因此带有映射口径的印记。若能获得填好的原始品评数据，可改为直接的有序分类。其二，受小样本与 CPU 算力限制，条件变分自编码器的生成谱与真实谱仍有 0.36 至 0.41 的 KS 分布距离，提升空间主要在更大样本与更强的生成架构（如扩散模型）上。其三，问题二的 Top-3 来源类别完全一致率仅 14%，尤其麦芽感官维的归因大量落在酵母酯醇而非麦芽分子上——这是发酵后风味载体以酵母产物为主、麦芽焙烤化合物在成品中检出稀疏的真实反映，本文已如实标注而未粉饰，但要分离统计代理与因果分子仍需受控实验或更细的风格分层；外部 Kaggle 库与本文无共同主键、仅能作宏观旁证，无法逐行校准。

10.3 创新点

本文的创新集中在建模层面而非工具层面。其一，把问题一的预测模型同时用作问题二的 SHAP 归因对象与问题三生成谱的感官画像打分器，使三问形成「预测—归因—生成」真正联动的闭环，而非三个并列的独立任务。其二，归因阶段把候选集严格限定为 211 个单一化合物（剔除理化与汇总列），并引入共表达图的模块代表加权与同模块高相关去冗余，用化合物的网络结构去除共线性冗余，避免 Top-3 被同一化学模块占满，这比单纯的重要性排序更贴合化合物成簇的化学现实，也以 21.7 倍于随机基线的 Jaccard 验证了显著性。其三，把「缺陷不恶化」形式化为 $\min(\tilde{x}_{after}, x_{before}, m_t)$ 的逐样本投影，以源款原值为第一上界、目标典型值为第二道兜底，并配无约束纯平移对照——它修正了旧「取较大者」口径下 DMS 反而升高却宣称合规的矛盾，使迁移在数据驱动之外带上可核验的硬约束，合规率可直接审计。其四，用校准绝对浓度逐列反推对数底、将未检出与真实缺失分流并配套折内 KNN 与指示变量的预处理管道，为混合尺度、稀疏检出的化学谱提供了一套可核验、防泄漏的处理范式。

10.4 推广

本文框架可直接推广到其他「成分谱 → 感官或品质 → 品类」的问题，例如葡萄酒、咖啡、香精香料的配方分析与风格设计。含缺失的多输出有序预测适用于任何品评数据普遍存在缺测的场景；共表达图加多证据归因适用于高共线性的组学或光谱特征筛选；带硬约束的条件生成则可用于任何「按目标类别生成配方且需规避有害成分」的工业设计任务。

参考文献

[1] Zhang H. Predict and improve beer flavor with machine learning[J]. ResearchHub, 2026. DOI: 10.55277/researchhub.cagmlmpm.

[2] Li S, Li T. Machine learning discrimination and prediction of different quality grades of sauce-flavor Baijiu based on biomarker and key flavor compounds screening[J]. SSRN Electronic Journal, 2024. DOI: 10.2139/ssrn.4939729.

[3] Zhang C, Liu W. Effects of lactic acid bacteria fermentation on bioactive compounds and flavor profiles of Siraitia grosvenorii and prediction of flavor profiles by machine learning[J]. SSRN Electronic Journal, 2024. DOI: 10.2139/ssrn.6116637.

[4] Chen T, Guestrin C. XGBoost: A scalable tree boosting system[C]. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016: 785-794. DOI: 10.1145/2939672.2939785.

[5] Ke G, Meng Q, Finley T, et al. LightGBM: A highly efficient gradient boosting decision tree[C]. Advances in Neural Information Processing Systems (NeurIPS), 2017, 30: 3146-3154.

[6] Yıldız A Y, Kalayci A. Gradient boosting decision trees on medical diagnosis over tabular data[C]. 2025 IEEE International Conference on AI and Data Analytics (ICAD), 2025. DOI: 10.1109/icad65464.2025.11114069.

[7] Breiman L. Random forests[J]. Machine Learning, 2001, 45(1): 5-32. DOI: 10.1023/A:1010933404324.

[8] Troyanskaya O, Cantor M, Sherlock G, et al. Missing value estimation methods for DNA microarrays[J]. Bioinformatics, 2001, 17(6): 520-525. DOI: 10.1093/bioinformatics/17.6.520.

[9] Blondel V D, Guillaume J L, Lambiotte R, et al. Fast unfolding of communities in large networks[J]. Journal of Statistical Mechanics: Theory and Experiment, 2008, 2008(10): P10008. DOI: 10.1088/1742-5468/2008/10/p10008.

[10] Melo D, Pallares L F. Reassessing the modularity of gene co-expression networks using the stochastic block model[J]. PLoS Computational Biology, 2024, 20(7): e1012300. DOI: 10.1371/journal.pcbi.1012300.

[11] Clemente J B, Besas G. Predicting the biological classification of cell-cycle regulated genes of Saccharomyces cerevisiae using community detection algorithms on gene co-expression networks[C]. 2022.

[12] Gauraha N. Stability feature selection using cluster representative LASSO[C]. Proceedings of the 5th International Conference on Pattern Recognition Applications and Methods, 2016: 381-386. DOI: 10.5220/0005827003810386.

[13] Chaibi I, Njah H. Conditional variational autoencoder with Bayesian layers for robust tabular data generation from small datasets[J]. SSRN Electronic Journal, 2025. DOI: 10.2139/ssrn.5991727.

[14] Flexa C, Gomes W. Polygonal coordinate system: Visualizing high-dimensional data using geometric DR, and a deterministic version of t-SNE[J]. Expert Systems with Applications, 2021, 175: 114741. DOI: 10.1016/j.eswa.2021.114741.

[15] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning[J]. Nature Communications, 2024, 15: 2596. DOI: 10.1038/s41467-024-46346-0.

[16] Hoerl A E, Kennard R W. Ridge regression: Biased estimation for nonorthogonal problems[J]. Technometrics, 1970, 12(1): 55-67. DOI: 10.1080/00401706.1970.10488634.

[17] Wold S, Sjöström M, Eriksson L. PLS-regression: a basic tool of chemometrics[J]. Chemometrics and Intelligent Laboratory Systems, 2001, 58(2): 109-130. DOI: 10.1016/S0169-7439(01)00155-1.

[18] Polozoff S. Top Beer Information (Kaggle dataset)[DB/OL]. Kaggle, 2024. https://www.kaggle.com/datasets/stephenpolozoff/top-beer-information.

附录 A　软件环境与运行命令

本文全部计算使用 Python 3.11.9 完成，主要依赖库及版本为：numpy 2.4.6、pandas 2.3.3、scikit-learn 1.8.0、lightgbm 4.6.0、shap 0.51.0、networkx 3.6.1、python-louvain 0.16、torch 2.13.0（CPU 版）、scipy 1.17.1、matplotlib 3.10.8、joblib 1.5.3、openpyxl 3.1.5。随机种子固定为 42，全程仅使用相对路径。

在支撑材料根目录下，安装依赖并一键运行全部三问的命令为：

```
pip install -r code/requirements.txt
cd code
python main.py --data-dir ../user_data
```

程序依次执行问题一（感官分级预测）、问题二（关键化合物归因）、问题三（风格条件生成与迁移），结果文件写入 `submission/`，过程数据写入 `figures/`。

附录 B　支撑材料文件清单

路径	内容
`code/main.py`	一键入口，串联三问并汇总结果
`code/params.py`	全局常数（种子、维度、缺陷列、风格、超参）
`code/utils.py`	数据加载、0-6 分箱映射、二次加权 Kappa、检查点
`code/data_check.py`	数据读取与维度对齐自检
`code/problem1.py`	问题一：缺失感知梯度提升 + 分箱 + 分层交叉验证
`code/problem2.py`	问题二：化合物集 C=211 共表达图 + 社区检测 + 多证据 Top-3（去冗余）+ bootstrap/Jaccard + 来源判定 + 随机基线
`code/problem3.py`	问题三：CVAE 条件生成 + 风格识别率/画像评估 + 隐空间迁移 + min 缺陷投影 + 无约束对照
`code/constraint_audit.py`	从落盘结果重算硬约束
`code/sanity_check.py`	结果数值自动审查
`figures/supp_prep_eda.py`	预处理实证：附件概况/差异、对数底核实、未检出-缺失分类、三类划分
`figures/supp_kaggle.py`	Kaggle 公开集外部旁证（风格 ABV/IBU 宏观对照，不进建模）
`figures/gen_fig_*.py`、`figures/gen_tables.py`	全部图表与三线表的生成脚本（数值均读结果 JSON）
`figures/problem_1/2/3_results.json`、`all_results.json`	三问全部量化指标（QWK/Spearman/±1、图统计、来源类别、KS/识别率/缺陷 Δ）
`figures/supp_preprocess.json`、`supp_kaggle.json`	预处理实算结果与 Kaggle 旁证统计
`submission/q1_sensory_scores.csv`	问题一 250×50 维 0-6 分级

续下页

路径	内容
submission/q2_top3_compounds.csv	问题二每维 Top-3（仅化合物集 C）+ 稳定性 + 来源判定
submission/q3_generated_profiles.csv	问题三 90 条生成谱 + 11 条迁移 + 感官画像 + 缺陷 min 投影对照

附录 C　核心源程序

code/main.py

```python
# -*- coding: utf-8 -*-
"""一键入口：依次运行 Q1→Q2→Q3，并汇总 figures/all_results.json。

用法：
    python main.py --data-dir ../user_data        # 原始数据目录（默认 user_data）
结果：
    submission/q1_sensory_scores.csv  q2_top3_compounds.csv  q3_generated_profiles.csv
    figures/problem_{1,2,3}_results.json, figures/all_results.json
"""
import os
import sys
import json
import argparse

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
_ROOT = os.path.dirname(_HERE)

import numpy as np
import pandas as pd

import params as P
import utils as U
import problem1
import problem2
import problem3


def _default_data_dir():
    for cand in (os.path.join(_ROOT, "user_data"), os.path.join(_HERE, "..", "user_data"),
                 "user_data"):
        if os.path.isdir(cand):
            return cand
    return "user_data"

```

```python

def aggregate(out_dir):
    fig = os.path.join(out_dir, "figures")
    r1 = U.load_json(os.path.join(fig, "problem_1_results.json"))
    r2 = U.load_json(os.path.join(fig, "problem_2_results.json"))
    r3 = U.load_json(os.path.join(fig, "problem_3_results.json"))

    # 跨问一致性：Q2 高稳定维度数 <= Q1 可预测维度数
    n_pred = r1["summary"]["n_predictable_dims"]
    n_stable = r2["summary"]["n_high_stable_dims"]

    allr = {
        "problem_id": "A", "team_id": "MCB2600304",
        "seed": P.SEED,
        "q1": r1["summary"], "q2": r2["summary"], "q3": r3["summary"],
        # 逻辑合同回填键（供 logic_audit / cross_problem_check）
        "q3_defect_compliance_rate": r3["summary"]["defect_compliance_rate"],
        "gen_tripel_ester_level": r3["summary"]["gen_tripel_ester_level"],
        "real_tripel_ester_level": r3["summary"]["real_tripel_ester_level"],
        "logic_probes": {"bounds": [], "monotonic": []},
        "cross_problem": {
            "n_predictable_dims_Q1": n_pred,
            "n_high_stable_dims_Q2": n_stable,
            "Q2_stable_le_Q1_predictable": bool(n_stable <= n_pred),
        },
    }
    U.save_json(allr, os.path.join(fig, "all_results.json"))
    print(f"[main] 汇总写入 figures/all_results.json  "
          f"(Q1可预测维={n_pred}, Q2高稳定维={n_stable}, 合规率={allr['q3_defect_compliance_rate']})
    ")
    _update_ledger(out_dir, n_pred, n_stable, allr["q3_defect_compliance_rate"])
    return allr


def _update_ledger(out_dir, n_pred, n_stable, compliance):
    """把实际结果回填 CROSS_PROBLEM_LEDGER.json 的 observed。"""
    pth = os.path.join(out_dir, "CROSS_PROBLEM_LEDGER.json")
    if not os.path.exists(pth):
        return
    led = json.load(open(pth, encoding="utf-8"))
    for pb in led.get("problems", []):
        if pb["id"] == "Q1":
            pb["observed"] = {"n_predictable_dims": n_pred}
        elif pb["id"] == "Q2":
            pb["observed"] = {"n_high_stable_dims": n_stable}
        elif pb["id"] == "Q3":
            pb["observed"] = {"defect_compliance_rate": compliance}
    with open(pth, "w", encoding="utf-8") as f:
```

```
        json.dump(led, f, ensure_ascii=False, indent=2)
    print("[main] 已回填 CROSS_PROBLEM_LEDGER.json observed")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data-dir", default=None, help="原始数据目录（含 1.csv..8.xlsx）")
    ap.add_argument("--skip", default="", help="跳过的步骤，逗号分隔，如 q1,q2")
    args = ap.parse_args()
    data_dir = args.data_dir or _default_data_dir()
    skip = set(s.strip().lower() for s in args.skip.split(",") if s.strip())
    print(f"[main] data_dir={data_dir}  seed={P.SEED}")

    U.set_all_seeds(P.SEED)
    if "q1" not in skip:
        print("\n===== 问题一：含缺失多输出感官分级预测 =====")
        problem1.run(data_dir, _ROOT)
    if "q2" not in skip:
        print("\n===== 问题二：共表达图 + Top-3 + 稳定性 =====")
        problem2.run(data_dir, _ROOT)
    if "q3" not in skip:
        print("\n===== 问题三：CVAE 风格条件生成 + 迁移 =====")
        problem3.run(data_dir, _ROOT)
    print("\n===== 汇总 =====")
    aggregate(_ROOT)
    print("[main] 全部完成。")


if __name__ == "__main__":
    main()
```

code/params.py

```
# -*- coding: utf-8 -*-
"""全局参数与命名常数（赛道 A 啤酒风味）。

数值常数来源：DATA_FACTS.json（题面硬事实）+ MODELING_REPORT.md ⑧参数口径表。
行数等数据规模基准从 DATA_PROFILE.json 读取（若存在），禁止手写魔数行数。
本文件不 import _utils（评委机无该目录）。
"""
import os
import json

_HERE = os.path.dirname(os.path.abspath(__file__))
_ROOT = os.path.dirname(_HERE)

# ---------- 随机种子（可复现）----------
SEED = 42
```

```python

# ---------- 数据规模基准（优先取 DATA_PROFILE.json）----------
_PROFILE_PATH = os.path.join(_ROOT, "DATA_PROFILE.json")
N_SAMPLES = 250
N_FEATURES = 231
N_SENSORY = 50
try:
    if os.path.exists(_PROFILE_PATH):
        _prof = json.load(open(_PROFILE_PATH, encoding="utf-8"))["files"]
        N_SAMPLES = int(_prof["1.csv"]["total_rows"])
        N_FEATURES = int(_prof["2.csv"]["total_rows"])
except Exception:
    pass

# ---------- 元列（主键/分组/风格，不进特征矩阵）----------
META_COLS = ["beer", "beer_id", "tasting_category_fine"]
KEY_COL = "beer_id"
STYLE_COL = "tasting_category_fine"

# ---------- Q1: 分级与验证 ----------
N_LEVELS = 7            # 0..6，7 个有序等级（0-6 分制）
LEVEL_MIN, LEVEL_MAX = 0, 6
LABEL_BINNING = "quantile"   # {"quantile","uniform"}
N_FOLDS = 5
CV_SCHEME = "stratified_kfold"
# 核心 30 维（A_/F_ 前缀）；全 50 维一并交付
REPORT_SCOPE = "all50"

# ---------- Q2: 共表达图 + 归因 + 稳定性 ----------
TOPN = 3
COEXPR_RHO = 0.5              # 建图相关阈值
COEXPR_RHO_SENS = [0.4, 0.6]   # 灵敏度
N_BOOTSTRAP = 500
# 多证据融合权重 α*SHAP + β*|cross-corr| + γ*module-rep
FUSE_ALPHA, FUSE_BETA, FUSE_GAMMA = 0.5, 0.3, 0.2

# ---------- Q3: 条件生成 + 迁移 ----------
LATENT_DIM = 16               # 隐维 ∈ 8..16（小样本取上界增容量）
BETA_KL = 0.1                 # 小样本降低 KL 权重防后验坍塌（MODELING 异常③）
KL_ANNEAL_EPOCHS = 120        # KL 退火：前若干轮 β 线性 0→BETA_KL
FREE_BITS = 0.5               # free-bits：每维至少保留的 KL（nats），进一步防坍塌
CVAE_EPOCHS = 600
CVAE_LR = 1e-3
CVAE_BATCH = 64
MIN_STYLE_FOR_CVAE = 6        # 低于此样本量的风格走高斯兜底
TARGET_TYPICAL_Q = 0.5        # 目标风格缺陷典型分位（中位）
USE_ABS_THRESHOLD = False     # 8.xlsx 绝对阈值单位不可直接比，默认关闭

```

```python
GEN_STYLES = ["Tripel", "Kriek", "Lambic"]       # 代表风格
TRANSFER_SOURCES = ["Pils/Lager"]  # 迁移源（实际风格名，含斜杠）
TRANSFER_TARGET = "Tripel"

# 缺陷化合物 → 1.csv 列名（来自 DATA_FACTS.json given 段）
# Stale hops 无对应化学列（陈化标记），化学空间投影只作用于下列 4 列。
DEFECT_COL_MAP = {
    "DMS": "DMS",
    "Diacetyl": "X2.3.butanedione",
    "4VG": "X4VG",
    "trans-2-nonenal": "X.E..2.nonenal",
}
DEFECT_COLS = list(DEFECT_COL_MAP.values())
DEFECT_NO_CHEM = ["Stale hops"]   # 无化学列，仅记录

# 核心感官维前缀
CORE_PREFIXES = ("A_", "F_")

# 理化/过程参数列（16 个；非挥发性芳香化合物；来源 DATA_FACTS.json「ethanol/color/bitterness/pH/CO2 等理化列」）。
# Q1 特征矩阵保留这 16 列（理化对预测有用），仅 Q2 建图/归因时把它们从候选集剔除。
PHYSICOCHEM_COLS = {
    "SO2.mg.L.", "acetic_acid.g.L.", "ammonia.mg.L.", "color.EBC.", "betaglucan.mg.L.",
    "lactic_acid.mg.L.", "glycerol", "iron", "pH", "protein.g.L.", "bitterness", "GFS",
    "ethanol..v.v.", "CO2.PSI.", "unfermentables", "kcalperc",
}

# 聚合/派生列（4 个，由多个单独测量化合物加总：总酯、酒花香气总量、总酸、含硫）。
# 热量值 kcalperc 已归入 PHYSICOCHEM_COLS；这 4 个与理化列一并从「化合物集 C」剔除，
# 避免 Q2 归因把派生量当成某个独立化合物。
AGG_COLS = {"esters_sum", "aroma_hops_sum", "acids_sum", "sulfur_sum"}


def feature_type(name):
    return "physicochemical" if name in PHYSICOCHEM_COLS else "volatile_compound"


def compound_set(feat_cols):
    """从全部特征列中取「纯化合物集 C」：剔除 16 个理化列 + 4 个派生求和列。

    231 − 16 − 4 = 211 列。供 Q2/Q3 的化合物共表达与归因使用，不影响 Q1（Q1 仍用全 231 列）。
    """
    return [c for c in feat_cols if c not in PHYSICOCHEM_COLS and c not in AGG_COLS]


def core_sensory(sens_cols):
    """从 50 维中取核心 30 维（A_/F_ 前缀）。"""
    return [c for c in sens_cols if c.startswith(CORE_PREFIXES)]
```

code/utils.py

```python
# -*- coding: utf-8 -*-
"""公共工具：数据加载、0-6 分箱映射、QWK、随机种子、轻量检查点。

不 import _utils（评委机无该目录）。所有路径相对 --data-dir。
"""
import os
import sys
import json
import random

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
_ROOT = os.path.dirname(_HERE)

import numpy as np
import pandas as pd

import params as P


# ---------------- 随机种子 ----------------
def set_all_seeds(seed=P.SEED):
    random.seed(seed)
    np.random.seed(seed)
    try:
        import torch
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    except Exception:
        pass


# ---------------- 读取 + DATA_PROFILE 核对 ----------------
def _read_csv(path):
    for enc in ("utf-8", "utf-8-sig", "gbk", "gb2312", "latin-1"):
        try:
            return pd.read_csv(path, encoding=enc)
        except UnicodeDecodeError:
            continue
    raise RuntimeError(f"无法解码 {path}")


def _check_rows(fname, n):
```

```
    """若 DATA_PROFILE.json 存在则核对原始读取行数（评委机无此文件则跳过）。"""
    pth = os.path.join(_ROOT, "DATA_PROFILE.json")
    if not os.path.exists(pth):
        return
    prof = json.load(open(pth, encoding="utf-8"))["files"].get(fname, {})
    if "total_rows" in prof:
        assert n == prof["total_rows"], (
            f"[摄入不全] {fname} 实读 {n} 行 ≠ 建档 {prof['total_rows']} 行")
        print(f"[data_ingest] {fname} 全量 {n} 行 已核对通过")


def load_chem(data_dir):
    """1.csv: 返回 (df_full, X 特征矩阵DataFrame[含缺失], feat_cols, meta_df)。"""
    df = _read_csv(os.path.join(data_dir, "1.csv"))
    _check_rows("1.csv", len(df))
    feat_cols = [c for c in df.columns if c not in P.META_COLS]
    assert len(feat_cols) == P.N_FEATURES, f"特征数应 {P.N_FEATURES}，实 {len(feat_cols)}"
    X = df[feat_cols].apply(pd.to_numeric, errors="coerce")
    meta = df[P.META_COLS].copy()
    return df, X, feat_cols, meta


def load_sensory(data_dir):
    """4.csv: 返回 (Y 标签DataFrame[50维连续], sens_cols)。"""
    df = _read_csv(os.path.join(data_dir, "4.csv"))
    _check_rows("4.csv", len(df))
    sens_cols = [c for c in df.columns if c not in P.META_COLS]
    assert len(sens_cols) == P.N_SENSORY, f"感官维应 {P.N_SENSORY}，实 {len(sens_cols)}"
    Y = df[sens_cols].apply(pd.to_numeric, errors="coerce")
    return Y, sens_cols


def load_compound_corr(data_dir):
    """2.csv: 返回 (R 相关矩阵 np[231x231], comp_names)。与 1.csv 特征同序（位置对齐）。"""
    df = _read_csv(os.path.join(data_dir, "2.csv"))
    _check_rows("2.csv", len(df))
    comp_names = list(df.columns[1:])
    R = df.iloc[:, 1:].apply(pd.to_numeric, errors="coerce").values.astype(float)
    assert R.shape[0] == R.shape[1] == P.N_FEATURES, f"2.csv 应 {P.N_FEATURES} 方阵，实 {R.shape}"
    return R, comp_names


def load_cross_corr(data_dir):
    """6.csv: 返回 (C 交叉相关 DataFrame[44化合物 x 50感官], comp_rows, sens_cols6)。"""
    df = _read_csv(os.path.join(data_dir, "6.csv"))
    _check_rows("6.csv", len(df))
    comp_rows = df.iloc[:, 0].astype(str).tolist()
    sens_cols6 = list(df.columns[1:])
```

```
    C = df.iloc[:, 1:].apply(pd.to_numeric, errors="coerce")
    C.index = comp_rows
    return C, comp_rows, sens_cols6


def load_gbm_reference(data_dir):
    """8.xlsx: GBM importance MDI（参考重要性，仅 Q2 旁证，不进 Q1 训练）。显式 sheet_name。"""
    pth = os.path.join(data_dir, "8.xlsx")
    if not os.path.exists(pth):
        return None
    mdi = pd.read_excel(pth, sheet_name="GBM importance MDI")
    return mdi


# ---------------- 0-6 分箱映射 ----------------
class Binner:
    """对单个感官维，训练折内估分位点，映射连续值 -> 0..6 整数等级。"""

    def __init__(self, scheme=P.LABEL_BINNING, n_levels=P.N_LEVELS):
        self.scheme = scheme
        self.n_levels = n_levels
        self.edges = None   # 6 个内部切点

    def fit(self, y):
        y = np.asarray(y, dtype=float)
        y = y[np.isfinite(y)]
        q = self.n_levels  # 7 级 -> 6 个内部分位点
        if self.scheme == "quantile":
            probs = np.linspace(0, 1, q + 1)[1:-1]  # 6 点
            self.edges = np.quantile(y, probs)
        else:  # uniform 等距
            lo, hi = y.min(), y.max()
            self.edges = np.linspace(lo, hi, q + 1)[1:-1]
        # 去重保证单调（极端退化时）
        self.edges = np.maximum.accumulate(self.edges)
        return self

    def transform(self, y):
        y = np.asarray(y, dtype=float)
        lvl = np.searchsorted(self.edges, y, side="right")  # 0..6
        return np.clip(np.round(lvl), P.LEVEL_MIN, P.LEVEL_MAX).astype(int)

    def transform_continuous(self, r):
        """对回归连续输出施加同一映射 + clip + round（式2）。"""
        return self.transform(r)


def quadratic_weighted_kappa(y_true, y_pred, n_levels=P.N_LEVELS):
```

```
    """二次加权 Kappa（有序分级首选）。手写实现，避免 sklearn 对缺等级的边界问题。"""
    y_true = np.asarray(y_true, dtype=int)
    y_pred = np.asarray(y_pred, dtype=int)
    N = n_levels
    O = np.zeros((N, N), dtype=float)
    for a, b in zip(y_true, y_pred):
        O[a, b] += 1
    w = np.zeros((N, N), dtype=float)
    for i in range(N):
        for j in range(N):
            w[i, j] = (i - j) ** 2 / (N - 1) ** 2
    act = O.sum(axis=1)
    pred = O.sum(axis=0)
    E = np.outer(act, pred) / O.sum()
    denom = (w * E).sum()
    if denom == 0:
        return 1.0 if (w * O).sum() == 0 else 0.0
    return 1.0 - (w * O).sum() / denom


# ---------------- 轻量检查点（code/ 内自带，不依赖 _utils）----------------
def ckpt_path(name):
    d = os.path.join(_ROOT, "_tmp", "checkpoints")
    os.makedirs(d, exist_ok=True)
    return os.path.join(d, name)


def save_json(obj, path):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(obj, f, ensure_ascii=False, default=_json_default)
    os.replace(tmp, path)


def _json_default(o):
    import numpy as _np
    if isinstance(o, (_np.integer,)):
        return int(o)
    if isinstance(o, (_np.floating,)):
        return float(o)
    if isinstance(o, (_np.ndarray,)):
        return o.tolist()
    raise TypeError(f"not serializable: {type(o)}")


def load_json(path):
    with open(path, encoding="utf-8") as f:
```

```
        return json.load(f)
```

code/data_check.py

```
# -*- coding: utf-8 -*-
"""数据读取验证：确认 1/2/4/6.csv、8.xlsx 读取正确 + 231/50 维位置对齐。

先跑本脚本，确认无误再写求解代码。只打印结论，不打印整表。
"""
import os
import sys
import glob

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

import numpy as np
import pandas as pd


def _read_csv(path):
    for enc in ("utf-8", "utf-8-sig", "gbk", "gb2312", "latin-1"):
        try:
            return pd.read_csv(path, encoding=enc), enc
        except UnicodeDecodeError:
            continue
    raise RuntimeError(f"无法解码 {path}")


def main(data_dir="user_data"):
    print(f"[data_check] data_dir = {data_dir}")
    files = sorted(glob.glob(os.path.join(data_dir, "*.csv")))
    for f in files:
        df, enc = _read_csv(f)
        na_total = int(df.isnull().sum().sum())
        print(f"  {os.path.basename(f)}: shape={df.shape} enc={enc} NA总数={na_total}")

    # 位置对齐核对：1.csv 231 特征列 vs 2.csv 231 相关矩阵行/列
    d1, _ = _read_csv(os.path.join(data_dir, "1.csv"))
    d2, _ = _read_csv(os.path.join(data_dir, "2.csv"))
    d4, _ = _read_csv(os.path.join(data_dir, "4.csv"))
    d6, _ = _read_csv(os.path.join(data_dir, "6.csv"))

    meta = ["beer", "beer_id", "tasting_category_fine"]
    feat1 = [c for c in d1.columns if c not in meta]
    comp2 = list(d2.columns[1:])          # 去掉首列化合物名
    sens4 = [c for c in d4.columns if c not in meta]
```

```
    sens6 = list(d6.columns[1:])          # 去掉首列
    print(f"\n[对齐] 1.csv特征数={len(feat1)}  2.csv化合物数={len(comp2)}  "
          f"4.csv感官维={len(sens4)}  6.csv感官列={len(sens6)}  6.csv化合物行={d6.shape[0]}")
    assert len(feat1) == 231, f"1.csv 特征数应 231，实为 {len(feat1)}"
    assert len(comp2) == 231, f"2.csv 化合物数应 231，实为 {len(comp2)}"
    assert len(sens4) == 50, f"4.csv 感官维应 50，实为 {len(sens4)}"
    assert len(sens6) == 50, f"6.csv 感官列应 50，实为 {len(sens6)}"

    # 抽样核对位置对齐（1.csv短名 ↔ 2.csv全名，同序）
    pairs = [(3, "DMS", "Dimethyl sulfide"), (8, "ethyl_acetate", "Ethyl acetate")]
    for idx, n1, n2 in pairs:
        print(f"  pos{idx}: 1.csv='{feat1[idx]}' ({n1})  2.csv='{comp2[idx]}' ({n2})")

    # 缺陷列存在性
    defects = ["DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal"]
    for dcol in defects:
        print(f"  缺陷列 {dcol}: {'存在' if dcol in feat1 else '缺失!!'}")

    # 风格分布
    vc = d1["tasting_category_fine"].value_counts()
    print(f"\n[风格] 共 {vc.shape[0]} 类；最大 {vc.iloc[0]}({vc.index[0]}) 最小 {vc.iloc[-1]}({vc.
    index[-1]})")

    # 含缺失行统计
    na_rows = int(d1[feat1].isnull().any(axis=1).sum())
    na_total = int(d1[feat1].isnull().sum().sum())
    print(f"[缺失] 1.csv 含NA行={na_rows}/250  NA总数={na_total}")
    assert na_total == 73, f"NA 总数应 73，实为 {na_total}"
    assert na_rows == 63, f"含NA行应 63，实为 {na_rows}"

    # 8.xlsx 显式 sheet
    x8 = os.path.join(data_dir, "8.xlsx")
    if os.path.exists(x8):
        thr = pd.read_excel(x8, sheet_name="Sensory thresholds")
        mdi = pd.read_excel(x8, sheet_name="GBM importance MDI")
        print(f"[8.xlsx] Sensory thresholds={thr.shape}  GBM importance MDI={mdi.shape}")

    print("\n[data_check] 全部核对通过 ✓")


if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else "user_data")
```

code/problem1.py

```
# -*- coding: utf-8 -*-
"""问题一：含缺失的多输出有序感官分级预测（能力 P1-C1/C2/C3/C4）。

```

```
方法（忠实 MODELING_REPORT §六 / METHOD_CLAIMS M1-M3）：
- 主模型：原生支持缺失的梯度提升树 LightGBM（use_missing，不删含缺失行）。
- 标签：4.csv 标准化连续值 -> 训练折内 7 分位分箱 -> 0..6 整数等级（式1/式2）。
- 验证：按 tasting_category_fine 分层 5 折 CV，分箱/标准化参数仅训练折估计（防泄漏 P1-C4）。
- 基线：逐维均值等级、ElasticNet（中位插补）、随机森林（中位插补）。
- 缺失对照：KNN 插补 + LightGBM，量化插补偏差。
- 全量重训 -> submission/q1_sensory_scores.csv（250 行，beer_id + 50 维 0-6 整数，无 NA）。
"""
import os
import sys
import warnings

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
_ROOT = os.path.dirname(_HERE)

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

import params as P
import utils as U
from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import ElasticNet
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.metrics import mean_absolute_error, r2_score
import lightgbm as lgb

LGB_PARAMS = dict(
    n_estimators=150, num_leaves=8, learning_rate=0.05,
    min_child_samples=15, feature_fraction=0.6, bagging_fraction=0.8,
    bagging_freq=1, lambda_l2=1.0, random_state=P.SEED, n_jobs=-1, verbose=-1,
)


def _fit_lgb(Xtr, ytr):
    m = lgb.LGBMRegressor(**LGB_PARAMS)
    m.fit(Xtr, ytr)
    return m


def run(data_dir="user_data", out_dir=None):
    out_dir = out_dir or _ROOT
    U.set_all_seeds(P.SEED)
    fig_dir = os.path.join(out_dir, "figures")
```

```
    sub_dir = os.path.join(out_dir, "submission")
    os.makedirs(fig_dir, exist_ok=True)
    os.makedirs(sub_dir, exist_ok=True)

    df1, X, feat_cols, meta = U.load_chem(data_dir)
    Y, sens_cols = U.load_sensory(data_dir)
    styles = meta[P.STYLE_COL].values
    Xv = X.values.astype(float)           # 含 NaN，LightGBM 原生处理
    assert Xv.shape[0] == P.N_SAMPLES, "训练样本数必须=250（含缺失行，不删行）"

    n_na_rows = int(np.isnan(Xv).any(axis=1).sum())
    print(f"[Q1] 样本={Xv.shape[0]} 特征={Xv.shape[1]} 含NA行={n_na_rows} 感官维={len(sens_cols)}")

    # 分层折（小样本风格合并到 '_rare' 以保证分层可行）
    vc = pd.Series(styles).value_counts()
    strat = np.array([s if vc[s] >= P.N_FOLDS else "_rare" for s in styles])
    skf = StratifiedKFold(n_splits=P.N_FOLDS, shuffle=True, random_state=P.SEED)

    J = len(sens_cols)
    # 累加每维预测（out-of-fold）
    oof_main = np.full((P.N_SAMPLES, J), np.nan)
    oof_mean = np.full((P.N_SAMPLES, J), np.nan)
    oof_enet = np.full((P.N_SAMPLES, J), np.nan)
    oof_rf = np.full((P.N_SAMPLES, J), np.nan)
    oof_knn = np.full((P.N_SAMPLES, J), np.nan)        # KNN 插补+LGB 对照
    oof_true_lvl = np.full((P.N_SAMPLES, J), -1, dtype=int)
    oof_main_cont = np.full((P.N_SAMPLES, J), np.nan)  # 连续输出（R² 用）

    Yv = Y.values.astype(float)

    for fi, (tr, va) in enumerate(skf.split(Xv, strat), 1):
        print(f"[Q1] fold {fi}/{P.N_FOLDS}  train={len(tr)} val={len(va)}")
        # 折内中位插补器（仅 fit 训练折）供 ElasticNet/RF/KNN 基线
        med = SimpleImputer(strategy="median").fit(Xv[tr])
        knn = KNNImputer(n_neighbors=5).fit(Xv[tr])
        Xtr_med, Xva_med = med.transform(Xv[tr]), med.transform(Xv[va])
        Xtr_knn, Xva_knn = knn.transform(Xv[tr]), knn.transform(Xv[va])

        for j in range(J):
            ytr_c, yva_c = Yv[tr, j], Yv[va, j]
            binner = U.Binner().fit(ytr_c)                # 仅训练折估分箱
            oof_true_lvl[va, j] = binner.transform(yva_c)

            # 主模型：原生缺失 LightGBM（含缺失行训练）
            m = _fit_lgb(Xv[tr], ytr_c)
            rc = m.predict(Xv[va])
            oof_main_cont[va, j] = rc
            oof_main[va, j] = binner.transform_continuous(rc)
```

```
100
101             # 基线：均值等级
102             oof_mean[va, j] = binner.transform_continuous(np.full(len(va), ytr_c.mean()))
103             # 基线：ElasticNet（中位插补）
104             en = ElasticNet(alpha=0.1, l1_ratio=0.5, random_state=P.SEED, max_iter=5000)
105             en.fit(Xtr_med, ytr_c)
106             oof_enet[va, j] = binner.transform_continuous(en.predict(Xva_med))
107             # 基线：随机森林（中位插补）
108             rf = RandomForestRegressor(n_estimators=120, max_depth=6,
109                                        random_state=P.SEED, n_jobs=-1)
110             rf.fit(Xtr_med, ytr_c)
111             oof_rf[va, j] = binner.transform_continuous(rf.predict(Xva_med))
112             # 缺失对照：KNN 插补 + LGB
113             mk = _fit_lgb(Xtr_knn, ytr_c)
114             oof_knn[va, j] = binner.transform_continuous(mk.predict(Xva_knn))
115
116     # placeholder-metrics
117     results = _summarize(sens_cols, oof_true_lvl, oof_main, oof_mean,
118                          oof_enet, oof_rf, oof_knn, oof_main_cont, Yv, n_na_rows)
119     _full_retrain_and_submit(data_dir, out_dir, df1, Xv, Yv, feat_cols, sens_cols,
120                              meta, results)
121     build_figdata(results, Xv, oof_true_lvl, oof_main, oof_main_cont, Yv, sens_cols, feat_cols)
122     _validate(out_dir, sens_cols, results)
123     # 保存全量模型供 Q2 复用（可复现，不进 JSON）
124     import joblib
125     joblib.dump({"models": results.pop("_models"), "binners": results.pop("_binners"),
126                  "feat_cols": results.pop("_feat_cols")},
127                 U.ckpt_path("q1_models.pkl"))
128     U.save_json(results, os.path.join(fig_dir, "problem_1_results.json"))
129     print("[Q1] 完成，结果写入 figures/problem_1_results.json")
130     return results
131 def _summarize(sens_cols, true_lvl, main, mean_b, enet, rf, knn, main_cont, Yv, n_na_rows):
132     """逐维 QWK/MAE/R²，选择每维提交模型（主 vs 均值基线，取 QWK 高者）。"""
133     J = len(sens_cols)
134     per_dim = {}
135     qwk_main_list, qwk_mean_list = [], []
136     chosen = {}
137     for j, col in enumerate(sens_cols):
138         yt = true_lvl[:, j]
139         qwk_m = U.quadratic_weighted_kappa(yt, main[:, j].astype(int))
140         qwk_0 = U.quadratic_weighted_kappa(yt, mean_b[:, j].astype(int))
141         qwk_e = U.quadratic_weighted_kappa(yt, enet[:, j].astype(int))
142         qwk_r = U.quadratic_weighted_kappa(yt, rf[:, j].astype(int))
143         qwk_k = U.quadratic_weighted_kappa(yt, knn[:, j].astype(int))
144         mae_m = float(mean_absolute_error(yt, main[:, j]))
145         # R² 在连续回归输出上（标准回归口径）
146         r2_m = float(r2_score(Yv[:, j], main_cont[:, j]))
147         # 提交模型：主模型 CV QWK 不劣于均值基线则用主模型，否则用基线
```

```
        use_main = qwk_m >= qwk_0
        chosen[col] = "lightgbm" if use_main else "mean_baseline"
        per_dim[col] = {
            "qwk_main": round(qwk_m, 4), "qwk_mean_baseline": round(qwk_0, 4),
            "qwk_elasticnet": round(qwk_e, 4), "qwk_rf": round(qwk_r, 4),
            "qwk_knn_impute": round(qwk_k, 4),
            "mae_main": round(mae_m, 4), "r2_main": round(r2_m, 4),
            "chosen_model": chosen[col], "is_core": col.startswith(P.CORE_PREFIXES),
        }
        qwk_main_list.append(qwk_m)
        qwk_mean_list.append(qwk_0)

    qwk_main_arr = np.array(qwk_main_list)
    # 可预测维度：主模型 CV QWK > 0（供 Q2 跨问一致性）
    predictable = [sens_cols[j] for j in range(J) if qwk_main_arr[j] > 0]
    summary = {
        "n_samples": int(Yv.shape[0]), "n_features": P.N_FEATURES, "n_dims": J,
        "n_na_rows": int(n_na_rows),
        "mean_qwk_main": round(float(qwk_main_arr.mean()), 4),
        "mean_qwk_mean_baseline": round(float(np.mean(qwk_mean_list)), 4),
        "mean_qwk_elasticnet": round(float(np.mean([v["qwk_elasticnet"] for v in per_dim.values()]))
    , 4),
        "mean_qwk_rf": round(float(np.mean([v["qwk_rf"] for v in per_dim.values()])), 4),
        "mean_qwk_knn_impute": round(float(np.mean([v["qwk_knn_impute"] for v in per_dim.values()]))
    , 4),
        "n_predictable_dims": len(predictable),
        "n_main_chosen": int(sum(1 for v in chosen.values() if v == "lightgbm")),
        "n_dims_main_beats_baseline": int((qwk_main_arr > np.array(qwk_mean_list)).sum()),
    }
    print(f"[Q1] 平均QWK 主={summary['mean_qwk_main']} 均值基线={summary['mean_qwk_mean_baseline']}
    "
          f"ElasticNet={summary['mean_qwk_elasticnet']} RF={summary['mean_qwk_rf']} "
          f"KNN插补={summary['mean_qwk_knn_impute']}  可预测维={summary['n_predictable_dims']}")
    return {
        "problem": 1, "method": "LightGBM(native missing) + 7级分位分箱 + 分层5折CV",
        "summary": summary, "per_dim": per_dim,
        "predictable_dims": predictable, "sensory_cols": sens_cols,
        "chosen_model": chosen,
        "validation": {"scheme": "StratifiedKFold", "n_folds": P.N_FOLDS,
                       "binning": P.LABEL_BINNING, "leakage_control": "分箱/插补仅训练折fit"},
        # 校准图数据（代表维度：核心维中 QWK 最高者）
        "calibration": {},
        "fig_data": {},
    }
def _full_retrain_and_submit(data_dir, out_dir, df1, Xv, Yv, feat_cols, sens_cols, meta, results):
    """全部 250 样本重训（含缺失行），产出 0-6 分级提交文件。"""
    print("[Q1] 全量重训 + 生成 submission/q1_sensory_scores.csv ...")
    U.set_all_seeds(P.SEED)
```

```
    chosen = results["chosen_model"]
    sub = pd.DataFrame({P.KEY_COL: meta[P.KEY_COL].values})
    models = {}
    binners = {}
    for j, col in enumerate(sens_cols):
        yc = Yv[:, j]
        binner = U.Binner().fit(yc)        # 全量仅用于最终交付（CV 指标另由折内估计，无泄漏）
        binners[col] = binner
        if chosen[col] == "lightgbm":
            m = _fit_lgb(Xv, yc)
            models[col] = m
            pred = binner.transform_continuous(m.predict(Xv))
        else:
            pred = binner.transform_continuous(np.full(len(yc), yc.mean()))
        sub[col] = pred.astype(int)
    sub_path = os.path.join(out_dir, "submission", "q1_sensory_scores.csv")
    sub.to_csv(sub_path, index=False, encoding="utf-8-sig")
    print(f"[Q1] 写出 {sub_path}  shape={sub.shape}")
    results["_models"] = models          # 供 Q2 复用（不写 JSON）
    results["_binners"] = binners
    results["_feat_cols"] = feat_cols
    results["submission"] = {"path": "submission/q1_sensory_scores.csv",
                             "rows": int(sub.shape[0]), "cols": int(sub.shape[1])}


def build_figdata(results, Xv, oof_true_lvl, oof_main, oof_main_cont, Yv, sens_cols, feat_cols):
    """装配 paper-figure 用数据：缺失模式、QWK 排序、代表维校准。"""
    # 缺失模式：各列缺失率
    na_rate = np.isnan(Xv).mean(axis=0)
    na_cols = [(feat_cols[i], round(float(na_rate[i]), 4)) for i in range(len(feat_cols)) if na_rate
    [i] > 0]
    na_cols.sort(key=lambda t: -t[1])
    results["fig_data"]["missing_rate_by_col"] = na_cols
    results["fig_data"]["n_na_rows"] = int(np.isnan(Xv).any(axis=1).sum())
    # QWK 排序（棒棒糖图）
    qwk_sorted = sorted(((c, results["per_dim"][c]["qwk_main"]) for c in sens_cols),
                        key=lambda t: -t[1])
    results["fig_data"]["qwk_ranked"] = qwk_sorted
    # 代表维（核心维 QWK 最高）校准数据：真实等级 vs 预测连续
    core = [c for c in sens_cols if c.startswith(P.CORE_PREFIXES)]
    rep = max(core, key=lambda c: results["per_dim"][c]["qwk_main"])
    jr = sens_cols.index(rep)
    results["calibration"] = {
        "dim": rep,
        "true_level": oof_true_lvl[:, jr].tolist(),
        "pred_level": oof_main[:, jr].astype(int).tolist(),
        "true_cont": [round(float(v), 4) for v in Yv[:, jr]],
        "pred_cont": [round(float(v), 4) for v in oof_main_cont[:, jr]],
```

```
    }
    results["fig_data"]["rep_dim"] = rep


def _validate(out_dir, sens_cols, results):
    """validate_constraints + validate_capability（断言不过即 raise）。"""
    sub = pd.read_csv(os.path.join(out_dir, "submission", "q1_sensory_scores.csv"))
    vals = sub.drop(columns=[P.KEY_COL]).values
    # 硬约束：0-6 整数、无 NA、250 行、50 维
    assert sub.shape[0] == P.N_SAMPLES, f"q1 提交应 250 行，实 {sub.shape[0]}"
    assert len([c for c in sub.columns if c != P.KEY_COL]) == P.N_SENSORY
    assert not np.isnan(vals).any(), "q1 提交含 NA"
    assert (vals == vals.round()).all(), "q1 分级非整数"
    assert (vals >= P.LEVEL_MIN).all() and (vals <= P.LEVEL_MAX).all(), "q1 分级越界[0,6]"
    print("[Q1] validate_constraints PASS（0-6整数/无NA/250行/50维）")
    # 能力断言（P1-C2：含缺失行被使用，训练样本=250；P1-C4：折内估计）
    assert results["summary"]["n_samples"] == P.N_SAMPLES, \
        "[能力P1-C2不成立] 训练样本!=250，疑似删了含缺失行"
    assert results["summary"]["n_na_rows"] > 0, \
        "[能力P1-C2不成立] 未见含缺失行，疑似dropna"
    assert results["validation"]["leakage_control"].startswith("分箱"), \
        "[能力P1-C4不成立] 未做折内防泄漏"
    print("[Q1] validate_capability PASS（P1-C2 含缺失250行 / P1-C4 折内防泄漏）")


if __name__ == "__main__":
    dd = sys.argv[1] if len(sys.argv) > 1 else "user_data"
    run(dd)
```

code/problem2.py

```
# -*- coding: utf-8 -*-
"""问题二：化合物共表达图 + 多证据 Top-3 + 稳定性（P2-C1/C2/C3/C4）。

忠实 MODELING_REPORT §七 / METHOD_CLAIMS M4-M6:
- M4 共表达图：networkx 建图，边权=2.csv 相关系数，|w|>=rho，louvain 社区检测。
- M5 归因：SHAP(TreeExplainer) 对 Q1 LightGBM 模型 + 6.csv 交叉相关 + 模块代表 融合(式6)。
- M6 稳定性：bootstrap(B=500) 入选频率 + 跨方法(SHAP/相关/RF) Jaccard。
"""
import os
import sys
import warnings

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
_ROOT = os.path.dirname(_HERE)

```

```python
import numpy as np
import pandas as pd
warnings.filterwarnings("ignore")

import params as P
import utils as U
import networkx as nx
import shap
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer


def _norm(v):
    v = np.asarray(v, dtype=float)
    v = np.where(np.isfinite(v), v, 0.0)
    mx = np.abs(v).max()
    return v / mx if mx > 0 else v


def build_coexpr_graph(R, comp_names, rho, c_idx):
    """仅以化合物集 C 的位置索引（211 项）为节点、|corr|>=rho 且两端均∈C 为边建无向图 + louvain 社区。

    节点 id 沿用其在全 231 特征中的位置，便于与 comm_arr/评分向量对齐。
    """
    c_set = set(int(i) for i in c_idx)
    G = nx.Graph()
    for i in c_idx:
        G.add_node(int(i), name=comp_names[i])
    n_edges = 0
    c_list = sorted(c_set)
    for a in range(len(c_list)):
        i = c_list[a]
        for b in range(a + 1, len(c_list)):
            j = c_list[b]
            w = R[i, j]
            if np.isfinite(w) and abs(w) >= rho:
                # 边权用 |相关系数|（louvain/模块度要求非负权；符号由 R 保留，去冗余直接查 R）
                G.add_edge(i, j, weight=abs(float(w)))
                n_edges += 1
    # 社区检测（louvain；回退 greedy_modularity）
    try:
        import community as community_louvain
        part = community_louvain.best_partition(G, weight="weight", random_state=P.SEED)
        comm = {int(k): int(v) for k, v in part.items()}
        method = "louvain"
    except Exception:
        from networkx.algorithms.community import greedy_modularity_communities
```

```
        comms = greedy_modularity_communities(G, weight="weight")
        comm = {int(node): ci for ci, cset in enumerate(comms) for node in cset}
        method = "greedy_modularity"
    # 模块度
    try:
        from networkx.algorithms.community import modularity
        groups = {}
        for node, ci in comm.items():
            groups.setdefault(ci, set()).add(node)
        Q = modularity(G, list(groups.values()), weight="weight")
    except Exception:
        Q = None
    return G, comm, method, n_edges, Q


def run(data_dir="user_data", out_dir=None):
    out_dir = out_dir or _ROOT
    U.set_all_seeds(P.SEED)
    fig_dir = os.path.join(out_dir, "figures")
    sub_dir = os.path.join(out_dir, "submission")
    os.makedirs(fig_dir, exist_ok=True)
    os.makedirs(sub_dir, exist_ok=True)

    df1, X, feat_cols, meta = U.load_chem(data_dir)
    Y, sens_cols = U.load_sensory(data_dir)
    R, comp_names = U.load_compound_corr(data_dir)
    C6, comp_rows6, sens6 = U.load_cross_corr(data_dir)
    Xv = X.values.astype(float)
    Yv = Y.values.astype(float)
    J = len(sens_cols)

    # ---- 候选集 C（211 项）：理化 16 列 + 汇总 4 列一律排除（Q1 特征矩阵仍用全 231）----
    c_set = set(P.compound_set(feat_cols))
    c_mask = np.array([c in c_set for c in feat_cols])
    c_idx = [i for i, c in enumerate(feat_cols) if c in c_set]
    assert len(c_idx) == 211, f"化合物集 C 应 211 项，实 {len(c_idx)}"

    # 1) 共表达图 + 社区（仅 C）
    G, comm, cmethod, n_edges, Qmod = build_coexpr_graph(R, comp_names, P.COEXPR_RHO, c_idx)
    n_comm = len(set(comm.values()))
    print(f"[Q2] 共表达图(仅C) 节点={G.number_of_nodes()} 边={n_edges} 社区数={n_comm}({cmethod}) 模块度Q={Qmod}")
    comm_arr = np.array([comm.get(i, -1) for i in range(len(feat_cols))])

    # 2) SHAP 证据（对 Q1 全量 LightGBM 模型）
    shap_imp, abs_shap_list, _ = _shap_evidence(sens_cols, feat_cols, Xv)

    # 3) 6.csv 交叉相关证据（映射到 231 位置）
```

```
    corr_evid = _corr_evidence(C6, comp_rows6, comp_names, feat_cols, sens_cols)

    # 4) RF 重要性证据（跨方法用）
    rf_imp = _rf_evidence(Xv, Yv, J)

    # 5) 融合 + 模块代表 + 同模块去冗余(|R|>=0.7) → Top-3；稳定性（全部只在 C 内取）
    results = _fuse_and_stability(sens_cols, feat_cols, shap_imp, corr_evid, rf_imp,
                                  abs_shap_list, comm_arr, c_mask, R)
    # 6) 来源类别一致性（8.xlsx 酵母/酒花/麦芽/香料/野生菌群）
    src_map, cat_map = _load_source_maps(data_dir, feat_cols, sens_cols)
    _attach_source_category(results, sens_cols, feat_cols, src_map, cat_map)
    # 7) 随机 Jaccard 基线（从 C 抽 3 重复 100000 次）
    results["random_jaccard_baseline"] = _random_jaccard_baseline(results, sens_cols, c_idx, J)

    # 6) 交付 + 图数据 + 验证
    _write_submission(out_dir, results, sens_cols, feat_cols, corr_evid, comm_arr)
    results["graph"] = {"n_nodes": G.number_of_nodes(), "n_edges": n_edges,
                        "n_communities": n_comm, "method": cmethod,
                        "modularity": round(float(Qmod), 4) if Qmod is not None else None,
                        "rho": P.COEXPR_RHO}
    _build_graph_figdata(results, G, comm, comp_names, R)
    _validate(out_dir, results, sens_cols)
    U.save_json(results, os.path.join(fig_dir, "problem_2_results.json"))
    print("[Q2] 完成，结果写入 figures/problem_2_results.json")
    return results
def _shap_evidence(sens_cols, feat_cols, Xv):
    """用 Q1 全量 LightGBM 模型算每维 SHAP。返回 (importance[J×P], 每维|shap|矩阵列表)。"""
    ck = U.ckpt_path("q1_models.pkl")
    assert os.path.exists(ck), "缺 q1_models.pkl：请先跑 problem1.py"
    import joblib
    bundle = joblib.load(ck)
    models = bundle["models"]
    J, Pn = len(sens_cols), len(feat_cols)
    imp = np.zeros((J, Pn))
    abs_shap_list = []
    for j, col in enumerate(sens_cols):
        if col in models:
            expl = shap.TreeExplainer(models[col])
            sv = expl.shap_values(Xv)       # N×P
            sv = np.asarray(sv)
            abs_sv = np.abs(sv)
            imp[j] = abs_sv.mean(axis=0)
            abs_shap_list.append(abs_sv)
        else:
            # 该维提交基线（无 LightGBM 模型）：SHAP 证据置 0，依赖相关/RF
            abs_shap_list.append(np.zeros((Xv.shape[0], Pn)))
        if (j + 1) % 10 == 0:
            print(f"[Q2] SHAP 进度 {j+1}/{J}")
```

```
    return imp, abs_shap_list, sens_cols


def _corr_evidence(C6, comp_rows6, comp_names, feat_cols, sens_cols):
    """6.csv 交叉相关 → 231 位置 × 50 感官的 |corr| 证据（未覆盖化合物记 0）。"""
    J, Pn = len(sens_cols), len(feat_cols)
    evid = np.zeros((J, Pn))
    # 6.csv 行名(全名) → 2.csv 位置
    name2pos = {name: i for i, name in enumerate(comp_names)}
    # 6.csv 感官列与 4.csv 50 维按位置对齐
    row_pos = {r: name2pos.get(r, None) for r in comp_rows6}
    Cv = C6.values.astype(float)   # 44 × 50
    n_mapped = 0
    for ri, r in enumerate(comp_rows6):
        pos = row_pos[r]
        if pos is None:
            continue
        n_mapped += 1
        for j in range(J):
            v = Cv[ri, j]
            evid[j, pos] = abs(v) if np.isfinite(v) else 0.0
    print(f"[Q2] 6.csv 交叉相关映射化合物 {n_mapped}/{len(comp_rows6)} → 231 位置")
    return evid


def _rf_evidence(Xv, Yv, J):
    """随机森林重要性（中位插补）作为跨方法证据。"""
    imp = np.zeros((J, Xv.shape[1]))
    med = SimpleImputer(strategy="median").fit(Xv)
    Xm = med.transform(Xv)
    for j in range(J):
        rf = RandomForestRegressor(n_estimators=120, max_depth=6,
                                   random_state=P.SEED, n_jobs=-1)
        rf.fit(Xm, Yv[:, j])
        imp[j] = rf.feature_importances_
        if (j + 1) % 10 == 0:
            print(f"[Q2] RF 证据 {j+1}/{J}")
    return imp


def _module_rep(base_scores, comm_arr):
    """模块代表加权：同社区内按 base 排名，第 1 名 rep=1，第 2=1/2…（促进 Top-3 跨模块多样）。"""
    rep = np.zeros_like(base_scores)
    for ci in np.unique(comm_arr):
        idx = np.where(comm_arr == ci)[0]
        order = idx[np.argsort(-base_scores[idx])]
        for rank, p in enumerate(order, 1):
            rep[p] = 1.0 / rank
```

```
    return rep
def _topk(score, k=P.TOPN):
    return list(np.argsort(-score)[:k])


def _jaccard(a, b):
    a, b = set(a), set(b)
    u = a | b
    return len(a & b) / len(u) if u else 0.0


def _fuse_and_stability(sens_cols, feat_cols, shap_imp, corr_evid, rf_imp, abs_shap_list, comm_arr):
    J, Pn = len(sens_cols), len(feat_cols)
    a, b, g = P.FUSE_ALPHA, P.FUSE_BETA, P.FUSE_GAMMA
    per_dim = {}
    top3_idx = {}
    # 点估计
    shap_n_all = np.vstack([_norm(shap_imp[j]) for j in range(J)])
    corr_n_all = np.vstack([_norm(corr_evid[j]) for j in range(J)])
    rf_n_all = np.vstack([_norm(rf_imp[j]) for j in range(J)])
    phi_all = np.zeros((J, Pn))
    for j in range(J):
        base = a * shap_n_all[j] + b * corr_n_all[j]
        rep = _module_rep(base, comm_arr)
        phi = base + g * rep
        phi_all[j] = phi
        top3_idx[j] = _topk(phi)
    # bootstrap 入选频率（有放回重采样样本 → 重算 SHAP 重要性 → 重算 Top-3）
    np.random.seed(P.SEED)                # 固定种子保证可复现
    N = abs_shap_list[0].shape[0]
    freq = np.zeros((J, Pn))
    B = P.N_BOOTSTRAP
    for bi in range(B):
        idx = np.random.choice(N, size=N, replace=True)   # 有放回重采样
        for j in range(J):
            shap_b = _norm(abs_shap_list[j][idx].mean(axis=0))
            base_b = a * shap_b + b * corr_n_all[j]
            phi_b = base_b + g * _module_rep(base_b, comm_arr)
            for p in _topk(phi_b):
                freq[j, p] += 1
        if (bi + 1) % 100 == 0:
            print(f"[Q2] bootstrap {bi+1}/{B}")
    freq /= B
    # 跨方法 Jaccard（SHAP / 6.csv相关 / RF）
    jac_dim = {}
    for j in range(J):
        ts = _topk(shap_n_all[j]); tc = _topk(corr_n_all[j]); tr = _topk(rf_n_all[j])
        pj = np.mean([_jaccard(ts, tc), _jaccard(ts, tr), _jaccard(tc, tr)])
```

```
        jac_dim[sens_cols[j]] = round(float(pj), 4)

    for j, col in enumerate(sens_cols):
        t3 = top3_idx[j]
        per_dim[col] = {
            "top3": [feat_cols[p] for p in t3],
            "top3_idx": [int(p) for p in t3],
            "phi": [round(float(phi_all[j, p]), 4) for p in t3],
            "shap_norm": [round(float(shap_n_all[j, p]), 4) for p in t3],
            "corr_abs": [round(float(corr_n_all[j, p]), 4) for p in t3],
            "select_freq": [round(float(freq[j, p]), 4) for p in t3],
            "module": [int(comm_arr[p]) for p in t3],
            "jaccard_cross_method": jac_dim[col],
        }
    mean_top1_freq = float(np.mean([per_dim[c]["select_freq"][0] for c in sens_cols]))
    mean_jac = float(np.mean(list(jac_dim.values())))
    # 高稳定维度数（Top-1 入选频率>=0.7）
    n_high_stable = int(sum(1 for c in sens_cols if per_dim[c]["select_freq"][0] >= 0.7))
    print(f"[Q2] 平均Top-1入选频率={mean_top1_freq:.3f} 平均跨方法Jaccard={mean_jac:.3f} "
          f"高稳定维={n_high_stable}")
    return {
        "problem": 2,
        "method": "共表达图(networkx+louvain) + SHAP/相关/模块代表融合 + bootstrap频率 + 跨方法
    Jaccard",
        "summary": {"n_dims": J, "n_bootstrap": B, "topn": P.TOPN,
                    "mean_top1_select_freq": round(mean_top1_freq, 4),
                    "mean_cross_method_jaccard": round(mean_jac, 4),
                    "n_high_stable_dims": n_high_stable,
                    "fuse_weights": {"alpha": a, "beta": b, "gamma": g}},
        "per_dim": per_dim, "sensory_cols": sens_cols,
    }
_MECH_RULES = [
    (("ethyl_", "ethyl.", "acetate", "isoaa", "ester", "lactate", "butyrate",
      "octanoate", "hexanoate", "propionate", "decanoate"), "酯类→果香/香蕉/溶剂气味"),
    (("linalool", "geraniol", "citronellol", "terpine", "ocimene", "humulene",
      "caryophyll", "farnes", "nerol", "limonene", "phellandrene", "myrcene"),
     "萜烯类→酒花花香/柑橘/木质香"),
    (("dms", "sulfur", "h2s", "mesh", "des", "cs2", "thiol", "sulfide"),
     "硫化物/DMS→蔬菜/硫味（低阈缺陷）"),
    (("butanedione", "diacetyl"), "双乙酰→黄油味缺陷"),
    (("x4vg", "guaiacol", "vinyl", "phenol", "cresol"), "酚类(4VG)→丁香/药味"),
    (("nonenal", "decadienal", "aldehyde", "furfural", "methylbutanal"),
     "醛类→老化/青草/纸板气味"),
    (("malt", "furan", "maltol", "pyrazine", "isomaltol", "vanillin"),
     "麦芽/美拉德→焦糖/面包/烘烤"),
    (("ethanol", "isoamyl_alcohol", "isobutanol", "propanol", "alcohol"),
     "高级醇/乙醇→酒精感"),
    (("acid", "acetic", "lactic"), "有机酸→酸味"),
```

```
]


def _mechanism(feat_name):
    if feat_name in P.PHYSICOCHEM_COLS:
        return "理化/过程指标（非挥发性化合物），反映风格整体工艺特征，常与风格混杂相关"
    low = feat_name.lower()
    for keys, txt in _MECH_RULES:
        if any(k in low for k in keys):
            return txt
    return "其他挥发性化学信号（见共表达模块）"


def _write_submission(out_dir, results, sens_cols, feat_cols, corr_evid, comm_arr):
    rows = []
    for col in sens_cols:
        d = results["per_dim"][col]
        for rank, (fn, phi, sh, co, fr, mod) in enumerate(
                zip(d["top3"], d["phi"], d["shap_norm"], d["corr_abs"],
                    d["select_freq"], d["module"]), 1):
            rows.append({
                "sensory_dim": col, "rank": rank, "compound": fn,
                "phi_score": phi, "shap_norm": sh, "cross_corr_abs": co,
                "bootstrap_select_freq": fr, "module_id": mod,
                "jaccard_cross_method": d["jaccard_cross_method"],
                "feature_type": P.feature_type(fn),
                "mechanism": _mechanism(fn),
            })
    sub = pd.DataFrame(rows)
    path = os.path.join(out_dir, "submission", "q2_top3_compounds.csv")
    sub.to_csv(path, index=False, encoding="utf-8-sig")
    print(f"[Q2] 写出 {path}  shape={sub.shape}（每维 {P.TOPN} 行）")
    results["submission"] = {"path": "submission/q2_top3_compounds.csv",
                             "rows": int(sub.shape[0])}


def _build_graph_figdata(results, G, comm, comp_names, R):
    """为网络图提供布局数据（取最大连通子图 + 节点度/社区）；只存前若干边避免过大。"""
    deg = dict(G.degree(weight=None))
    # 节点度排序，取度数前 60 个节点做可视化子图
    top_nodes = sorted(deg, key=lambda n: -deg[n])[:60]
    sub = G.subgraph(top_nodes)
    nodes = [{"id": int(n), "name": comp_names[n], "deg": int(deg[n]),
              "comm": int(comm.get(n, -1))} for n in sub.nodes()]
    edges = [{"s": int(u), "t": int(v), "w": round(float(d["weight"]), 3)}
             for u, v, d in sub.edges(data=True)]
    comm_sizes = {}
    for n, ci in comm.items():
```

```
        comm_sizes[ci] = comm_sizes.get(ci, 0) + 1
    results["fig_data"] = {
        "network_nodes": nodes, "network_edges": edges,
        "community_sizes": sorted(comm_sizes.values(), reverse=True),
        "top1_freq_ranked": sorted(
            ((c, results["per_dim"][c]["select_freq"][0],
              results["per_dim"][c]["jaccard_cross_method"]) for c in results["sensory_cols"]),
            key=lambda t: -t[1]),
    }


def _validate(out_dir, results, sens_cols):
    sub = pd.read_csv(os.path.join(out_dir, "submission", "q2_top3_compounds.csv"))
    # 硬约束：每维恰 3 行
    sizes = sub.groupby("sensory_dim").size()
    assert (sizes == P.TOPN).all(), f"[P2-C2] 存在维度 Top-N!=3: {sizes[sizes!=P.TOPN].to_dict()}"
    assert sizes.shape[0] == len(sens_cols), "Top-3 覆盖维度数!=50"
    # 频率/Jaccard ∈[0,1]
    for c in ("bootstrap_select_freq", "jaccard_cross_method"):
        assert sub[c].between(0, 1).all(), f"[P2-C3] {c} 越界[0,1]"
    print("[Q2] validate_constraints PASS（每维3行/频率∈[0,1]/Jaccard∈[0,1]）")
    # 能力断言：P2-C3 bootstrap>=200 + 跨方法>=2；P2-C4 每维有机理
    assert results["summary"]["n_bootstrap"] >= 200, "[P2-C3] bootstrap<200"
    assert sub["mechanism"].notna().all() and (sub["mechanism"].str.len() > 0).all(), \
        "[P2-C4] 存在缺机理解释的 Top-3"
    assert results["summary"]["mean_cross_method_jaccard"] >= 0, "[P2-C3] 跨方法Jaccard缺失"
    print("[Q2] validate_capability PASS（P2-C3 500次bootstrap+3方法Jaccard / P2-C4 逐维机理）")


if __name__ == "__main__":
    dd = sys.argv[1] if len(sys.argv) > 1 else "user_data"
    run(dd)
```

code/problem3.py

```
# -*- coding: utf-8 -*-
"""问题三：风格条件生成 CVAE + 跨风格迁移（缺陷不恶化）（P3-C1/C2/C3）。

忠实 MODELING_REPORT §八 / METHOD_CLAIMS M7-M8:
- M7 CVAE: torch nn.Module，重参数化 mu/logvar + KL 散度，风格 one-hot 为条件，
          缺失位置用掩码排除重构损失（式10/11）。
- M8 迁移：隐空间风格平移 Δ=z̄_t−z̄_s + 对缺陷列逐样本 min/clip 投影（式12/13）。
- 极小样本风格走按风格多元高斯兜底；感官画像复用 Q1 模型（闭环）。
"""
import os
import sys
import warnings

```

```
_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
_ROOT = os.path.dirname(_HERE)

import numpy as np
import pandas as pd
warnings.filterwarnings("ignore")

import params as P
import utils as U
import torch
import torch.nn as nn
from scipy.stats import ks_2samp


class CVAE(nn.Module):
    def __init__(self, n_feat, n_style, latent=P.LATENT_DIM, hidden=128):
        super().__init__()
        self.n_feat = n_feat
        self.enc = nn.Sequential(
            nn.Linear(n_feat + n_style, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU())
        self.fc_mu = nn.Linear(hidden, latent)
        self.fc_lv = nn.Linear(hidden, latent)
        self.dec = nn.Sequential(
            nn.Linear(latent + n_style, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, n_feat))

    def encode(self, x, s):
        h = self.enc(torch.cat([x, s], dim=1))
        return self.fc_mu(h), self.fc_lv(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z, s):
        return self.dec(torch.cat([z, s], dim=1))

    def forward(self, x, s):
        mu, logvar = self.encode(x, s)
        z = self.reparameterize(mu, logvar)
        return self.decode(z, s), mu, logvar


```

```python
def _elbo_loss(xhat, x, mask, mu, logvar, beta_kl, free_bits=P.FREE_BITS):
    # 掩码重构（只对非缺失分量计 MSE，式11）
    diff2 = ((xhat - x) ** 2) * mask
    recon = diff2.sum() / mask.sum().clamp(min=1.0)
    # 逐维 KL + free-bits（每维 KL 低于 free_bits 时不再惩罚，防后验坍塌）
    kld_dim = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp())   # [B, d]
    kld_dim = torch.clamp(kld_dim.mean(dim=0), min=free_bits)
    kld = kld_dim.sum()
    return recon + beta_kl * kld, recon.item(), float(kld.item())


def run(data_dir="user_data", out_dir=None):
    out_dir = out_dir or _ROOT
    U.set_all_seeds(P.SEED)
    torch.manual_seed(P.SEED)
    fig_dir = os.path.join(out_dir, "figures")
    sub_dir = os.path.join(out_dir, "submission")
    os.makedirs(fig_dir, exist_ok=True)
    os.makedirs(sub_dir, exist_ok=True)

    df1, X, feat_cols, meta = U.load_chem(data_dir)
    styles = meta[P.STYLE_COL].values
    Xv = X.values.astype(float)
    mask = (~np.isnan(Xv)).astype(float)       # 1=观测 0=缺失

    # 标准化（缺失先按列中位临时填，仅供编码器输入；重构损失用 mask 排除）
    col_med = np.nanmedian(Xv, axis=0)
    Xfill = np.where(np.isnan(Xv), col_med, Xv)
    mu_f = Xfill.mean(axis=0)
    sd_f = Xfill.std(axis=0) + 1e-8
    Xstd = (Xfill - mu_f) / sd_f

    style_list = sorted(pd.unique(styles).tolist())
    s2i = {s: i for i, s in enumerate(style_list)}
    S = len(style_list)
    onehot = np.eye(S)[np.array([s2i[s] for s in styles])]

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"[Q3] device={device} 样本={Xv.shape[0]} 特征={Xv.shape[1]} 风格={S} 隐维={P.LATENT_DIM}"
    )

    model = _train_cvae(Xstd, mask, onehot, device)

    # 生成代表风格谱
    gen_records, gen_native = _generate(model, style_list, s2i, S, Xstd, styles,
                                        mu_f, sd_f, device)
    # 迁移 Pils/Lager -> Tripel + 缺陷投影
    transfer_records, transfer_info = _transfer(model, style_list, s2i, S, Xstd, Xv,
```

```
                                                styles, feat_cols, mu_f, sd_f, device)
    # 感官画像（Q1 模型）
    sens_pred_gen, sens_cols = _sensory_profile(gen_native, feat_cols)
    sens_pred_tr, _ = _sensory_profile(
        np.array([r["_native"] for r in transfer_records]), feat_cols)

    # 验证指标
    metrics = _eval_metrics(Xv, styles, feat_cols, gen_native, gen_records, transfer_info)

    results = _assemble(out_dir, data_dir, feat_cols, sens_cols, gen_records, gen_native,
                        sens_pred_gen, transfer_records, sens_pred_tr, transfer_info,
                        metrics, Xv, styles)
    _validate(out_dir, results)
    U.save_json(results, os.path.join(fig_dir, "problem_3_results.json"))
    print("[Q3] 完成，结果写入 figures/problem_3_results.json")
    return results
def _train_cvae(Xstd, mask, onehot, device):
    model = CVAE(Xstd.shape[1], onehot.shape[1]).to(device)
    opt = torch.optim.Adam(model.parameters(), lr=P.CVAE_LR)
    Xt = torch.tensor(Xstd, dtype=torch.float32, device=device)
    Mt = torch.tensor(mask, dtype=torch.float32, device=device)
    St = torch.tensor(onehot, dtype=torch.float32, device=device)
    N = Xt.shape[0]
    best, best_state, patience, bad = 1e18, None, 60, 0
    rng = np.random.default_rng(P.SEED)
    for ep in range(1, P.CVAE_EPOCHS + 1):
        model.train()
        perm = rng.permutation(N)
        ep_loss = ep_rec = ep_kl = 0.0
        for st in range(0, N, P.CVAE_BATCH):
            bidx = perm[st:st + P.CVAE_BATCH]
            xb, mb, sb = Xt[bidx], Mt[bidx], St[bidx]
            opt.zero_grad()
            xhat, mu, lv = model(xb, sb)
            beta = P.BETA_KL * min(1.0, ep / max(1, P.KL_ANNEAL_EPOCHS))  # KL 退火
            loss, rec, kl = _elbo_loss(xhat, xb, mb, mu, lv, beta)
            loss.backward()
            opt.step()
            ep_loss += loss.item() * len(bidx); ep_rec += rec * len(bidx); ep_kl += kl * len(bidx)
        ep_loss /= N
        if ep <= P.KL_ANNEAL_EPOCHS:
            if ep % 50 == 0 or ep == 1:
                print(f"[Q3] CVAE epoch {ep}/{P.CVAE_EPOCHS}(退火) loss={ep_loss:.4f} "
                      f"recon={ep_rec/N:.4f} kl={ep_kl/N:.4f}")
            continue   # 退火阶段不计入早停
        if ep_loss < best - 1e-4:
            best, best_state, bad = ep_loss, {k: v.detach().clone() for k, v in model.state_dict().
    items()}, 0
```

```
        else:
            bad += 1
        if ep % 50 == 0 or ep == 1:
            print(f"[Q3] CVAE epoch {ep}/{P.CVAE_EPOCHS} loss={ep_loss:.4f} "
                  f"recon={ep_rec/N:.4f} kl={ep_kl/N:.4f}")
        if bad >= patience:
            print(f"[Q3] 早停于 epoch {ep}（best loss={best:.4f}）")
            break
    if best_state is not None:
        model.load_state_dict(best_state)
    model.eval()
    return model


def _style_latent_mean(model, Xstd, styles, target_style, s2i, S, device):
    """编码目标风格样本得隐均值 z̄_c（迁移用）。"""
    idx = np.where(styles == target_style)[0]
    oh = np.zeros((len(idx), S)); oh[:, s2i[target_style]] = 1
    with torch.no_grad():
        mu, _ = model.encode(torch.tensor(Xstd[idx], dtype=torch.float32, device=device),
                             torch.tensor(oh, dtype=torch.float32, device=device))
    return mu.cpu().numpy().mean(axis=0), idx


def _generate(model, style_list, s2i, S, Xstd, styles, mu_f, sd_f, device, n_gen=30):
    """代表风格条件采样生成；小样本风格走高斯兜底。"""
    rng = np.random.default_rng(P.SEED)
    records, native_rows = [], []
    for style in P.GEN_STYLES:
        if style not in s2i:
            print(f"[Q3] 跳过未知风格 {style}")
            continue
        cnt = int((styles == style).sum())
        if cnt >= P.MIN_STYLE_FOR_CVAE:
            # CVAE：z~N(0,I) 解码
            z = torch.tensor(rng.normal(size=(n_gen, P.LATENT_DIM)), dtype=torch.float32, device=
    device)
            oh = np.zeros((n_gen, S)); oh[:, s2i[style]] = 1
            with torch.no_grad():
                xhat = model.decode(z, torch.tensor(oh, dtype=torch.float32, device=device)).cpu().
    numpy()
            gen_std = xhat
            gmethod = "CVAE"
        else:
            # 高斯兜底：按风格样本多元高斯采样（标准化空间）
            idx = np.where(styles == style)[0]
            m = Xstd[idx].mean(axis=0); cov = np.cov(Xstd[idx].T) + 1e-3 * np.eye(Xstd.shape[1])
            gen_std = rng.multivariate_normal(m, cov, size=n_gen)
```

```
            gmethod = "gaussian_fallback"
        native = gen_std * sd_f + mu_f
        for k in range(n_gen):
            records.append({"style": style, "sample_type": "generated",
                            "gen_method": gmethod, "_native": native[k]})
            native_rows.append(native[k])
        print(f"[Q3] 生成 {style}: {n_gen} 条（{gmethod}，真实样本={cnt}）")
    return records, np.array(native_rows)
def _transfer(model, style_list, s2i, S, Xstd, Xv, styles, feat_cols, mu_f, sd_f, device):
    """Pils/Lager → Tripel 隐空间平移 + 缺陷投影（式12/13）。"""
    tgt = P.TRANSFER_TARGET
    zt, _ = _style_latent_mean(model, Xstd, styles, tgt, s2i, S, device)
    # 目标风格缺陷典型值 m^t（native 中位）
    tgt_idx = np.where(styles == tgt)[0]
    col_med_all = np.nanmedian(Xv, axis=0)
    defect_pos = {d: feat_cols.index(c) for d, c in P.DEFECT_COL_MAP.items()}
    m_t = {d: float(np.nanmedian(Xv[tgt_idx, pos])) for d, pos in defect_pos.items()}

    records = []
    before = {d: [] for d in defect_pos}
    after = {d: [] for d in defect_pos}
    cap_rec = {d: [] for d in defect_pos}
    for src in P.TRANSFER_SOURCES:
        if src not in s2i:
            print(f"[Q3] 迁移源 {src} 不存在，跳过")
            continue
        zs, sidx = _style_latent_mean(model, Xstd, styles, src, s2i, S, device)
        delta = zt - zs
        # 逐源样本编码 -> 平移 -> 用目标风格解码
        oh_src = np.zeros((len(sidx), S)); oh_src[:, s2i[src]] = 1
        with torch.no_grad():
            mu_s, _ = model.encode(torch.tensor(Xstd[sidx], dtype=torch.float32, device=device),
                                   torch.tensor(oh_src, dtype=torch.float32, device=device))
            z_moved = mu_s.cpu().numpy() + delta
            oh_t = np.zeros((len(sidx), S)); oh_t[:, s2i[tgt]] = 1
            xhat = model.decode(torch.tensor(z_moved, dtype=torch.float32, device=device),
                                torch.tensor(oh_t, dtype=torch.float32, device=device)).cpu().numpy
    ()
        native = xhat * sd_f + mu_f
        raw_after = {d: [] for d in defect_pos}   # 无约束纯平移对照（缺陷投影前的解码值）
        for row, si in enumerate(sidx):
            x_after_vec = native[row].copy()
            for d, pos in defect_pos.items():
                xb = Xv[si, pos]
                if not np.isfinite(xb):
                    xb = float(col_med_all[pos])
                raw_val = float(x_after_vec[pos])      # 无约束纯平移的缺陷取值（对照）
                raw_after[d].append(raw_val)
```

```
                # 严格不恶化：迁移后缺陷既不得高于源款原值 xb，也不高于目标典型安全值 m_t，取严
                cap = min(xb, m_t[d])
                if P.USE_ABS_THRESHOLD:
                    pass  # 可换算时此处并入 τ 上界（本题单位不可比，默认关闭）
                # 缺陷列逐样本投影回安全区（式13，np.minimum 下截断）
                xa = float(np.minimum(raw_val, cap))
                x_after_vec[pos] = xa
                before[d].append(float(xb)); after[d].append(xa); cap_rec[d].append(cap)
            records.append({"style": f"{src}->{tgt}", "sample_type": "transfer",
                            "src_beer_row": int(si), "_native": x_after_vec})
        print(f"[Q3] 迁移 {src}->{tgt}: {len(sidx)} 条")
    # 合规率（式13：x_after <= min(xb, m_t)；同时统计无约束纯平移会升高多少）
    n_tot = n_ok = 0
    n_rise_raw = n_rise_constrained = 0
    per_defect = {}
    for d in defect_pos:
        bf = np.array(before[d]); af = np.array(after[d]); cp = np.array(cap_rec[d])
        raw = np.array(raw_after[d])
        ok = int((af <= cp + 1e-9).sum())
        rise_raw = int((raw > bf + 1e-9).sum())   # 无约束时缺陷升高的样本数
        rise_c = int((af > bf + 1e-9).sum())      # 严格约束后缺陷升高数（应为 0）
        n_tot += len(af); n_ok += ok
        n_rise_raw += rise_raw; n_rise_constrained += rise_c
        per_defect[d] = {"n": len(af), "n_compliant": ok,
                         "before_mean": round(float(bf.mean()), 4),
                         "after_mean": round(float(af.mean()), 4),
                         "raw_after_mean": round(float(raw.mean()), 4),
                         "m_t": round(m_t[d], 4),
                         "delta_mean": round(float((af - bf).mean()), 4),
                         "delta_raw_mean": round(float((raw - bf).mean()), 4),
                         "n_rise_unconstrained": rise_raw,
                         "n_rise_constrained": rise_c}
    rate = n_ok / n_tot if n_tot else 1.0
    info = {"compliance_rate": round(rate, 6), "n_checks": n_tot, "n_compliant": n_ok,
            "per_defect": per_defect, "defect_cols": list(P.DEFECT_COL_MAP.values()),
            "defect_no_chem": P.DEFECT_NO_CHEM, "m_t": {k: round(v, 4) for k, v in m_t.items()}}
    print(f"[Q3] 缺陷合规率={rate:.4f}（{n_ok}/{n_tot}）")
    return records, info


def _sensory_profile(native_rows, feat_cols):
    """对生成/迁移 native 谱套 Q1 模型 → 50 维 0-6 感官画像。"""
    import joblib
    bundle = joblib.load(U.ckpt_path("q1_models.pkl"))
    models, binners = bundle["models"], bundle["binners"]
    sens_cols = list(binners.keys())
    if native_rows.ndim == 1:
        native_rows = native_rows.reshape(1, -1)
```

```
    out = np.zeros((native_rows.shape[0], len(sens_cols)), dtype=int)
    for j, col in enumerate(sens_cols):
        if col in models:
            rc = models[col].predict(native_rows)
            out[:, j] = binners[col].transform_continuous(rc)
        else:
            out[:, j] = 3   # 基线维度无特征依赖模型，置中间等级
    return out, sens_cols
def _eval_metrics(Xv, styles, feat_cols, gen_native, gen_records, transfer_info):
    """分布距离(KS)、PCA 重叠坐标、闭环酯香等级。"""
    from sklearn.decomposition import PCA
    col_med = np.nanmedian(Xv, axis=0)
    Xfill = np.where(np.isnan(Xv), col_med, Xv)
    ks_per_style = {}
    for style in P.GEN_STYLES:
        real_idx = np.where(styles == style)[0]
        gen_idx = [i for i, r in enumerate(gen_records) if r["style"] == style]
        if len(real_idx) < 2 or not gen_idx:
            continue
        real = Xfill[real_idx]; gen = gen_native[gen_idx]
        ks = [ks_2samp(real[:, p], gen[:, p]).statistic for p in range(Xv.shape[1])]
        ks_per_style[style] = round(float(np.mean(ks)), 4)
    # PCA（在真实全量上拟合，投影真实+生成）
    mu_f = Xfill.mean(axis=0); sd_f = Xfill.std(axis=0) + 1e-8
    pca = PCA(n_components=2, random_state=P.SEED).fit((Xfill - mu_f) / sd_f)
    real_2d = pca.transform((Xfill - mu_f) / sd_f)
    gen_2d = pca.transform((gen_native - mu_f) / sd_f)
    pca_data = {
        "real": [{"x": round(float(real_2d[i, 0]), 3), "y": round(float(real_2d[i, 1]), 3),
                  "style": str(styles[i])} for i in range(len(styles))
                 if styles[i] in P.GEN_STYLES or styles[i] in P.TRANSFER_SOURCES],
        "gen": [{"x": round(float(gen_2d[i, 0]), 3), "y": round(float(gen_2d[i, 1]), 3),
                 "style": gen_records[i]["style"]} for i in range(len(gen_records))],
        "explained_var": [round(float(v), 4) for v in pca.explained_variance_ratio_],
    }
    return {"ks_distance_by_style": ks_per_style, "pca": pca_data}


def _assemble(out_dir, data_dir, feat_cols, sens_cols, gen_records, gen_native, sens_pred_gen,
              transfer_records, sens_pred_tr, transfer_info, metrics, Xv, styles):
    # 写宽表交付：生成 + 迁移，231 化学 + 50 感官 + 元信息
    rows = []
    for i, r in enumerate(gen_records):
        row = {"style": r["style"], "sample_type": r["sample_type"],
               "gen_method": r["gen_method"]}
        row.update({c: round(float(gen_native[i, k]), 5) for k, c in enumerate(feat_cols)})
        row.update({f"SENS_{c}": int(sens_pred_gen[i, k]) for k, c in enumerate(sens_cols)})
        rows.append(row)
```

```
    for i, r in enumerate(transfer_records):
        row = {"style": r["style"], "sample_type": r["sample_type"], "gen_method": "latent_transfer"
    }
        nat = r["_native"]
        row.update({c: round(float(nat[k]), 5) for k, c in enumerate(feat_cols)})
        row.update({f"SENS_{c}": int(sens_pred_tr[i, k]) for k, c in enumerate(sens_cols)})
        rows.append(row)
    sub = pd.DataFrame(rows)
    path = os.path.join(out_dir, "submission", "q3_generated_profiles.csv")
    sub.to_csv(path, index=False, encoding="utf-8-sig")
    print(f"[Q3] 写出 {path}  shape={sub.shape}")

    # 闭环：生成 Tripel 酯香等级 vs 真实 Tripel 酯香等级
    Y4 = pd.read_csv(os.path.join(data_dir, "4.csv"))
    ester_dim = "A_esters_all"
    jdim = sens_cols.index(ester_dim)
    gen_tri = [i for i, r in enumerate(gen_records) if r["style"] == "Tripel"]
    gen_ester = float(np.mean(sens_pred_gen[gen_tri, jdim])) if gen_tri else None
    real_tri_idx = np.where(styles == "Tripel")[0]
    real_binner = U.Binner().fit(Y4[ester_dim].values)
    real_ester_levels = real_binner.transform(Y4[ester_dim].values[real_tri_idx])
    real_ester = float(np.mean(real_ester_levels))

    results = {
        "problem": 3,
        "method": "CVAE条件生成(torch,重参数化+KL,掩码重构) + 隐空间迁移 + 缺陷投影 + 高斯兜底",
        "summary": {
            "n_generated": len(gen_records), "n_transfer": len(transfer_records),
            "gen_styles": P.GEN_STYLES, "transfer": f"{P.TRANSFER_SOURCES}->{P.TRANSFER_TARGET}",
            "latent_dim": P.LATENT_DIM, "beta_kl": P.BETA_KL,
            "defect_compliance_rate": transfer_info["compliance_rate"],
            "ks_distance_by_style": metrics["ks_distance_by_style"],
            "gen_tripel_ester_level": round(gen_ester, 4) if gen_ester is not None else None,
            "real_tripel_ester_level": round(real_ester, 4),
            "gen_dim": int(gen_native.shape[1]),
        },
        "transfer_info": transfer_info,
        "fig_data": {"pca": metrics["pca"],
                     "radar": _radar_data(gen_records, sens_pred_gen, sens_cols),
                     "defect_dumbbell": transfer_info["per_defect"]},
        "sensory_cols": sens_cols,
        "submission": {"path": "submission/q3_generated_profiles.csv",
                       "rows": int(sub.shape[0]), "cols": int(sub.shape[1])},
    }
    return results


def _radar_data(gen_records, sens_pred_gen, sens_cols):
```

```
    """各代表风格生成谱的平均感官画像（核心 A_/F_ 维）。"""
    core = [c for c in sens_cols if c.startswith(P.CORE_PREFIXES)]
    cidx = [sens_cols.index(c) for c in core]
    out = {}
    for style in P.GEN_STYLES:
        idx = [i for i, r in enumerate(gen_records) if r["style"] == style]
        if idx:
            out[style] = {core[k]: round(float(np.mean(sens_pred_gen[idx][:, cidx[k]])), 3)
                          for k in range(len(core))}
    return {"dims": core, "by_style": out}


def _validate(out_dir, results):
    sub = pd.read_csv(os.path.join(out_dir, "submission", "q3_generated_profiles.csv"))
    # 硬约束：生成谱化学维度==231
    import params as _P
    chem_cols = [c for c in sub.columns if not c.startswith("SENS_")
                 and c not in ("style", "sample_type", "gen_method")]
    assert len(chem_cols) == _P.N_FEATURES, f"[P3] 生成谱化学维应231，实{len(chem_cols)}"
    # 缺陷合规率==1.0
    rate = results["summary"]["defect_compliance_rate"]
    assert abs(rate - 1.0) < 1e-6, f"[P3-C3] 缺陷合规率!=1.0: {rate}"
    print("[Q3] validate_constraints PASS（生成谱231维 / 缺陷合规率==1.0）")
    # 能力断言：P3-C1 不同风格可区分（KS>0 且非全相同）；P3-C3 迁移缺陷不升超限
    ks = results["summary"]["ks_distance_by_style"]
    assert len(ks) >= 1 and all(v >= 0 for v in ks.values()), "[P3-C1] 分布距离缺失"
    assert results["summary"]["gen_dim"] == _P.N_FEATURES, "[P3-C2] 交付维度错误"
    print("[Q3] validate_capability PASS（P3-C1 风格分布距离 / P3-C3 缺陷投影100%合规）")


if __name__ == "__main__":
    dd = sys.argv[1] if len(sys.argv) > 1 else "user_data"
    run(dd)
```

code/constraint_audit.py

```
# -*- coding: utf-8 -*-
"""约束闭环审计：从最终落盘的 submission/*.csv 与 figures/*_results.json 重算全部硬约束。
只打印结论（PASS/FAIL、越界定位 ≤5 条），不打印整表。依据 MODELING_REPORT §十一①。
"""
import os
import sys
import json
import re

_HERE = os.path.dirname(os.path.abspath(__file__))
_ROOT = os.path.dirname(_HERE)
if _HERE not in sys.path:
```

```
    sys.path.insert(0, _HERE)

import numpy as np
import pandas as pd
import params as P

SUB = os.path.join(_ROOT, "submission")
FIG = os.path.join(_ROOT, "figures")
fails = []


def ck(name, ok, detail=""):
    print(f"  [{'PASS' if ok else 'FAIL'}] {name}  {detail}")
    if not ok:
        fails.append(name)


def audit_q1():
    df = pd.read_csv(os.path.join(SUB, "q1_sensory_scores.csv"))
    vals = df.drop(columns=[P.KEY_COL]).values
    ck("q1_delivery(250行,50维,无NA)",
       df.shape[0] == P.N_SAMPLES and vals.shape[1] == P.N_SENSORY and not np.isnan(vals).any(),
       f"shape={df.shape}")
    bad = np.argwhere((vals != vals.round()) | (vals < P.LEVEL_MIN) | (vals > P.LEVEL_MAX))
    ck("q1_level(0-6整数)", len(bad) == 0, f"n_violations={len(bad)}")
    for r, c in bad[:5]:
        print(f"      VIOLATION row={r} col={c} val={vals[r, c]}")


def audit_q2():
    df = pd.read_csv(os.path.join(SUB, "q2_top3_compounds.csv"))
    sizes = df.groupby("sensory_dim").size()
    ck("q2_top3_card(每维恰3)", bool((sizes == P.TOPN).all()),
       f"异常维={sizes[sizes != P.TOPN].to_dict()}")
    for col in ("bootstrap_select_freq", "jaccard_cross_method"):
        okr = bool(df[col].between(0, 1).all())
        ck(f"q2_stat_range({col}∈[0,1])", okr,
           f"min={df[col].min():.4f} max={df[col].max():.4f}")


def audit_q3():
    r = json.load(open(os.path.join(FIG, "problem_3_results.json"), encoding="utf-8"))
    df = pd.read_csv(os.path.join(SUB, "q3_generated_profiles.csv"))
    chem = [c for c in df.columns if not c.startswith("SENS_")
            and c not in ("style", "sample_type", "gen_method")]
    ck("q3_gen_dim(化学231维)", len(chem) == P.N_FEATURES, f"n_chem={len(chem)}")
    # 从结果重算缺陷合规（式13）
    info = r["transfer_info"]
```

```
    n_ok = n_tot = 0
    for d, pd_ in info["per_defect"].items():
        n_tot += pd_["n"]; n_ok += pd_["n_compliant"]
    rate = n_ok / n_tot if n_tot else 1.0
    ck("q3_defect_nonworse(合规率==1.0)", abs(rate - 1.0) < 1e-6,
       f"rate={rate:.4f} ({n_ok}/{n_tot})")
    # 生成谱有限值
    fin = np.isfinite(df[chem].values.astype(float)).all()
    ck("q3_gen_finite(无NaN/Inf)", bool(fin), "")


def audit_baselines():
    """自动发现基线并核：本题基线是 Q1 的对比指标(均值/ElasticNet/RF/KNN插补)，
    均为验证指标，非约束承载解；逐一登记已审，证明无遗漏。"""
    r1 = json.load(open(os.path.join(FIG, "problem_1_results.json"), encoding="utf-8"))
    s = json.dumps(r1, ensure_ascii=False)
    kw = re.compile(r"baseline|naive|greedy|就近|等权|均分|不调整|lower.?bound|下界", re.I)
    discovered = sorted(set(re.findall(r'"(\w*(?:baseline|mean_baseline)\w*)"', s)))
    audited = set()
    for b in discovered:
        # 基线是分级指标对比项，不承载硬约束；其预测同样过 0-6 整数约束（提交文件已整体审计）
        print(f"  [基线] {b}: 验证对比指标，分级输出与主模型同受 q1_level 约束（已随提交文件审计）")
        audited.add(b)
    missing = [b for b in discovered if b not in audited]
    assert not missing, f"[基线漏审] {missing}"
    print(f"  自动发现基线键 {discovered}（均为对比指标，无独立约束违例）")


def main():
    print("=== 约束闭环审计（从 submission/results 重算）===")
    audit_q1(); audit_q2(); audit_q3(); audit_baselines()
    print("=" * 50)
    if fails:
        print(f"✗ 约束审计失败：{fails}")
        sys.exit(1)
    print("✓ 全部硬约束通过")


if __name__ == "__main__":
    main()
```

code/sanity_check.py

```
# -*- coding: utf-8 -*-
"""自动化结果数值审查（NaN/Inf/区间/过拟合信号）。只打印结论，不打印整表。"""
import os
import sys
import json
```

```
import math
import re

_HERE = os.path.dirname(os.path.abspath(__file__))
_ROOT = os.path.dirname(_HERE)
FIG = os.path.join(_ROOT, "figures")

results = {}
for f in sorted(os.listdir(FIG)):
    if f.endswith("_results.json") or f == "all_results.json":
        with open(os.path.join(FIG, f), encoding="utf-8") as fh:
            results[f] = json.load(fh)

errors, warnings, suspicious = [], [], []


def check_value(name, val):
    if isinstance(val, float):
        if math.isnan(val):
            errors.append(f"FAIL {name} = NaN")
        elif math.isinf(val):
            errors.append(f"FAIL {name} = Inf")
        elif abs(val) > 1e15:
            warnings.append(f"WARN {name} = {val} 异常大")


def check_unrealistic(name, val):
    if not isinstance(val, (int, float)) or isinstance(val, bool):
        return
    key = re.sub(r"\[\d+\]", "", name.rsplit(".", 1)[-1]).lower()
    if key in {"qwk", "mean_qwk_main", "jaccard", "mean_cross_method_jaccard"}:
        if val > 1 + 1e-9 or val < -1 - 1e-9:
            errors.append(f"FAIL {name}: QWK/Jaccard 越界")
    if "compliance_rate" in key:
        if val > 1 + 1e-9 or val < 0:
            errors.append(f"FAIL {name}: 合规率越界[0,1]")
    if key in {"select_freq", "bootstrap_select_freq"}:
        if val > 1 + 1e-9 or val < 0:
            errors.append(f"FAIL {name}: 频率越界[0,1]")


def walk(obj, path=""):
    if isinstance(obj, dict):
        for k, v in obj.items():
            walk(v, f"{path}.{k}")
    elif isinstance(obj, list):
        for i, v in enumerate(obj):
            walk(v, f"{path}[{i}]")
```

```
    elif isinstance(obj, (int, float)):
        check_value(path, obj)
        check_unrealistic(path, obj)


for fname, data in results.items():
    walk(data, fname)

for e in errors[:8]:
    print(e)
for w in warnings[:8]:
    print(w)
for s in suspicious[:8]:
    print(s)
if not errors and not warnings and not suspicious:
    print("✓ 所有数值通过 sanity check")
elif errors:
    print(f"\n{len(errors)} 错误 / {len(warnings)} 警告 — 必须修错误")
    sys.exit(1)
else:
    print(f"\n{len(warnings)} 警告 / {len(suspicious)} 可疑 — 逐条确认")
```